2）根据设计思想，采用C或C++语言描述算法，关键之处给出注释。

6.4.7 答案与解析

一、单项选择题

01. A

当无向连通图存在权值相同的多条边时，最小生成树可能是不唯一的；另外，这是一个无向连通图，因此最小生成树必定存在，从而选择A。

02. C

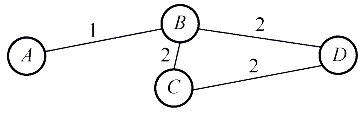
因为无向连通图的最小生成树不一定唯一，所以用不同算法生成的最小生成树可能不同，但当无向连通图的最小生成树唯一时，不同算法生成的最小生成树必定是相同的。

03. A

最小生成树算法是基于贪心策略的，每次总是选取权值最小且满足条件的边，若各边权值不同，则每次选择的新顶点也是唯一的，因此最小生成树唯一，A正确。对于B，若无向图本身就是一棵树，则最小生成树就是它本身，这时就是唯一的。对于C，选取的$n-1$条边可能构成回路。对于D，含有n个顶点、$n-1$条边的子图可能构成回路，也可能不连通。

04. A

若图的边数小于$n-1$，则图不存在最小生成树；若无向连通图的边数等于$n-1$，则最小生成树唯一，即图本身，所以图的边数一定大于$n-1$，A正确。若最小生成树不唯一，则一定存在权值相等的边，但未必是权值最小的边，如下图所示，B错误。最小生成树可能不唯一，但代价一定相同，C错误。当图的各边的权值互不相等时，图的最小生成树是唯一的，D错误。



05. A

$U=\{1,2,3\}$，$V-U=\{4,5,\cdots\}$，候选边只能是这两个顶点集之间的边，只有选项A符合题意。

06. C

若选取边(1, 3)则会构成回路。

07. A

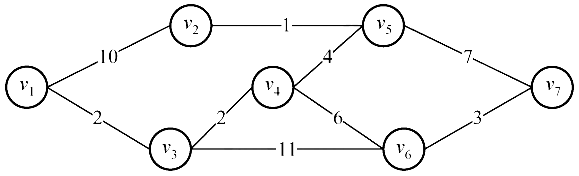
选项A正确，见严蔚敏老师的《数据结构》教材。Dijkstra算法适合求解有回路的带权图的最短路径，也可以求任意两个顶点的最短路径，不适合求带负权值的最短路径问题。在用Floyd算法求两个顶点的最短路径时，当最短路径发生更改时，$path_{k-1}$就不是$path_k$的子集。

08. C

在负权图中，Dijkstra算法既不能保证每次选出的顶点都是真正的最近顶点，又不能保证已确定的最短路径不再被改变，因此Dijkstra算法不允许边的权为负，选项I正确。求每对顶点间的最短路径需要调用Dijkstra算法n次，时间复杂度为$O(n^3)$，选项II错误。Floyd算法求每对顶点间的最短路径允许有负边的存在，但不允许包含总权值为负的回路，选项III正确。

09. B

题目所描述的图G如下图所示。A, B, C, D对应的路径长度分别为18, 13, 15, 24。应用Dijkstra算法不难求出最短路径为$v_1\rightarrow v_3\rightarrow v_4\rightarrow v_6\rightarrow v_7$。



10．D

在 Dijkstra 算法的执行过程中，只可能修改从源点 0 到集合 $V-S$ 中某个顶点的最短路径。

11．A

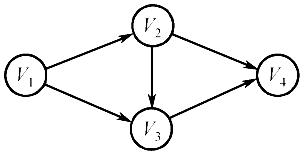
使用深度优先遍历，若从有向图上的某个顶点 u 出发，在 DFS(u)结束之前出现一条从顶点 v 到 u 的边，v 在生成树上是 u 的子孙，图中必定存在包含 u 和 v 的环，因此深度优先遍历可以检测一个有向图是否有环。拓扑排序时，当某顶点不为任何边的头时才能加入序列，存在环时环中的顶点一直是某条边的头，不能加入拓扑序列。也就是说，还存在无法找到下一个可以加入拓扑序列的顶点，则说明此图存在回路。求最短路径是允许图有环的。至于关键路径能否判断一个图有环，则存在一些争议。关键路径本身虽然不允许有环，但求关键路径的算法本身无法判断是否有环，判断是否有环是求关键路径的第一步——拓扑排序。

12．D

若图 G 中存在一条从 v_j 到 v_i 的路径，说明 V_j 是 V_i 的前驱，则要把 V_j 消去以后才能消去 V_i，从而拓扑序列中必然先输出 v_j，再输出 v_i，这显然与题意矛盾。

13．D

对于 I，若有向图中存在环，运行拓扑排序算法后，肯定会剩下有环的子图，在此环中无法再找到入度为 0 的顶点，拓扑排序也就无法再运行。对于 II，若两个顶点之间不存在祖先或子孙关系，则它们在拓扑序列中的前后关系是任意的，因此使用栈和队列都可以，因为入栈或队列的都是入度为 0 的顶点。III 项是难点，若拓扑序列唯一，则很自然联想到一个线性的有向图，下图的拓扑序列也唯一，但却不满足该条件。对于 IV，若入度为 0 的顶点不唯一，则这些顶点均可作为拓扑序列的起点；若出度为 0 的顶点不唯一，则这些顶点均可作为拓扑序列的终点。



14．C

强连通图是指有向图中任意顶点对之间都存在两条相反的路径，这意味着强连通图中一定存在环，因此不能进行拓扑排序，选项 I 正确。假设顶点 a 和 b 的入度均为 0，且分别有两条孤从 a 和 b 指向同一顶点 c，则产生的拓扑序列可以是 abc，但是此时并无一条弧<a, b>，选项 II 错误。如图 1 和 2 所示的有向图对应的拓扑序列都是 $abcd$，且都是唯一的，选项 III 错误。

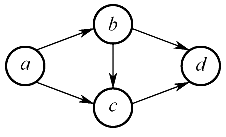


图 1

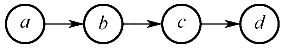


图 2

15．D

一个有向图中的顶点不能排成一个拓扑序列，表明其中存在一个顶点数目大于 1 的回路（环），该回路构成一个强连通分量，从而答案选择选项 D。

16．C

本图的拓扑排序序列有 *ABCFDEG*, *ABCDFEG*, *ABCDEFG*, *ABDCFEG* 和 *ABDCEFG*。读者应能把这一类经典习题的拓扑序列全部写出来。

17．A

拓扑序列的过程：找到入度为 0 的顶点，删除该顶点及其所有出边，并将顶点加入拓扑序列，重复直至所有顶点都加入拓扑序列。选择入度为 0 的顶点 v_1，删除与 v_1 有关的边；此时顶点 v_3 的入度为 0，选择 v_3，删除与 v_3 有关的边；以此类推，得出 G 的拓扑序列。

18．B

无向图的邻接矩阵存储中，每条边存储两次，且 $A[i][j]=A[j][i]$。

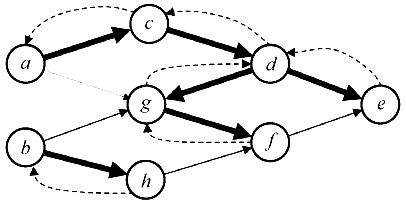
19．C

此题一直以来争议较大，因为有些书中漏掉了“有序”二字。可以证明，对有向图中的顶点适当地编号，使其邻接矩阵为三角矩阵且主对角元素全为零的充分必要条件是，该有向图可以进行拓扑排序。若这个题目把“有序”二字去掉，显然应选择选项 D。但此题题干中已经指出是“有序的拓扑序列”，因此应选选项 C。需要注意的是，若一个有向图的邻接矩阵为三角矩阵（对角线以上或以下的元素为 0），则图中必不存在环，因此其拓扑序列必然存在。

20．A

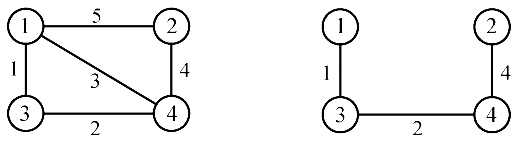
设图中有顶点 v_i，它有后继顶点 v_j，即存在边<v_i, v_j>。根据 DFS 的规则，v_i 入栈后，必先遍历完其后继顶点后 v_i 才会出栈，也就是说 v_i 会在 v_j 之后出栈，在如题所指的过程中，v_i 在 v_j 后打印。v_i 和 v_j 具有任意性，因此由上面的规律看出，输出顶点的序列是逆拓扑有序序列。

对有向无环图利用深度优先搜索进行拓扑排序的例子如下：如下图所示，退出 DFS 栈的顺序为 *efgdcahb*，此图的一个拓扑序列为 *bhacdgfe*。该方法的每一步均是先输出当前无后继的结点，即对每个结点 v，先递归地求出 v 的每个后继的拓扑序列。

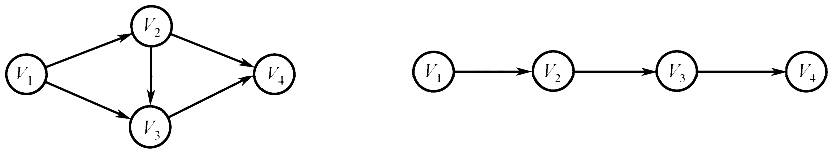


21．C

有向图邻接矩阵的第 V 行中 1 的个数是顶点 V 的出度，而有向图中顶点的度为入度与出度之和，选项 I 错。无向图的邻接矩阵一定是对称矩阵，但当有向图中任意两个顶点之间有边相连，且是两条方向相反的有向边时，有向图的邻接矩阵也是一个对称矩阵，选项 II 错。最小生成树中的 $n-1$ 条边不能保证是图中权值最小的 $n-1$ 条边，因为权值最小的 $n-1$ 条边并不一定能使图连通。在下图中，左图的最小生成树如下图所示，权值为 3 的边不在其最小生成树中。



有向无环图的拓扑序列唯一并不能唯一确定该图。在下图所示的两个有向无环图中，拓扑序列都为 V_1, V_2, V_3, V_4，IV 错。注意，很多辅导书对该命题的判断是错误的。

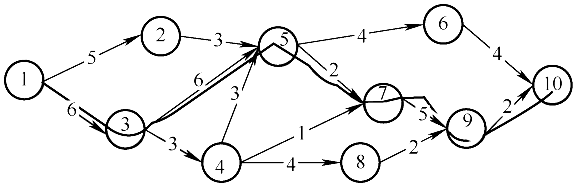


22．C

观察题图，从 V_0 到 V_8 的最长路径为 $V_0 \rightarrow V_1 \rightarrow V_4 \rightarrow V_6 \rightarrow V_8$，长度为 6 + 1 + 9 + 2 = 18。

23．C

题目描述的图如下，得到关键路径的长度为 21，图中画出的两条路径都是关键路径。



24．C

一个事件的最迟发生时间 = min{以该事件为尾的弧的活动的最迟开始时间}，或 min{以该事件为尾的弧所指事件的最迟发生时间与该弧的活动的持续时间之差}。改变 AOE 网中任何一个活动的持续时间，需要重新计算关键活动，可能导致关键路径的改变。

25．C

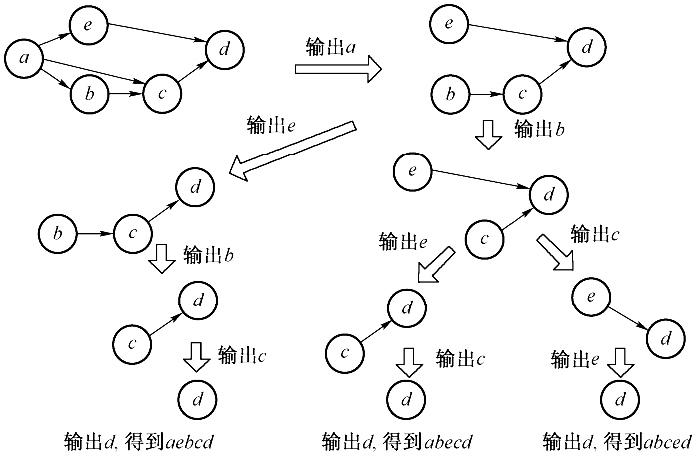
若改变的是所有关键路径上的公共活动，则不一定会产生不同的关键路径（延长必然不会导致，只有缩短才有可能导致）。根据关键路径的定义，可知选项 II 正确。关键路径是源点到终点的最长路径，只有所有关键路径的长度都缩短时，整个图的关键路径才能有效缩短，但也不能任意缩短，一旦缩短到一定程度，该关键活动就可能变成非关键活动。

26．A

邻接矩阵第 i 列值全为∞，说明顶点 i 没有入边，为整个工程的开始，若关键路径存在，则该顶点一定是起点。不能确定关键路径是否存在，也不能确定其对应的无向图的连通分量个数。

27．B

拓扑排序的过程如下图所示。



可以得到 3 种不同的拓扑序列，即 *abced*, *abecd* 和 *aebcd*。

28．A

最小生成树的树形可能不唯一（因为可能存在权值相同的边），但代价一定是唯一的，I 正

确。若权值最小的边有多条并且构成环状，则总有权值最小的边将不出现在某棵最小生成树中，II 错误。设 N 个结点构成环，N 条边的权值相等，从不同的顶点开始执行 Prim 算法，只要选取任意不同的 $N-1$ 条边，就能得到不同的最小生成树，III 错误。当最小生成树唯一时（各边的权值不同），Prim 算法和 Kruskal 算法得到的最小生成树相同，IV 错误。

29．C

从 a 到各顶点的最短路径的求解过程下：

顶点	第 1 轮	第 2 轮	第 3 轮	第 4 轮	第 5 轮
b	(a, b) 2				
c	(a, c) 5	(a, b, c) 3			
d	∞	(a, b, d) 5	(a, b, d) 5	(a, b, d) 5	
e	∞	∞	(a, b, c, e) 7	(a, b, c, e) 7	(a, b, d, e) 6
f	∞	∞	(a, b, c, f) 4		
集合 S	$\{a, b\}$	$\{a, b, c\}$	$\{a, b, c, f\}$	$\{a, b, c, f, d\}$	$\{a, b, c, f, d, e\}$

后续目标顶点依次为 f, d, e。

本题也可用排除法：对于 A，若下一个顶点为 d，路径 a, b, d 的长度为 5，而 a, b, c, f 的长度仅为 4，显然错误。同理可排除 B。将 f 加入集合 S 后，采用上述方法也可排除 D。

30．C

对角线以下元素均为零，表明只有顶点 i 到 j（$i<j$）可能有边，而顶点 j 到 i 一定没有边，即有向图是一个无环图，因此一定存在拓扑序列。对于拓扑序列是否唯一，试举一例：设有向图的邻接矩阵 $\begin{bmatrix}0&1&1\\0&0&0\\0&0&0\end{bmatrix}$，存在两个拓扑序列，因此该图存在可能不唯一的拓扑序列。

结论：对于任一有向图，若它的邻接矩阵中对角线以下（或以上）的元素均为零，则存在拓扑序列（可能不唯一）。反之，若图存在拓扑序列，却不一定能满足邻接矩阵中主对角线以下的元素均为零，但是可以通过适当地调整结点编号，使其邻接矩阵满足前述性质。

31．C

找出 AOE 网的全部关键路径为 *bdcg*、*bdeh* 和 *bfh*。根据性质，只有当所有关键路径的活动时间同时减少时，才能缩短工期。即正确选项中的路径必须能涵盖所有的关键路径。选项 A 和 B 不能涵盖 *bfh* 这条路径，选项 D 不能涵盖 *bdcg* 这条路径，只有选项 C 能涵盖所有关键路径，因此只有加快 f 和 d 的进度才能缩短工期（建议在图中检验）。

32．D

按照拓扑排序的算法，每次都选择入度为 0 的结点从图中删除，此图中一开始只有结点 3 的入度为 0；删除结点 3 后，只有结点 1 的入度为 0；删除结点 1 后，只有结点 4 的入度为 0；删除结点 4 后，结点 2 和结点 6 的入度都为 0，此时选择删除不同的结点，会得出不同的拓扑序列，分别处理完毕后可知可能的拓扑序列为 3, 1, 4, 2, 6, 5 和 3, 1, 4, 6, 2, 5。

33．C

从 V_4 开始，Kruskal 算法选中的第一条边一定是权值最小的(V_1, V_4)，B 错误。V_1 和 V_4 已经可达，因此含有 V_1 和 V_4 的权值为 8 的第二条边一定符合 Prim 算法，排除 A 和 D。

34．C

第一个顶点和最后一个顶点相同的路径称为回路；序列中顶点不重复出现的路径称为简单路径；回路显然不是简单路径，选项 I 错误。稀疏图是边比较少的情况，邻接矩阵存储的空间复杂度为 $O(n^2)$，必将浪费大量的空间，而邻接表存储的空间复杂度为 $O(n+e)$，所以应选用邻接

表，选项 II 错误。存在回路的有向图不存在拓扑序列，若拓扑排序输出结束后所余下的顶点都有前驱，则说明只得到了部分顶点的拓扑有序序列，图中存在回路，选项 III 正确。

35．B　公众号：小兔网盘 免费分享无水印PDF

根据 Dijkstra 算法，从顶点 1 到其余各顶点的最短路径如下表所示。

顶　点	第 1 轮	第 2 轮	第 3 轮	第 4 轮	第 5 轮
2	5 $v_1 \rightarrow v_2$	5 $v_1 \rightarrow v_2$			
3	∞	∞	7 $v_1 \rightarrow v_2 \rightarrow v_3$		
4	∞	11 $v_1 \rightarrow v_5 \rightarrow v_4$	11 $v_1 \rightarrow v_5 \rightarrow v_4$	11 $v_1 \rightarrow v_5 \rightarrow v_4$	11 $v_1 \rightarrow v_5 \rightarrow v_4$
5	4 $v_1 \rightarrow v_5$				
6	∞	9 $v_1 \rightarrow v_5 \rightarrow v_6$	9 $v_1 \rightarrow v_5 \rightarrow v_6$	9 $v_1 \rightarrow v_5 \rightarrow v_6$	
集合 S	{1, 5}	{1, 5, 2}	{1, 5, 2, 3}	{1, 5, 2, 3, 6}	{1, 5, 2, 3, 6, 4}

快速解法。依次观察从顶点 1 到其他顶点的最短路径长度：顶点 1 到顶点 2 的最短路径长度为 5；顶点 1 到顶点 3 的最短路径长度为 5 + 2 = 7；顶点 1 到顶点 4 的最短路径长度为 11；顶点 1 到顶点 5 的最短路径长度为 4；顶点 1 到顶点 6 的最短路径长度为 4 + 5 = 9；最终 `dist[]`={0, 5, 7, 11, 4, 9}，根据 `dist` 数组值从小到大选择顶点顺序为 1, 5, 2, 3, 6, 4。

36．B

采用邻接表作为 AOV 网的存储结构进行拓扑排序，需要对 n 个顶点做入栈、出栈、输出各一次，当处理 e 条边时，需要检测这 n 个顶点的边链表结点，共需要的时间为 $O(n+e)$。若采用邻接矩阵作为 AOV 网的存储结构进行拓扑排序，在处理 e 条边时需对每个顶点检测相应矩阵中的某一行，寻找与它相关联的边，以便对这些顶点的入度减 1，需要的时间代价为 $O(n^2)$。

【补充】有两种常用的拓扑排序算法：基于 BFS 的算法和基于 DFS 的算法。本题未指明采用哪种算法，因此只需验证一种算法即可（说明两种算法在对应条件下的时间复杂度相同）。

基于 BFS 的算法的思想：首先找到所有入度为 0 的结点，将它们加入一个队列，并将它们作为拓扑序列的起始部分；然后依次从队列中取出结点，并删除它们与后继结点的所有边。若某个后继结点的入度变为 0，则将它也加入队列，并将它加入拓扑序列，重复这个过程。

基于 DFS 的算法的思想：在 DFS 调用过程中设定一个时间标记，当 DFS 调用结束时，对各结点计时，进而按结束时间从大到小排序，可以得到一个拓扑序列。

37．D

拓扑排序每次选取入度为 0 的结点输出，经观察不难发现拓扑序列前两位一定是 1, 5 或 5, 1（因为只有 1 和 5 的入度均为 0，且其他结点都不满足仅有 1 或仅有 5 作为前驱）。

38．C

活动 d 的最早开始时间等于该活动弧的起点所表示的事件的最早发生时间，活动 d 的最早开始时间等于事件 2 的最早发生时间 $\max\{a, b+c\} = \max\{3, 12\} = 12$。活动 d 的最迟开始时间等于该活动弧的终点所表示的事件的最迟发生时间与该活动所需时间之差，先算出图中关键路径长度为 27（对于不复杂的选择题，找出所有路径计算长度），那么事件 4 的最迟发生时间为 $\min\{27-g\} = \min\{27-6\} = 21$，活动 d 的最迟开始时间为 $21-d = 21-7 = 14$。

常规方法：按照关键路径算法算得到下表。

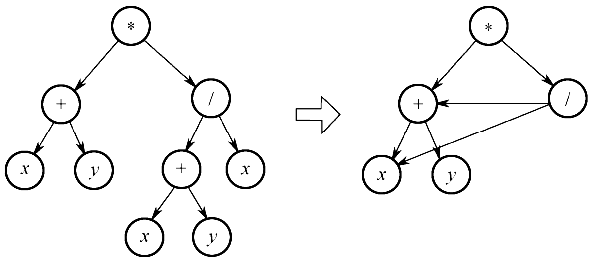
	v_1	v_2	v_3	v_4	v_5	v_6
$v_e(i)$	0	12	8	19	18	27
$v_l(i)$	0	12	8	21	18	27

	a	b	c	d	e	f	g	h
$e(i)$	0	8	0	12	12	8	19	18
$l(i)$	9	8	0	14	12	8	21	18
$l(i)-e(i)$	9	0	0	2	0	0	2	0

从表中可知，活动 d 的最早开始时间和最迟开始时间分别为 12 和 14，所以选择选项 C。

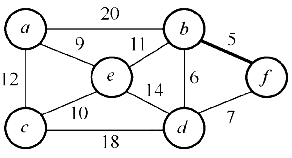
39．A

先将该表达式转换成有向二叉树，该二叉树中有些顶点是重复的，为了节省存储空间，去除重复的顶点，将有向二叉树去重转换成有向无环图，如下图所示。

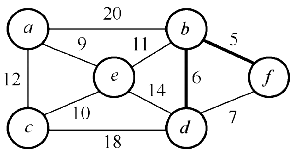


40．A

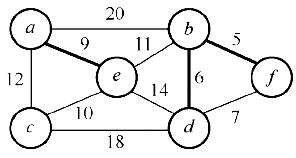
Kruskal 算法：按权值递增顺序依次选取 $n-1$ 条边，并保证这 $n-1$ 条边不构成回路。初始构造一个仅含 n 个顶点的森林；第一步，选取权值最小的边(b, f)加入最小生成树；第二步，剩余边中权值最小的边为(b, d)，加入最小生成树，第二步操作后权值最小的边(d, f)不能选，因为会与之前已选取的边形成回路；接下来依次选取权值 9、10、11 对应的边加入最小生成树，此时 6 个顶点形成了一棵树，最小生成树构造完成。按照上述过程，加到最小生成树的边依次为(b, f), (b, d), (a, e), (c, e), (b, e)。其生成过程如下所示。



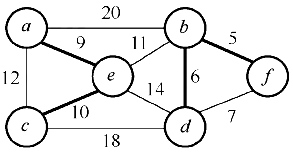
第一步
选取边< b, f >



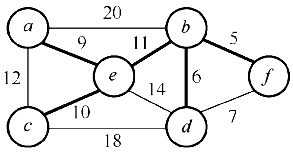
第二步
选取边< b, d >



第三步
选取边< a, e >



第四步
选取边< c, e >



第五步
选取边< b, e >

41．B

根据题干所提供的信息可知：

①图 G 为有向无环图，因此一定存在拓扑序列和逆拓扑序列。

②DFS 的性质是顶点 v_i 的所有后继顶点 v_j 出栈后，v_i 才会出栈。

③本题要求执行输出语句后立刻退出递归，即执行完输出语句后立即出栈，因此后入栈的

顶点先输出，结合②不难得出只有输出顶点 v_i 的所有后继顶点 v_j 后，v_i 才会输出。

综合上述分析，输出的顶点序列是逆拓扑有序序列。

42．B

关键路径是指权值之和最大而非边数最多的路径，选项 A 错误。选项 B 是关键路径的概念。无论是存在一条还是存在多条关键路径，增加任意一个关键活动的时间都会延长工程的工期，因为关键路径始终是权值之和最大的那条路径，选项 C 错误。仅有一条关键路径时，减少关键活动的时间会缩短工程的工期；存在多条关键路径时，缩短一条关键活动的时间不一定会缩短工程的工期，缩短了路径长度的那条关键路径不一定还是关键路径，选项 D 错误。

43．A

求拓扑序列的过程：从图中选择无入边的结点，输出该结点并删除该结点的所有出边，重复上述过程，直至全部结点都已输出，这样求得的拓扑序列为 *ABCDEF*。每次输出一个结点并删除该结点的所有出边后，都发现有且仅有一个结点无入边，因此该拓扑序列唯一。

44．C

在执行 Dijkstra 算法时，首先初始化 dist[]，若顶点 1 到顶点 i（i = 2, 3, 4, 5）有边，就初始化为边的权值；若无边，就初始化为∞；初始化顶点集 S 只含顶点 1。Dijkstra 算法每次选择一个到顶点 1 距离最近的顶点 j 加入顶点集 S，并判断由顶点 1 绕行顶点 j 后到任意一个顶点 k 是否距离更短，若距离更短（dist[j] + arcs[j][k] < dist[k]），则将 dist[x]更新为 dist[j] + arcs[j][k]；重复该过程，直至所有顶点都加入顶点集 S。数组 dist 的变化过程如下图所示，可知将第二个顶点 5 加入顶点集 S 后，数组 dist 更新为 21, 3, 14, 6。

$$\text{dist}\{26,3,\infty,6\} \xrightarrow{\text{顶点3入S}} \{25,3,\infty,6\} \xrightarrow{\text{顶点5入S}} \{21,3,14,6\}$$

45．B

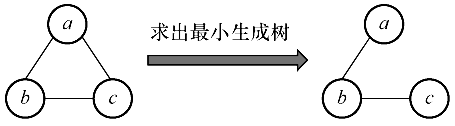
在 AOE 网中，活动的时间余量 = 结束顶点的最迟开始时间 – 开始顶点的最早开始时间 – 该活动的持续时间。根据关键路径算法得到下表：

结点编号	1	2	3	4	5	6
最早开始时间 $v_e(i)$	0	2	5	8	9	12
最迟开始时间 $v_l(i)$	0	4	5	8	11	12

c 的时间余量 = $v_l(3) - v_e(2) - 1 = 5 - 2 - 1 = 2$，$g$ 的时间余量 = $v_l(6) - v_e(3) - 1 = 12 - 5 - 1 = 6$，$h$ 的时间余量 = $v_l(5) - v_e(4) - 1 = 11 - 8 - 1 = 2$，$j$ 的时间余量 = $v_l(6) - v_e(5) - 1 = 12 - 9 - 1 = 2$。

46．B

Prim 算法和 Kruskal 算法用于求解最小生成树，最小生成树中某顶点到其余各顶点的路径不一定具有最短路径的性质。例如，在下图所求得的最小生成树中，a 到 c 的路径长度为 2，但原图中 a 到 c 的最短路径长度为 1。



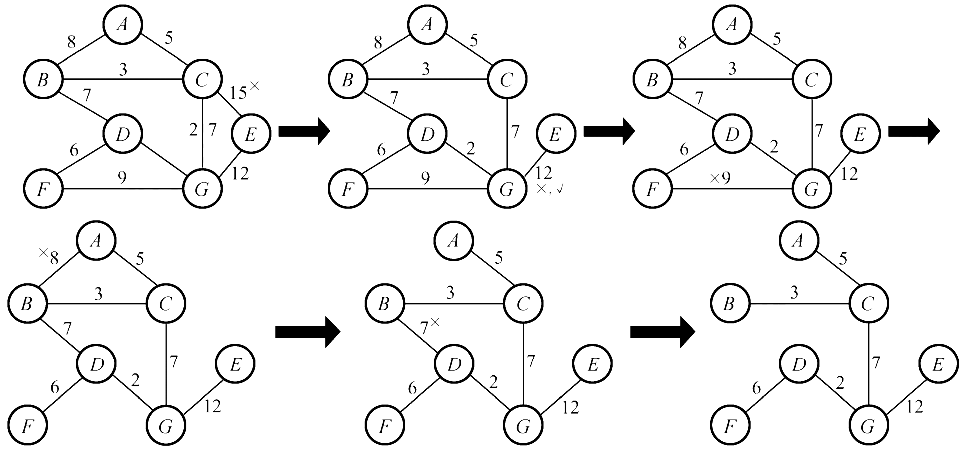
图的广度优先搜索算法总按距离由近到远来遍历图中的每个顶点，因此可用来求解非带权图（或各边权值均相同）的单源最短路径问题。

二、综合应用题

01．【解答】

这种方法是正确的。

经过“破圈法”之后，最终没有回路，因此一定可以构造出一棵生成树。下面证明这棵生成树是最小生成树。记“破圈法”生成的树为 T，假设 T 不是最小生成树，则必然存在最小生成树 T_0，使得它与 T 的公共边尽可能多，则将 T_0 与 T 取并集，得到一个图，此图中必然存在回路，“破圈法”的定义就是从回路中去除权最大的边，因此此时生成的 T 的权必然是最小的，这与原假设矛盾，从而 T 是最小生成树。下图说明了“破圈法”的过程：



02.【解答】

1）该图的邻接矩阵为

$$A=\begin{array}{c} \\ 1\\2\\3\\4\\5\\6\\7 \end{array}\begin{array}{c} \begin{array}{ccccccc} 1 & 2 & 3 & 4 & 5 & 6 & 7 \end{array} \\ \begin{bmatrix} 0 & 3 & 3 & 6 & \infty & \infty & \infty \\ \infty & 0 & 4 & \infty & 5 & \infty & \infty \\ \infty & \infty & 0 & \infty & 4 & \infty & \infty \\ \infty & \infty & \infty & 0 & \infty & 5 & \infty \\ \infty & \infty & \infty & \infty & 0 & \infty & 3 \\ \infty & \infty & 3 & \infty & \infty & 0 & 7 \\ \infty & \infty & \infty & \infty & \infty & \infty & 0 \end{bmatrix} \end{array}$$

得到的深度优先遍历序列为 1, 2, 3, 5, 7, 4, 6。

2）解题思路：当某个顶点只有出弧而没有入弧时，其他顶点无法到达这个顶点，不可能与其他顶点和边构成强连通分量（这个单独的顶点构成一个强连通分量）。

① 顶点 1 无入弧构成第一个强连通分量。删除顶点 1 及所有以之为尾的弧。

② 顶点 2 无入弧构成一个强连通分量。删除顶点 2 及所有以之为尾的弧。

③ ……

以此类推，最后得到每个顶点都是一个强连通分量，所以强连通分量数目为 7。

3）该图的两个拓扑序列如下：

① 1, 2, 4, 6, 3, 5, 7

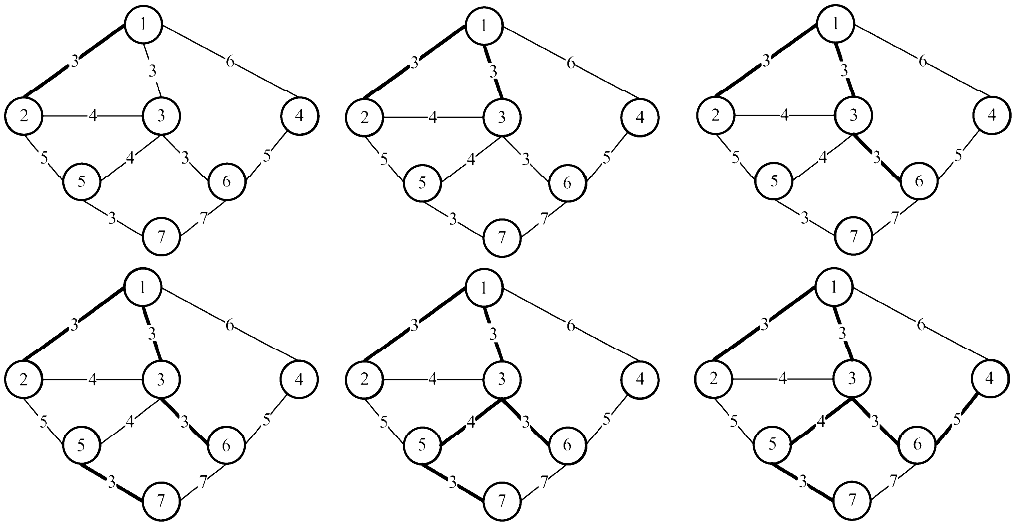
② 1, 4, 2, 6, 3, 5, 7

4）若视该图为无向图：

用 Prim 算法生成最小生成树的过程如下：

1—2，1—3，3—6，3—5，5—7，6—4（图略）。

用 Kruskal 算法生成最小生成树的过程如下图所示。



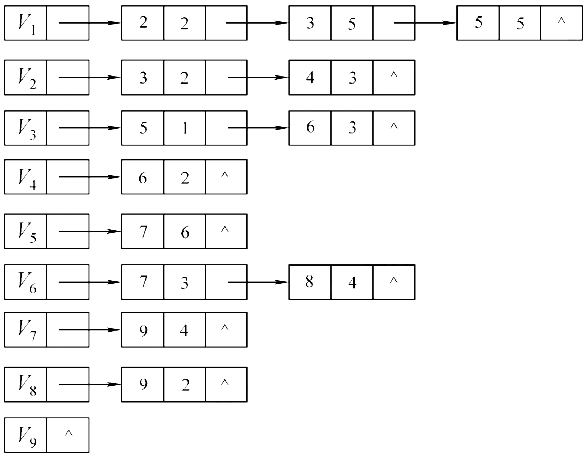
03.【解答】

根据 Dijkstra 算法，求从顶点 1 到其余各顶点的最短路径如下表所示。

顶点	从顶点 1 到各终点的 dist 值				
	第 1 轮	第 2 轮	第 3 轮	第 4 轮	
2	**7** **v_1, v_2**				
3	11 v_1, v_3	**11** **v_1, v_3**			
4	∞	16 v_1, v_2, v_4	**16** **v_1, v_2, v_4**		
5	∞	∞	18 v_1, v_3, v_5	**18** **v_1, v_3, v_5**	
6	∞	∞	19 v_1, v_3, v_6	19 v_1, v_3, v_6	**19** **v_1, v_3, v_6**
集合 S	**{1, 2}**	**{1, 2, 3}**	**{1, 2, 3, 4}**	**{1, 2, 3, 4, 5}**	**{1, 2, 3, 4, 5, 6}**

04.【解答】

1）该图的邻接表表示如下图所示。



求关键路径的算法如下：

① 输入 e 条弧<j, k>，建立 AOE 网的存储结构。

② 从源点 v_1 出发，令 $v_e(1)=0$，求 $v_e(j)$，$2\leqslant j\leqslant n$。

③ 从汇点 v_n 出发，令 $v_l(n)=v_e(n)$，求 $v_l(i)$，$1\leqslant i\leqslant n-1$。

④ 根据各顶点的 v_e 和 v_1 值，求每条弧 s（活动）的最早开始时间 $e(s)$和最晚开始时间 $l(s)$，其中 $e(s)=l(s)$为关键活动。

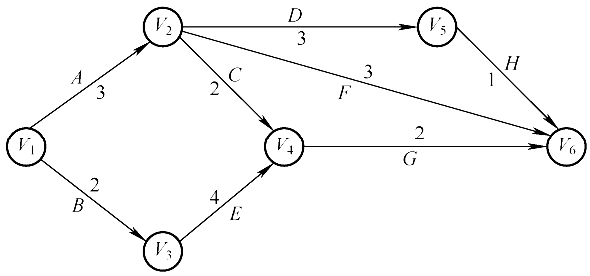
2）根据以上算法可以得到至少需要时间 16。

3）关键路径为(V_1, V_3, V_5, V_7, V_9)。

4）活动 a_2, a_6, a_9, a_{12} 加速，可以缩短工程所需的时间。

05.【解答】

1）根据题表可以画出 AOE 网如下图所示。



求解各事件和活动的最早发生时间与最迟发生时间公式分别如下：

① v_e(源点)=0，$v_e(k)=\text{Max}\{v_e(j)+\text{Weight}(v_j, v_k)\}$，$\text{Weight}(v_j, v_k)$表示从 v_j 指向 v_k 的弧的权值。

② v_l(汇点)=v_e(汇点)，$v_l(j)=\text{Min}\{v_l(k)-\text{Weight}(v_j, v_k)\}$，$\text{Weight}(v_j, v_k)$表示从 v_j 指向 v_k 的弧的权值。

③ 若边<v_k, v_j>表示活动 a_i，则有 $e(i)=v_e(k)$。

④ 若边<v_k, v_j>表示活动 a_i，则有 $l(i)=v_l(j)-\text{Weight}(v_k, v_j)$。

⑤ $d(i)=l(i)-e(i)$。

关键路径即由 $d(i)=0$ 的 i 构成。

2）根据上述公式，各事件的最早发生时间 v_e 和最迟发生时间 v_l 如下表所示。

	v_1	v_2	v_3	v_4	v_5	v_6
$v_e(i)$	0	3	2	6	6	8
$v_l(i)$	0	4	2	6	7	8

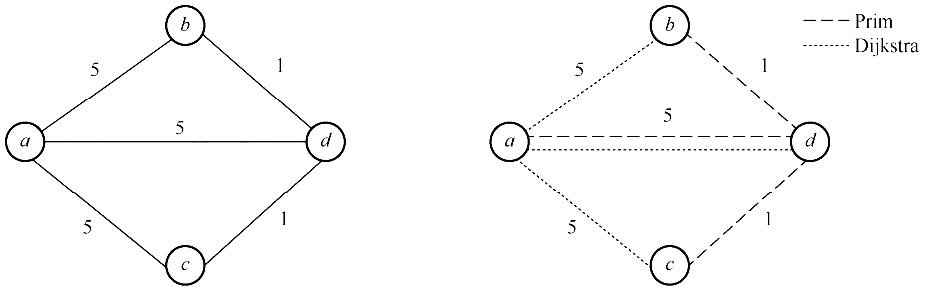
3）根据上述公式，各活动最早发生时间 e、最迟发生时间 l 和时间余量 $d(i)=l(i)-e(i)$如下表所示。

	A	B	C	D	E	F	G	H
$e(i)$	0	0	3	3	2	3	6	6
$l(i)$	1	0	4	4	2	5	6	7
$l(i)-e(i)$	1	0	1	1	0	2	0	1

所以关键路径为 B、E、G，完成该工程最少需要 8（单位依题意而定）。

06.【解答】

Dijkstra 算法每一步都会贪婪地选择与源点 v_0 最近的下一条边，直到 v_0 连接到图中所有顶点。Prim 算法（已知是最小生成树算法）与 Dijkstra 算法高度相似，但是在每个阶段，它贪婪地选择与该阶段已加入 MST 中任意一个顶点最近的下一条边。显然，Dijkstra 算法可以产生一棵生成树，但该树不一定是最小生成树，只需举出一个反例即可，以下图 G 为例（将 a 作为源点）。



(a) 图 G　　(b) 两种算法产生的生成树

Dijkstra 算法得到的路径集合为$\{(a, b), (a, c), (a, d)\}$，该生成树的总权值为 5 + 5 + 5 = 15。

Prim 算法得到的边集合为$\{(a, d), (b, d), (c, d)\}$，该最小生成树的总权值为 5 +1 + 1 = 7。

显然，Dijkstra 算法得到的生成树不一定是最小生成树。

07.【解答】

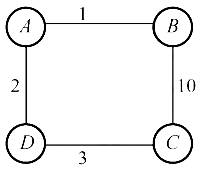
本节前面给出了 DFS 实现拓扑排序的思想，下面是利用 DFS 求各顶点结束时间的代码（在 DFS 的基础上加入了 time 变量）。将结束时间从大到小排序，即可得到拓扑序列。

```
bool visited[MAX_VERTEX_NUM];          //访问标记数组
void DFSTraverse(Graph G){
    for(v=0;v<G.vexnum;++v)
        visited[v]=FALSE;              //初始化访问标记数组
    time=0;
    for(v=0;v<G.vexnum;++v)            //本代码从 v=0 开始遍历
        if(!visited[v]) DFS(G,v);
}
void DFS(Graph G,int v){
    visited[v]=TRUE;
    visit(v);
    for(w=FirstNeighbor(G,v);w>=0;w=NextNeighbor(G,v,w))
        if(!visited[w]){               //w 为 v 的尚未访问的邻接点
            DFS(G,w);
        }
    time=time+1;finishTime[v]=time;
}
```

08.【解答】

该方法不一定能（或不能）求得最短路径。

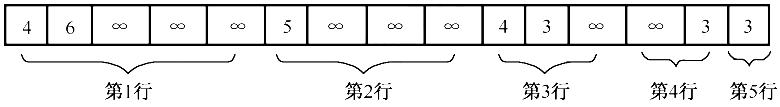
例如，对于下图所示的带权图，若按照题中的原则，从 A 到 C 的最短路径是 $A\rightarrow B\rightarrow C$，事实上其最短路径是 $A\rightarrow D\rightarrow C$。



09.【解答】

1）在上三角矩阵 A[6][6]中，第 1 行至第 5 行主对角线上方的元素个数分别为 5, 4, 3, 2, 1，

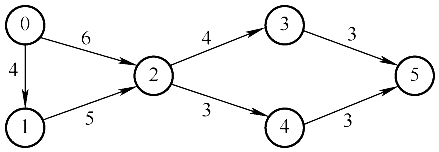
由此可以画出压缩存储数组中的元素所属行的情况，如下图所示。



可以用“平移”的思想，将前5个、后4个、后3个、后2个、后1个元素，分别移动到矩阵对角线（“0”）右边的行上。图 G 的邻接矩阵 $\boldsymbol{A}$ 为

$$\boldsymbol{A}=\begin{bmatrix}0 & 4 & 6 & \infty & \infty & \infty\\ \infty & 0 & 5 & \infty & \infty & \infty\\ \infty & \infty & 0 & 4 & 3 & \infty\\ \infty & \infty & \infty & 0 & \infty & 3\\ \infty & \infty & \infty & \infty & 0 & 3\\ \infty & \infty & \infty & \infty & \infty & 0\end{bmatrix}$$

2）有向带权图 G 如下图所示。



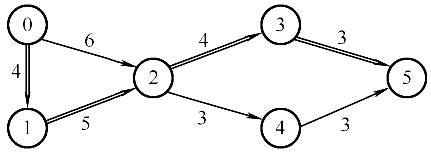
3）先计算各个事件的最早发生时间，得到 $v_e()$和 $v_l()$数组如下表所示。

i	0	1	2	3	4	5
$v_e(i)$	0	4	9	13	12	16
$v_l(i)$	0	4	9	13	13	16

接下来计算所有活动的最早和最迟发生时间 $e()$和 $l()$，如下表所示。

	$a_{0\to1}$	$a_{0\to2}$	$a_{1\to2}$	$a_{2\to3}$	$a_{2\to4}$	$a_{3\to5}$	$a_{4\to5}$
$e()$	0	0	4	9	9	13	12
$l()$	0	3	4	9	10	13	13
$l()-e()$	0	3	0	0	1	0	1

满足 $l()-e()=0$ 的路径就是关键路径，所以关键路径为 $a_{0\to1}$, $a_{1\to2}$, $a_{2\to3}$, $a_{3\to5}$，如下图所示（双线箭头表示），长度为 $4+5+4+3=16$。



按求关键路径的公式计算较为复杂，建议考生面临此类题时直接穷举各条路径即可。

10.【解答】

本题初看起来感觉难度较大，但仔细分析就可发现考查的其实是邻接表的数据结构。

1）图题中给出的是一个简单的网络拓扑图，可以抽象为无向图。

2）图的常用存储结构有邻接矩阵法和邻接表法，其中邻接表法属于链式存储结构，因此本题的基本思路就是写出邻接表的数据类型定义，并根据题意调整相应的边表结点和顶点表结点的成员变量。具体分析如下：邻接表由表头顶点和弧顶点组成。根据题目给出的图和表，可将顶点分为三类：路由器、网络和链路。路由器是连接网络和链路的载体，因此可将它作为表头顶点。网络和链路则是连接路由器的边，因此可将它们作为弧顶

点。为了简化代码，可将网络和链路的结构合并为一个，用一个标志位来区分它们，这样就可用邻接表来实现图的存储。链式存储结构如下图所示。

弧结点的两种基本形态

Flag=1	Next
ID	
IP	
Metric	

Flag=2	Next
Prefix	
Mask	
Metric	

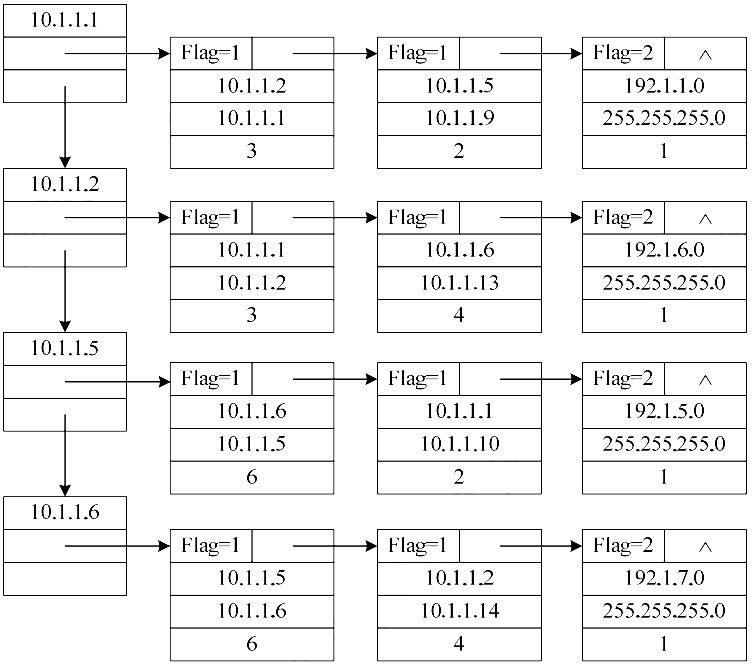
表头结点结构示意

RouterID
LN_link
Next

其数据类型定义如下：

```
typedef struct{
unsigned int ID, IP;
}LinkNode;                  //Link 的结构
typedef struct{
unsigned int Prefix, Mask;
}NetNode;                   //Net 的结构
typedef struct Node{
int Flag;                   //Flag=1 为 Link;Flag=2 为 Net
union{
    LinkNode Lnode;
    NetNode Nnode
}LinkORNet;
Unsigned int Metric;
struct Node *next;
}ArcNode;                   //弧结点
typedef struct hNode{
unsigned int RouterID;
ArcNode *LN_link;
Struct hNode *next;
}HNODE;                     //表头结点
```

对应表的链式存储结构示意图如下所示。



3）计算结果如下表所示。

	目的网络	路　径	代价（费用）
步骤1	192.1.1.0/24	直接到达	1
步骤2	192.1.5.0/24	R1→R3→192.1.5.0/24	3
步骤3	192.1.6.0/24	R1→R2→192.1.6.0/24	4
步骤4	192.1.7.0/24	R1→R2→R4→192.1.7.0/24	8

11.【解答】

1）Prim 算法属于贪心策略。算法从一个任意的顶点开始，一直长大到覆盖图中的所有顶点为止。算法的每一步在连接树集合 S 的顶点和其他顶点的边中，选择一条使得树的总权重增加最小的边加入集合 S。当算法终止时，S 就是最小生成树。

① S 中顶点为 A，候选边为$(A, D), (A, B), (A, E)$，选择(A, D)加入 S。

② S 中顶点为 A, D，候选边为$(A, B), (A, E), (D, E), (C, D)$，选择$(D, E)$，加入 S。

③ S 中顶点为 A, D, E，候选边为$(A, B), (C, D), (C, E)$，选择(C, E)加入 S。

④ S 中顶点为 A, D, E, C，候选边为$(A, B), (B, C)$，选择(B, C)加入 S。

⑤ S 就是最小生成树。

依次选出的边为

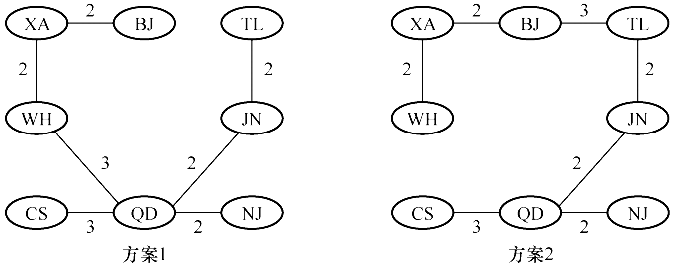
$$(A, D), (D, E), (C, E), (B, C)$$

2）图 G 的 MST 是唯一的。第一小题的最小生成树包括了图中权值最小的 4 条边，其他边除(A, E)外都比这 4 条边大，但若用(A, E)替换同权值的(C, E)，A, D, E 三个顶点构成了回路，因此不能替换，所以此图的 MST 唯一。

3）当带权连通图的任意一个环中所包含的边的权值均不相同时，其 MST 是唯一的。此题不要求回答充分必要条件，所以回答一个限制边权值的充分条件即可。

12.【解答】

1）为了求解最经济的方案，可把问题抽象为求无向带权图的最小生成树。可以采用手动 Prim 算法或 Kruskal 算法作图。注意本题的最小生成树有两种构造，如下图所示。



方案的总费用为 16。

2）存储题中的图可采用邻接矩阵（或邻接表）。构造最小生成树采用 Prim 算法（或 Kruskal 算法）。

3）TTL = 5，即 IP 分组的生存时间（最大传递距离）为 5，方案 1 中 TL 和 BJ 的距离过远，TTL = 5 不足以让 IP 分组从 H1 传送到 H2，因此 H2 不能收到 IP 分组。而方案 2 中 TL 和 BJ 邻近，H2 可以收到 IP 分组。

13.【解答】

1）算法的基本设计思想：建立图 G 各顶点的入度表 `degree[]`。选择入度为 0 的顶点 v，将 v 的所有邻接点的入度减 1，重复执行这个过程。若每次选中的入度为 0 的顶点有且仅有一个，且共进行了 `G.numVertices` 次，则意味着存在唯一的拓扑序列，返回 1，否则不存在拓扑序列，或者存在多个拓扑序列，返回 0。

2）用 C 语言描述的算法如下：

```
int uniquely(MGraph G){//判定有向图是否存在唯一的拓扑序列
    int *degree,i,j,count=0,in0=-1,prev_in0;
    degree=(int*)malloc(G.numVertices*sizeof(int));
    for(j=0;j<G.numVertices;j++){//计算各顶点的入度
        degree[j]=0;
        for(i=0;i<G.numVertices;i++)
            degree[j]+=G.Edge[i][j];
        if(degree[j]==0){
            if(in0==-1) in0=j;          //入度为 0 的顶点
            else in0=-2;                //有多个入度为 0 的顶点
        }
    }
    while(in0>=0){
        count++;
        prev_in0=in0;
        in0=-1;
        for(j=0;j<G.numVertices;j++)
            if(G.Edge[prev_in0][j]>0){
                if(--degree[j]==0){     //邻接点入度值减 1
                    if(in0==-1) in0=j;  //入度为 0 的顶点
                    else in0=-2;        //有多个入度为 0 的顶点
                }
            }
    }
    free(degree);
    if(count==G.numVertices) return 1;
    else return 0;
}
```

归纳总结

1．关于图的基本操作

本章中的很多程序对采用邻接表或邻接矩阵的存储结构都适用，主要原因是在图的基本操作函数中保持了相同的参数和返回值，而封闭了内部实现细节。

例如，取 x 邻接顶点 y 的下一个邻接顶点的函数 NextNeighbor(G,x,y)。

1）用邻接矩阵作为存储结构

```
int NextNeighbor(MGraph& G,int x,int y){
    if(x!=-1 && y!=-1){
        for(int col=y+1;col<G.vexnum;col++)
            if(G.Edge[x][col]>0 && G.Edge[x][col]<maxWeight)
return col;             //maxWeight 代表∞
    }
    return -1;
}
```

2）用邻接表作为存储结构

```
int NextNeighbor(ALGraph& G,int x,int y){
    if(x!=-1){                                  //顶点 x 存在
        ArcNode *p=G.vertices[x].first;         //对应边链表第一个边结点
        while(p!=NULL && p->data!=y)            //寻找邻接顶点 y
            p=p->next;
        if(p!=NULL && p->next!=NULL)
```

```
                return p->next->data;          //返回下一个邻接顶点
            }
            return -1;
        }
```

2. 关于图的遍历、连通性、生成树、关键路径的几个要点

1）在执行图的遍历时，因为图中可能存在回路，且图的任意一个顶点都可能与其他顶点相连，所以在访问完某个顶点后可能会沿某些边又回到了曾经访问过的顶点。因此，需要设置一个辅助数组visited[]标记顶点是否已被访问过，避免重复访问。

2）深度优先搜索时利用回溯法对图遍历，一般利用递归方法实现，每当向前递归查找某一邻接结点之前，必须判断该结点是否访问过。另外，递归算法均可借助栈来实现非递归算法，深度优先搜索也不例外，具体程序见6.3.4节的综合应用题03。

3）广度优先搜索是一种分层的遍历过程，每向前走一步可能访问一批顶点，不像深度优先搜索那样有回退的情况。因此，它不是一个递归的过程。

4）一个给定的图的邻接矩阵表示是唯一的，但对于邻接表来说，若边的输入先后次序不同，则生成的邻接表表示也不同。

5）图的最小生成树首先必须是带权连通图，其次要在 n 个顶点的图中选择 $n-1$ 条边将其连通，使得其权值总和达到最小，且不出现回路。

6）加速某一关键活动不一定能缩短整个工程的工期，因为AOE网中可能存在多条关键路径。可能存在称为桥的一种特殊关键活动，它位于所有的关键路径上，只有它加速才会缩短整个工期。

思维拓展

【网易有道笔试题】求一个无向连通图的割点。割点的定义是，若除去此结点和与其相关的边，无向图不再连通，描述算法。

提　示

要判断一个点是否为割点，最简单直接的方法是，先把这个点和所有与它相关的边从图中去掉，再用深搜或广搜来判断剩下的图的连通性，这种方法适合判断给定结点是否为割点；还有一种比较复杂的方法可以快速找出所有割点，有兴趣的读者可自行搜索相关资料。

第 7 章

查　找

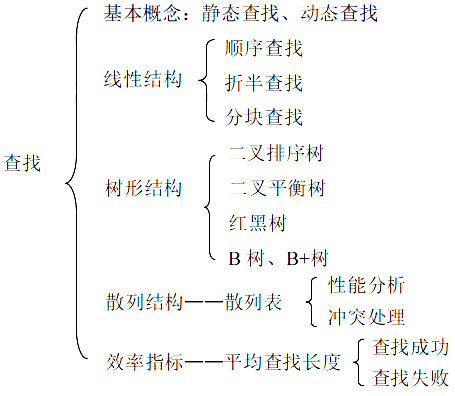
【考纲内容】

（一）查找的基本概念
（二）顺序查找法
（三）分块查找法
（四）折半查找法
（五）树形查找
　　二叉搜索树；平衡二叉树；红黑树
（六）B 树及其基本操作、B+树的基本概念
（七）散列（Hash）表
（八）查找算法的分析及应用

扫一扫

视频讲解

【知识框架】



【复习提示】

本章是考研命题的重点。对于折半查找，应掌握折半查找的过程、构造判定树、分析平均查找长度等。对于二叉排序树、二叉平衡树和红黑树，要了解它们的概念、性质和相关操作等。B 树和 B+树是本章的难点。对于 B 树，考研大纲要求掌握插入、删除和查找的操作过程；对于 B+树，仅要求了解其基本概念和性质。对于散列查找，应掌握散列表的构造、冲突处理方法（各种方法的处理过程）、查找成功和查找失败的平均查找长度、散列查找的特征和性能分析。

7.1　查找的基本概念

1）查找。在数据集合中寻找满足某种条件的数据元素的过程称为查找。查找的结果一般分为两种：一是查找成功，即在数据集合中找到了满足条件的数据元素；二是查找失败。

2）查找表。用于查找的数据集合称为查找表，它由同一类型的数据元素（或记录）组成。对查找表的常见操作有：①查询符合条件的数据元素；②插入、删除数据元素。

3）静态查找表。若一个查找表的操作只涉及查找操作，则无须动态地修改查找表，此类查找表称为静态查找表。与此对应，需要动态地插入或删除的查找表称为动态查找表。适合静态查找表的查找方法有顺序查找、折半查找、散列查找等；适合动态查找表的查找方法有二叉排序树的查找、散列查找等。

4）关键字。数据元素中唯一标识该元素的某个数据项的值，使用基于关键字的查找，查找结果应该是唯一的。例如，在由一个学生元素构成的数据集合中，学生元素中“学号”这一数据项的值唯一地标识一名学生。

5）平均查找长度。在查找过程中，一次查找的长度是指需要比较的关键字次数，而平均查找长度则是所有查找过程中进行关键字的比较次数的平均值，其数学定义为

$$\mathrm{ASL}=\sum_{i=1}^{n}P_iC_i$$

式中，n 是查找表的长度；P_i 是查找第 i 个数据元素的概率，一般认为每个数据元素的查找概率相等，即 $P_i=1/n$；C_i 是找到第 i 个数据元素所需进行的比较次数。平均查找长度是衡量查找算法效率的最主要的指标。

7.2 顺序查找和折半查找

7.2.1 顺序查找

顺序查找也称线性查找，它对顺序表和链表都是适用的。对于顺序表，可通过数组下标递增来顺序扫描每个元素；对于链表，可通过指针 next 来依次扫描每个元素。顺序查找通常分为对一般的无序线性表的顺序查找和对按关键字有序的线性表的顺序查找。下面分别进行讨论。

1．一般线性表的顺序查找

作为一种最直观的查找方法，其基本思想：①从线性表的一端开始，逐个检查关键字是否满足给定的条件；②若查找到某个元素的关键字满足给定条件，则查找成功，返回该元素在线性表中的位置；③若已经查找到表的另一端，但还没有查找到符合给定条件的元素，则返回查找失败的信息。下面给出其算法，后面说明了算法中引入的“哨兵”的作用。

```
typedef struct{          //查找表的数据结构（顺序表）
    ElemType *elem;      //动态数组基址
    int TableLen;        //表的长度
}SSTable;
int Search_Seq(SSTable ST,ElemType key){
    ST.elem[0]=key;                                    //“哨兵”
    for(int i=ST.TableLen;ST.elem[i]!=key;--i);//从后往前找
    return i;   //若查找成功，则返回元素下标；若查找失败，则返回0
}
```

上述算法中，将 ST.elem[0]称为哨兵，引入它的目的是使得 Search_Seq 内的循环不必判断数组是否会越界。算法从尾部开始查找，若找到 ST.elem[i]==key 则返回 i 值，查找成功。否则一定在查找到 ST.elem[0]==key 时跳出循环，此时返回的是 0，查找失败。在程序中引入“哨兵”，可以避免很多不必要的判断语句，从而提高程序效率。

对于有 n 个元素的表，给定值 key 与表中第 i 个元素相等，即定位第 i 个元素时，需进行 $n-i+1$

次关键字的比较，即 $C_i=n-i+1$。查找成功时，顺序查找的平均长度为

$$\mathrm{ASL}_{\text{成功}}=\sum_{i=1}^{n}P_i(n-i+1)$$

当每个元素的查找概率相等，即 $P_i=1/n$ 时，有

$$\mathrm{ASL}_{\text{成功}}=\sum_{i=1}^{n}P_i(n-i+1)=\frac{n+1}{2}$$

查找不成功时，与表中各关键字的比较次数显然是 $n+1$ 次，即 $\mathrm{ASL}_{\text{不成功}}=n+1$。

通常，查找表中记录的查找概率并不相等。若能预先得知每个记录的查找概率，则应先对记录的查找概率进行排序，使表中记录按查找概率由小至大重新排列。

综上所述，顺序查找的缺点是当 n 较大时，平均查找长度较大，效率低；优点是对数据元素的存储没有要求，顺序存储或链式存储皆可。对表中记录的有序性也没有要求，无论记录是否按关键字有序，均可应用。同时还需注意，对链表只能进行顺序查找。

2．有序线性表的顺序查找

若在查找之前就已知表是关键字有序的，则查找失败时可以不用再比较到表的另一端就能返回查找失败的信息，从而降低查找失败的平均查找长度。假设表 L 是按关键字从小到大排列的，查找的顺序是从前往后，待查找元素的关键字为 key，当查找到第 i 个元素时，发现第 i 个元素的关键字小于 key，但第 $i+1$ 个元素的关键字大于 key，这时就可返回查找失败的信息，因为第 i 个元素之后的元素的关键字均大于 key，所以表中不存在关键字为 key 的元素。

命题追踪　有序线性表的顺序查找的应用（2013）

可以用如图 7.1 所示的判定树来描述有序线性表的查找过程。树中的圆形结点表示有序线性表中存在的元素；矩形结点称为失败结点（若有 n 个结点，则相应地有 $n+1$ 个查找失败结点），它描述的是那些不在表中的数据值的集合。若查找到矩形结点，则说明查找失败。

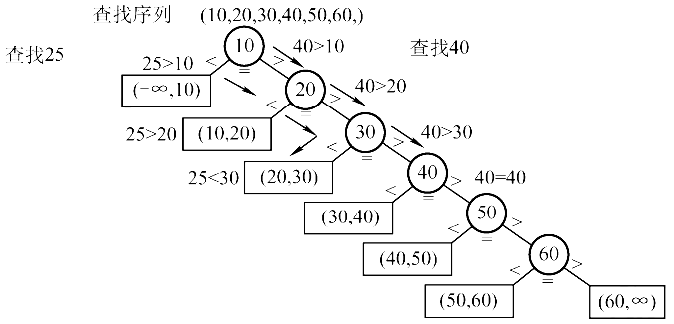


图 7.1　有序顺序表上的顺序查找判定树

在有序线性表的顺序查找中，查找成功的平均查找长度和一般线性表的顺序查找一样。查找失败时，查找指针一定走到了某个失败结点。这些失败结点是我们虚构的空结点，实际上是不存在的，所以到达失败结点时所查找的长度等于它上面的一个圆形结点的所在层数。查找不成功的平均查找长度在相等查找概率的情形下为

$$\mathrm{ASL}_{\text{不成功}}=\sum_{j=1}^{n}q_j(l_j-1)=\frac{1+2+\cdots+n+n}{n+1}=\frac{n}{2}+\frac{n}{n+1}$$

式中，q_j 是到达第 j 个失败结点的概率，在相等查找概率的情形下，它为 $1/(n+1)$；l_j 是第 j 个失败结点所在的层数。当 $n=6$ 时，$\mathrm{ASL}_{\text{不成功}}=6/2+6/7=3.86$，比一般的顺序查找好一些。

注意，有序线性表的顺序查找和后面的折半查找的思想是不一样的，且有序线性表的顺序查找中的线性表可以是链式存储结构，而折半查找中的线性表只能是顺序存储结构。

7.2.2 折半查找

折半查找也称二分查找，它仅适用于有序的顺序表。

命题追踪 ▶ 分析对比给定查找算法与折半查找的效率（2016）

折半查找的基本思想：①首先将给定值key与表中中间位置的元素比较，若相等，则查找成功，返回该元素的存储位置；②若不等，则所需查找的元素只能在中间元素以外的前半部分或后半部分（例如，在查找表升序排列时，若key大于中间元素，则所查找的元素只可能在后半部分），然后在缩小的范围内继续进行同样的查找。重复上述步骤，直到找到为止，或确定表中没有所需要查找的元素，则查找不成功，返回查找失败的信息。算法如下：

```
int Binary_Search(SSTable L,ElemType key){
    int low=0,high=L.TableLen-1,mid;
    while(low<=high){
        mid=(low+high)/2;                    //取中间位置
        if(L.elem[mid]==key)
            return  mid;                     //查找成功则返回所在位置
        else if(L.elem[mid]>key)
            high=mid-1;                      //从前半部分继续查找
        else
            low=mid+1;                       //从后半部分继续查找
    }
    return  -1;                              //查找失败，返回-1
}
```

当折半查找算法选取中间结点时，既可以采用向下取整，又可以采用向上取整。但每次查找的取整方式必须相同，这部分内容请读者结合本节部分习题来理解。

命题追踪 ▶ 折半查找的查找路径的判断（2015）

例如，已知11个元素的有序表{7, 10, 13, 16, 19, 29, 32, 33, 37, 41, 43}，要查找值为11和32的元素，指针low和high分别指向表的下界和上界，mid则指向表的中间位置⌊(low+high)/2⌋。

下面来说明查找11的过程（查找32的过程请读者自行分析）：

```
  7   10   13   16   19   29   32   33   37   41   43
  ↑low                  ↑mid                  ↑high
```

第一次查找时，将中间位置元素与key比较。因为11＜29，说明待查元素若存在，则必在范围[low,mid-1]内，令high指向位置mid-1，high=mid-1=5，mid=(1+5)/2=3，第二次查找范围为[1,5]。

```
  7   10   13   16   19   29   32   33   37   41   43
  ↑low      ↑mid      ↑high
```

第二次查找时，将中间位置元素与key比较。因为11＜13，说明待查元素若存在，则必在范围[low,mid-1]内，令high指向位置mid-1，high=mid-1=2，mid=(1+2)/2=1，第三次查找范围为[1,2]。

```
  7   10   13   16   19   29   32   33   37   41   43
low↑   ↑high
mid↑
```

第三次查找时，将中间位置元素与key比较。因为11＞7，说明待查元素若存在，则必在范围[mid+1,high]内。令low=mid+1=2，mid=(2+2)/2=2，第四次查找范围为[2, 2]。

```
 7   10   13   16   19   29   32   33   37   41   43
low↑  ↑high
      ↑mid
```

第四次查找，此时子表只含有一个元素，且 10≠11，所以表中不存在待查元素。

命题追踪 ▶ 分析给定二叉树树形能否构成折半查找判定树（2017）

折半查找的过程可用图 7.2 所示的二叉树来描述，称为判定树。树中每个圆形结点表示一个记录，结点中的值为该记录的关键字值；树中最下面的叶结点都是方形的，它表示查找失败的区间。从判定树可以看出，查找成功时的查找长度为从根结点到目的结点的路径上的结点数，而查找失败时的查找长度为从根结点到对应失败结点的父结点的路径上的结点数；每个结点值均大于其左子结点值，且均小于其右子结点值。若有序序列有 n 个元素，则对应的判定树有 n 个圆形的非叶结点和 $n+1$ 个方形的叶结点。显然，判定树是一棵平衡二叉树（见 7.3.2 节）。

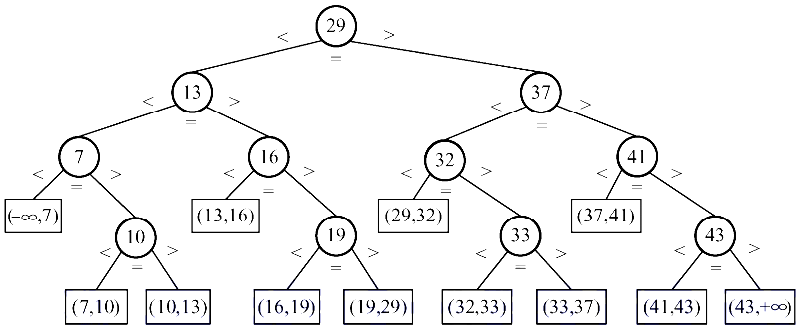


图 7.2　描述折半查找过程的判定树

命题追踪 ▶ 折半查找的最多比较次数的分析（2010、2023）

由上述分析可知，用折半查找法查找到给定值的比较次数最多不会超过树的高度。在等概率查找时，查找成功的平均查找长度为

$$\mathrm{ASL}=\frac{1}{n}\sum_{i=1}^{n}l_i=\frac{1}{n}(1\times1+2\times2+\cdots+h\times2^{h-1})=\frac{n+1}{n}\log_2(n+1)-1\approx\log_2(n+1)-1$$

式中，h 是树的高度，并且元素个数为 n 时树高 $h=\lceil\log_2(n+1)\rceil$。所以，折半查找的时间复杂度为 $O(\log_2 n)$，平均情况下比顺序查找的效率高。

在图 7.2 所示的判定树中，在等概率情况下，查找成功（圆形结点）的 ASL = (1×1 + 2×2 + 3×4 + 4×4)/11 = 3，查找失败（方形结点）的 ASL = (3×4 + 4×8)/12 = 11/3。

命题追踪 ▶ 折半查找的适用场景（2024）

因为折半查找需要方便地定位查找区域，所以它要求线性表必须具有随机存取的特性。因此，该查找法仅适合于顺序存储结构，不适合于链式存储结构，且要求元素按关键字有序排列。

7.2.3　分块查找

分块查找也称索引顺序查找，它吸取了顺序查找和折半查找各自的优点，既有动态结构，又适于快速查找。

分块查找的基本思想：将查找表分为若干子块。块内的元素可以无序，但块间的元素是有序的，即第一个块中的最大关键字小于第二个块中的所有记录的关键字，第二个块中的最大关键字小于第三个块中的所有记录的关键字，以此类推。再建立一个索引表，索引表中的每个元素含有

各块的最大关键字和各块中的第一个元素的地址，索引表按关键字有序排列。

分块查找的过程分为两步：第一步是在索引表中确定待查记录所在的块，可以顺序查找或折半查找索引表；第二步是在块内顺序查找。

例如，关键码集合为{88, 24, 72, 61, 21, 6, 32, 11, 8, 31, 22, 83, 78, 54}，按照关键码值 24, 54, 78, 88，分为 4 个块和索引表，如图 7.3 所示。

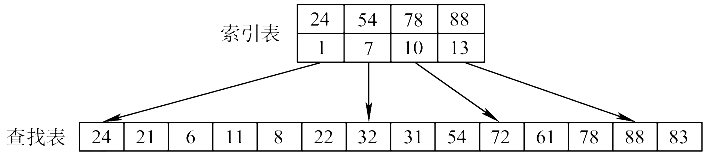


图 7.3　分块查找示意图

分块查找的平均查找长度为索引查找和块内查找的平均长度之和。设索引查找和块内查找的平均查找长度分别为 L_I 和 L_S，则分块查找的平均查找长度为

$$\mathrm{ASL} = L_I + L_S$$

将长度为 n 的查找表均匀地分为 b 块，每块有 s 个记录，在等概率情况下，若在块内和索引表中均采用顺序查找，则平均查找长度为

$$\mathrm{ASL} = L_I + L_S = \frac{b+1}{2} + \frac{s+1}{2} = \frac{s^2+2s+n}{2s}$$

此时，若 $s=\sqrt{n}$，则平均查找长度取最小值 $\sqrt{n}+1$。

虽然索引表占用了额外的存储空间，索引查找也增加了一定的系统开销，但由于其分块结构，使得在块内查找时的范围较小，与顺序查找相比，分块查找的总体效率提升了不少。

7.2.4　本节试题精选

一、单项选择题

01. 顺序查找适合于存储结构为（　）的线性表。

A. 顺序存储结构或链式存储结构　　B. 散列存储结构

C. 索引存储结构　　D. 压缩存储结构

02. 由 n 个数据元素组成的两个表：一个递增有序，一个无序。采用顺序查找算法，对有序表从头开始查找，发现当前元素已不小于待查元素时，停止查找，确定查找不成功，已知查找任意一个元素的概率是相同的，则在两种表中成功查找（　）。

A. 平均时间后者小　　B. 平均时间两者相同

C. 平均时间前者小　　D. 无法确定

03. 对长度为 n 的有序单链表，若查找每个元素的概率相等，则顺序查找表中任意一个元素的查找成功的平均查找长度为（　）。

A. $n/2$　　B. $(n+1)/2$　　C. $(n-1)/2$　　D. $n/4$

04. 对长度为 3 的顺序表进行查找，若查找第一个元素的概率为 1/2，查找第二个元素的概率为 1/3，查找第三个元素的概率为 1/6，则查找任意一个元素的平均查找长度为（　）。

A. 5/3　　B. 2　　C. 7/3　　D. 4/3

05. 下列关于二分查找的叙述中，正确的是（　）。

A. 表必须有序，表可以顺序方式存储，也可以链表方式存储

B. 表必须有序且表中数据必须是整型、实型或字符型

C. 表必须有序，而且只能从小到大排列

D. 表必须有序，且表只能以顺序方式存储

06. 在一个顺序存储的有序线性表上查找一个数据时，既可以采用折半查找，也可以采用顺序查找，但前者比后者的查找速度（　）。

A. 必然快　　B. 取决于表是递增还是递减

C. 在大部分情况下要快　　D. 必然不快

07. 折半查找过程所对应的判定树是一棵（　）。

A. 最小生成树　　B. 平衡二叉树　　C. 完全二叉树　　D. 满二叉树

08. 折半查找和二叉排序树的时间性能（　）。

A. 相同　　B. 有时不相同　　C. 完全不同　　D. 无法比较

09. 在有 11 个元素的有序表 A[1,2,…,11]中进行折半查找（⌊(low+high)/2⌋），查找元素 A[11]时，被比较的元素下标依次是（　）。

A. 6, 8, 10, 11　　B. 6, 9, 10, 11　　C. 6, 7, 9, 11　　D. 6, 8, 9, 11

10. 已知有序表(13, 18, 24, 35, 47, 50, 62, 83, 90, 115, 134)，当二分查找值为 90 的元素时，查找成功的元素比较次数为（　）。

A. 1　　B. 2　　C. 4　　D. 6

11. 若有序表的关键字序列为{*b*, *c*, *d*, *e*, *f*, *g*, *q*, *r*, *s*, *t*}，则在二分查找关键字 *b* 的过程中，进行比较的关键字依次为（　）。

A. *f*, *c*, *b*　　B. *f*, *d*, *b*　　C. *g*, *c*, *b*　　D. *g*, *d*, *b*

12. 对表长为 *n* 的有序表进行折半查找，其判定树的高度为（　）。

A. $\lceil \log_2(n+1) \rceil$　　B. $\lfloor \log_2(n+1) \rfloor - 1$　　C. $\lceil \log_2 n \rceil$　　D. $\lfloor \log_2 n \rfloor - 1$

13. 已知一个长度为 16 的顺序表，其元素按关键字有序排列，若采用折半查找算法查找一个不存在的元素，则比较的次数至少是（　），至多是（　）。

A. 4　　B. 5　　C. 6　　D. 7

14. 具有 12 个关键字的有序表中，对每个关键字的查找概率相同，折半查找算法查找成功的平均查找长度为（　），折半查找查找失败的平均查找长度为（　）。

A. 37/12　　B. 35/12　　C. 39/13　　D. 49/13

15. 下列关于查找的说法中，正确的是（　）。（注，涉及下节内容）

A. 若数据元素保持有序，则查找时就可以采用折半查找法

B. 折半查找与二叉查找树的时间性能在最坏情况下是相同的

C. 折半查找法的平均查找长度一定小于顺序查找法

D. 折半查找法查找一个元素大约需要 $O(\log_2 n)$次关键字比较

16. 采用分块查找时，数据的组织方式为（　）。

A. 数据分成若干块，每块内数据有序

B. 数据分成若干块，每块内数据不必有序，但块间必须有序，每块内最大（或最小）的数据组成索引块

C. 数据分成若干块，每块内数据有序，每块内最大（或最小）的数据组成索引块

D. 数据分成若干块，每块（除最后一块外）中数据个数需相同

17. 对有 2500 个记录的索引顺序表（分块表）进行查找，最理想的块长为（　）。

A. 50　　B. 125　　C. 500　　D. $\lceil \log_2 2500 \rceil$

18. 设顺序存储的某线性表共有 123 个元素，按分块查找的要求等分为 3 块。若对索引表采

用顺序查找法来确定子块，且在确定的子块中也采用顺序查找法，则在等概率情况下，分块查找成功的平均查找长度为（ ）。

A. 21　　B. 23　　C. 41　　D. 62

19. 为提高查找效率，对有65025个元素的有序顺序表建立索引顺序结构，在最好情况下查找到表中已有元素最多需要执行（ ）次关键字比较。

A. 10　　B. 14　　C. 16　　D. 21

20. 【2010统考真题】已知一个长度为16的顺序表L，其元素按关键字有序排列，若采用折半查找法查找一个L中不存在的元素，则关键字的比较次数最多是（ ）。

A. 4　　B. 5　　C. 6　　D. 7

21. 【2015统考真题】下列选项中，不能构成折半查找中关键字比较序列的是（ ）。

A. 500, 200, 450, 180　　B. 500, 450, 200, 180

C. 180, 500, 200, 450　　D. 180, 200, 500, 450

22. 【2016统考真题】在有n（$n>1000$）个元素的升序数组A中查找关键字x。查找算法的伪代码如下所示。

```
k=0;
while(k<n 且 A[k]<x) k=k+3;
if(k<n 且 A[k]==x) 查找成功;
else if(k-1<n 且 A[k-1]==x) 查找成功;
else if(k-2<n 且 A[k-2]==x) 查找成功;
else 查找失败;
```

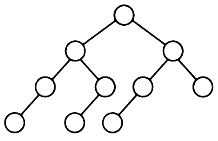
本算法与折半查找算法相比，有可能具有更少比较次数的情形是（ ）。

A. 当x不在数组中　　B. 当x接近数组开头处

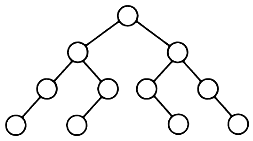
C. 当x接近数组结尾处　　D. 当x位于数组中间位置

23. 【2017统考真题】下列二叉树中，可能成为折半查找判定树（不含外部结点）的是（ ）。

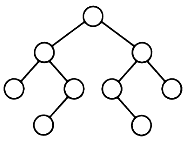
A.



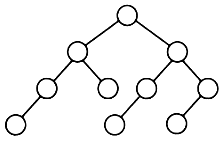
B.



C.



D.



24. 【2023统考真题】对含600个元素的有序顺序表进行折半查找，关键字间的比较次数最多是（ ）。

A. 9　　B. 10　　C. 30　　D. 300

25. 【2024统考真题】下列数据结构中，不适合直接使用折半查找的是（ ）。

I. 有序链表　　II. 无序数组　　III. 有序静态链表　　IV. 无序静态链表

A. 仅I、III　　B. 仅II、IV　　C. 仅II、III、IV　　D. I、II、III、IV

二、综合应用题

01. 若对有n个元素的有序顺序表和无序顺序表进行顺序查找，试就下列三种情况分别讨论两者在相等查找概率时的平均查找长度是否相同。

1）查找失败。

2）查找成功，且表中只有一个关键字等于给定值k的元素。

3）查找成功，且表中有若干关键字等于给定值 k 的元素，要求一次查找能找出所有元素。

02. 有序顺序表中的元素依次为 017, 094, 154, 170, 275, 503, 509, 512, 553, 612, 677, 765, 897, 908。

1）试画出对其进行折半查找的判定树。

2）若查找 275 或 684 的元素，将依次与表中的哪些元素比较？

3）计算查找成功的平均查找长度和查找不成功的平均查找长度。

03. 已知一个有序顺序表 A[0…8n-1]的表长为 8n，并且表中没有关键字相同的数据元素。假设按下述方法查找一个关键字值等于给定值 X 的数据元素：首先在 A[7],A[15],A[23],…,A[8k-1],…,A[8n-1]中进行顺序查找，若查找成功，则算法报告成功位置并返回；若不成功，则当 A[8k-1]<X<A[8×(k+1)-1]时，可确定一个缩小的查找范围 A[8k]~A[8×(k+1)-2]，然后可在这个范围内执行折半查找。特殊情况：若 X>A[8n-1]的关键字，则查找失败。

1）画出描述上述查找过程的判定树。

2）计算相等查找概率下查找成功的平均查找长度。

04. 写出折半查找的递归算法。初始调用时，low 为 1，high 为 ST.length。

05. 线性表中各结点的检索概率不等时，可用如下策略提高顺序检索的效率：若找到指定的结点，则将该结点和其前驱结点（若存在）交换，使得经常被检索的结点尽量位于表的前端。试设计在顺序结构和链式结构的线性表上实现上述策略的顺序检索算法。

06. 已知一个 n 阶矩阵 A 和一个目标值 k。该矩阵无重复元素，每行从左到右升序排列，每列从上到下升序排列。请设计一个在时间上尽可能高效的算法，判断矩阵中是否存在目标值 k。例如，矩阵为 $\begin{bmatrix}1 & 4 & 7\\ 2 & 5 & 8\\ 3 & 6 & 9\end{bmatrix}$，目标值为 8，判断存在。要求：

1）给出算法的基本设计思想。

2）根据设计思想，采用 C 或 C++语言描述算法，关键之处给出注释。

3）说明你的算法的时间复杂度和空间复杂度。

07.【2013 统考真题】设包含 4 个数据元素的集合 S = {'do', 'for', 'repeat', 'while'}，各元素的查找概率依次为 $p_1 = 0.35, p_2 = 0.15, p_3 = 0.15, p_4 = 0.35$。将 S 保存在一个长度为 4 的顺序表中，采用折半查找法，查找成功时的平均查找长度为 2.2。

1）若采用顺序存储结构保存 S，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？

2）若采用链式存储结构保存 S，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？

7.2.5　答案与解析

一、单项选择题

01. A

顺序查找是指从表的一端开始向另一端查找。它不要求查找表具有随机存取的特性，可以是顺序存储结构或链式存储结构。

02. B

对于顺序查找，不管线性表是有序的还是无序的，成功查找第一个元素的比较次数为 1，成

功查找第二个元素的比较次数为 2，以此类推，即每个元素查找成功的比较次数只与其位置有关（与是否有序无关），因此查找成功的平均时间两者相同。

03. B

在有序单链表上做顺序查找，查找成功的平均查找长度与在无序顺序表或有序顺序表上做顺序查找的平均查找长度相同，都是$(n+1)/2$。

04. A

在长度为 3 的顺序表中，查找第一个元素的查找长度为 1，查找第二个元素的查找长度为 2，查找第三个元素的查找长度为 3，所以有

$$\mathrm{ASL}_{\text{成功}}=\frac{1}{2}\times 1+\frac{1}{3}\times 2+\frac{1}{6}\times 3=\frac{5}{3}$$

05. D

二分查找通过下标来定位中间位置元素，所以应采用顺序存储，且二分查找能够进行的前提是查找表是有序的，但具体是从大到小还是从小到大的顺序则不做要求。

06. C

折半查找的快体现在一般情况下，在大部分情况下要快，但是对于某些特殊情况，顺序查找可能会快于折半查找。例如，查找一个含 1000 个元素的有序表中的第一个元素时，顺序查找的比较次数为 1 次，而折半查找的比较次数却将近 10 次。

07. B

A 显然排除。对于选项 C，考点精析示例中的判定树就不是完全二叉树。由选项 C 也可排除选项 D，且满二叉树对结点数有要求。只可能选择选项 B。事实上，由折半查找的定义不难看出，每次把一个数组从中间结点分割时，总是把数组分为结点数相差最多不超过 1 的两个子数组，从而使得对应的判定树的两棵子树高度差的绝对值不超过 1，所以应是平衡二叉树。

08. B

折半查找的性能分析可以用二叉判定树来衡量，平均查找长度和最大查找长度都是$O(\log_2 n)$；二叉排序树的查找性能与数据的输入顺序有关，最好情况下的平均查找长度与折半查找相同，但最坏情况即形成单支树时，其查找长度为$O(n)$。

09. B

依据折半查找算法的思想，第一次`mid`=⌊`(1+11)/2`⌋=6，第二次`mid`=⌊`[(6+1)+11]/2`⌋=9，第三次`mid`=⌊`[(9+1)+11]/2`⌋=10，第四次`mid`=11。

10. B

开始时`low`指向 13，`high`指向 134，`mid`指向 50，比较第一次 90 > 50，所以将`low`指向 62，`high`指向 134，`mid`指向 90，第二次比较找到 90。

11. A

在折半查找算法中，`mid`取值的方式是确定的，要么采用向上取整，要么采用向下取整，而不能出现两种情况。对于选项 A，第 1 次比较的元素是 f，为向下取整；第 2 次比较的元素是 c，为向下取整；第 3 次比较的元素是 b，为向下取整，查找成功，符合二分查找。对于选项 B，第 1 次比较的元素是 f，为向下取整；第 2 次比较的元素是 d，为向上取整，两次`mid`取值的方式不同，不符合二分查找。对于选项 C，第 1 次比较的元素是 g，为向上取整；第 2 次比较的元素是 c，为向下取整，不符合二分查找。对于选项 D，第 1 次比较的元素是 g，为向上取整；第 2 次比较的元素是 d，为正中间元素；第 3 次比较的元素为 b，为向下取整，不符合二分查找。

12. A

对 n 个结点的判定树，设结点总数 $n=2^h-1$，则 $h=\lceil\log_2(n+1)\rceil$。

【另解】特殊值代入法。直接将 $n=1$ 和 $n=2$ 的情况代入，仅有 A 满足要求。

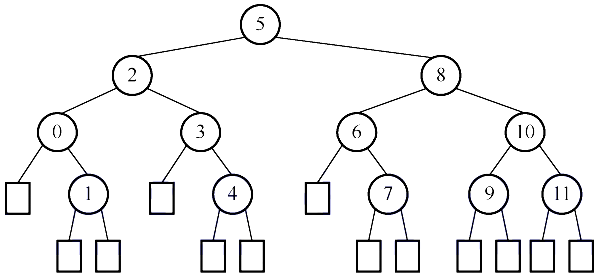
13．A、B

对于此类题，有两种做法：一种方法是，画出查找过程中构成的判定树，让最小的分支高度对应于最少的比较次数，让最大的分支高度对应于最多的比较次数，出现类似于长度为 15 的顺序表时，判定树刚好是一棵满树，此时最多比较次数与最少比较次数相等；另一种方法是，直接用公式求出最小的分支高度和最大分支高度，从前面的讲解不难看出最大分支高度为 $H=\lceil\log_2(n+1)\rceil=5$，这对应的就是最多比较次数，然后因为判定树不是一棵满树，所以至少应该是 4（由判定树的各分支高度最多相差 1 得出）。

注意，若求查找成功或查找失败的平均查找长度，则需要画出判定树来求解。此外，对长度为 n 的有序表，采用折半查找时，查找成功和查找失败的最多比较次数相同，均为 $\lceil\log_2(n+1)\rceil$。

14．A、D

假设有序表中元素为 A[0…11]，不难画出对它进行折半查找的判定树如下图所示，圆圈是查找成功结点，方形是虚构的查找失败结点。从而可以求出查找成功的 ASL = (1 + 2×2 + 3×4 + 4×5)/12 = 37/12，查找失败的 ASL = (3×3 + 4×10)/13。



注　意

对于本类题目，应先根据所给 n 的值，画出如上图所示的折半查找判定树。另外，查找失败结点的 ASL 不是到图中的方形结点，而是到方形结点上一层的圆形结点。

15．D

折半查找法不仅要求数据元素有序，而且要求必须为顺序存储，选项 A 错误。折半查找法在最坏情况下的时间性能为 $O(\log_2 n)$，二叉查找树在最坏情况下的时间性能为 $O(n)$，选项 B 错误。在每个元素查找概率不同的情况下，折半查找法的平均查找长度可能大于顺序查找法，选项 C 错误。

16．B

通常情况下，在分块查找的结构中，不要求每个索引块中的元素个数都相等。

17．A

设块长为 b，索引表包含 n/b 项，索引表的 ASL = $(n/b+1)/2$，块内的 ASL = $(b+1)/2$，总 ASL = 索引表的 ASL + 块内的 ASL = $(b+n/b+2)/2$，其中对于 $b+n/b$，由均值不等式知 $b=n/b$ 时有最小值，此时 $b=\sqrt{n}$。则最理想块长为 $\sqrt{2500}=50$。

18．B

根据公式 $\mathrm{ASL}=L_1+L_S=\dfrac{b+1}{2}+\dfrac{s+1}{2}=\dfrac{s^2+2s+n}{2s}$，其中 $b=n/s$, $s=123/3$, $n=123$，代入不难得出 ASL 为 23。所以选择选项 B。另一方面，可根据穷举法来一步步模拟。对于 A 块中的元素，查找过程的第一步是先找到 A 块，由于是顺序查找，找到 A 块只需一步，然后在 A 块中顺序查找，因此 A 块内各元素查找长度分别为 2, 3, 4, …, 42。对于 B 块，采用类似的方法，但查找到 B 块要比查找到

A 块多一步，因此 B 块内各元素查找长度为 3, 4, 5,…, 43。同理，C 块中各个元素查找长度为 4, 5, 6,…, 44。所以平均查找长度为(2 + 3 + 4 +…+ 42 + 3 + 4 + 5 +…+ 43 + 4 + 5 + 6 +…+ 44)/123 = 23。

19. C

为使查找效率最高，每个索引块的大小应是$\sqrt{65025}$=255，为每个块建立索引，则索引表中索引项的个数为 255。若对索引项和索引块内部都采用折半查找，则查找效率最高，为$\lceil \log_2(255+1) \rceil + \lceil \log_2(255+1) \rceil = 16$。

20. B

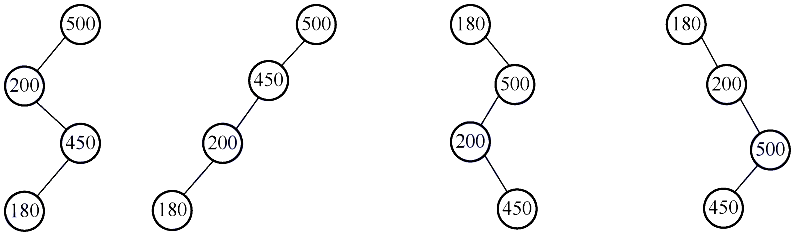
折半查找法在查找不成功时和给定值进行关键字的比较次数最多为树的高度，即$\lfloor \log_2 n \rfloor + 1$或$\lceil \log_2(n+1) \rceil$。在本题中，$n = 16$，所以比较次数最多为 5。

注 意

在折半查找判定树中的方形结点是虚构的，它不计入比较的次数。

21. A

画出查找路径图，因为折半查找判定树是一棵二叉排序树，看其是否满足二叉排序树的要求。



显然，选项 A 的查找路径不满足。

22. B

本题为送分题。

该程序采用跳跃式的顺序查找法查找升序数组中的 x。显然，x 越靠前，比较次数越少。

23. A

对于给定的一个有序查找表，其对应的折半查找判定树是确定且唯一的。在折半查找算法中，mid=⌊(low+high)/2⌋，因此若表中初始有 $2n + 1$ 个元素，则 mid 分割后，左右子树各有 n 个元素；若表中初始有 $2n$ 个元素，则 mid 分割后，左子树有 $n - 1$ 个元素，右子树有 n 个元素。即左子树的元素个数或者与右子树的元素个数相等，或者比右子树少一个。若令 mid=⌈(low+high)/2⌉，不难理解，左子树的元素个数或者与右子树的元素个数相等，或者比右子树多一个。对于选项 A，树中每个左子树都与右子树的结点个数相等，或者多一个结点，符合向上取整的规则。对于选项 B、C、D，存在有的左子树比右子树多一个结点，有的左子树比右子树少一个结点，不符合折半查找的规则。

24. B

用折半查找法查找给定值的比较次数最多不超过折半查找判定树的高度。折半查找判定树的树高 $h = \lceil \log_2(n+1) \rceil$，将 $n = 600$ 代入，结果为 10。

25. D

折半查找必须满足两个条件：①数组（或顺序表），折半查找的上一次查找和本次查找可能相隔很远的距离，如依次查找下标为 $n/2, n/4, n/8, \cdots$ 的元素，若采用链表（或静态链表），则会使

得时间复杂度非常高。②有序，只有在有序的情况下才能根据上一次的比较情况舍弃一半的序列。

二、综合应用题

01.【解答】

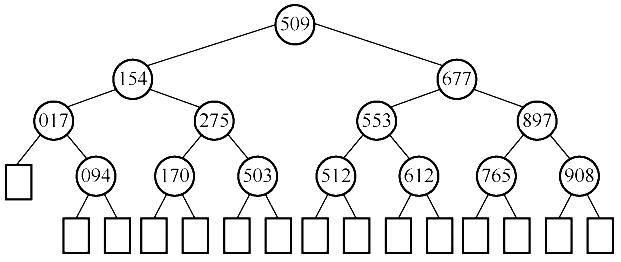
1）平均查找长度不同。因为有序顺序表查找到其关键字值比要查找值大的元素时就停止查找，并报告失败信息，不必查找到表尾；而无序顺序表必须查找到表尾才能确定查找失败。

2）平均查找长度相同。两者查找到表中元素的关键字值等于给定值时就停止查找。

3）平均查找长度不同。有序顺序表中关键字相等的元素相继排列在一起，只要查找到第一个就可以连续查找到其他关键字相同的元素。而无序顺序表必须查找全部表中的元素才能找出相同关键字的元素，因此所需的时间不同。

02.【解答】

1）判定树如下图所示。



2）若查找 275，依次与表中元素 509, 154, 275 进行比较，共比较 3 次。若查找 684，依次与表中元素 509, 677, 897, 765 进行比较，共比较 4 次。

3）在查找成功时，会找到图中的某个圆形结点，其平均查找长度为

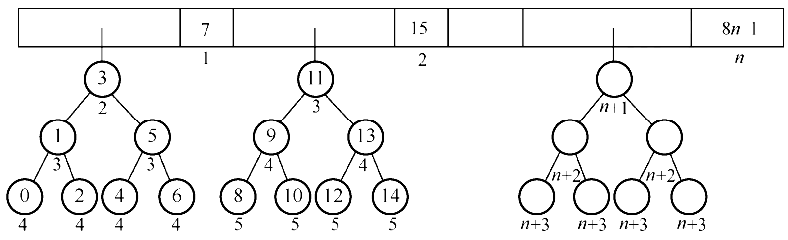
$$\mathrm{ASL}_{\text{成功}}=\frac{1}{14}\sum_{i=1}^{14}C_i=\frac{1}{14}(1+2\times2+3\times4+4\times7)=\frac{45}{14}$$

在查找失败时，会找到图中的某个方形结点，但这个结点是虚构的，最后一次的比较元素为其父结点（圆形结点），所以其平均查找长度为

$$\mathrm{ASL}_{\text{不成功}}=\frac{1}{15}\sum_{i=0}^{14}C_i'=\frac{1}{15}(3\times1+4\times14)=\frac{59}{15}$$

03.【解答】

1）先在 A[7],A[15],…,A[8n-1]内顺序查找，再在区间内折半查找。相应的判定树如下图所示。其中，每个关键字下的数字为其查找成功时的关键字比较次数。



2）等查找概率下，平均每个关键字查找成功的概率为 1/8n；0～7 之间的关键字，顺序比较 1 次后，进行折半查找，查找成功的平均查找长度为 2 + 3×2 + 4×4；8～15 之间的关键字，先顺序比较 2 次后，再进入折半查找；以此类推，8(n – 1)～8n – 1 之间的关键字，先顺序比较 n 次，再进入折半查找，如上图所示。因此，查找成功的平均查找长度为

$$\begin{aligned}\mathrm{ASL}_{\text{成功}}&=\frac{1}{8n}\sum_{i=0}^{8n-1}C_i=\frac{1}{8n}\left(\sum_{i=1}^{n}i+\sum_{i=2}^{n+1}i+2\sum_{i=3}^{n+2}i+4\sum_{i=4}^{n+3}i\right)\\&=\frac{1}{8n}\left(\sum_{i=1}^{n}(i+(i+1)+2(i+2)+4(i+3))\right)\\&=\frac{1}{8n}\sum_{i=1}^{n}(8i+17)=\frac{1}{n}\sum_{i=1}^{n}i+\frac{17}{8}=\frac{n+1}{2}+\frac{17}{8}\end{aligned}$$

04.【解答】

算法的基本思想：根据查找的起始位置和终止位置，将查找序列一分为二，判断所查找的关键字在哪一部分，然后用新的序列的起始位置和终止位置递归求解。

算法代码如下：

```
typedef struct{                 //查找表的数据结构
    ElemType    *elem;          //存储空间基址，建表时按实际长度分配，0 号留空
    int     length;             //表的长度
} SSTable;
int BinSearchRec(SSTable ST,ElemType key,int low,int high){
    if(low>high)
        return  0;
    mid=(low+high)/2;           //取中间位置
    if(key>ST.elem[mid])        //向后半部分查找
        BinSearchRec(ST,key,mid+1,high);
    else if(key<ST.elem[mid])   //向前半部分查找
        BinSearchRec(ST,key,low,mid-1);
    else                        //查找成功
        return  mid;
}
```

算法把规模为 n 的复杂问题经过多次递归调用转化为规模减半的子问题求解。时间复杂度为 $O(\log_2 n)$，算法中用到了一个递归工作栈，其规模与递归深度有关，也是 $O(\log_2 n)$。

05.【解答】

算法的基本思想：检索时可先从表头开始向后顺序扫描，若找到指定的结点，则将该结点和其前趋结点（若存在）交换。采用顺序表存储结构的算法实现如下：

```
int SeqSrch(RcdType R[],ElemType k){
//顺序查找线性表，找到后和其前面的元素交换
    int i=0;
    while((R[i].key!=k)&&(i<n))
        i++;                    //从前向后顺序查找指定结点
    if(i<n&&i>0){               //若找到，则交换
        temp=R[i];R[i]=R[i-1];R[i-1]=temp;
        return  --i;            //交换成功，返回交换后的位置
    }
     else return  -1;           //交换失败
}
```

链表的实现方式请读者自行思考。注意，链表方式实现的基本思想与上述思想相似，但要注意用链表实现时，在交换两个结点之前需要保存指向前一结点的指针。

06.【解析】

1）算法的基本设计思想：

从矩阵 $\boldsymbol{A}$ 的右上角（最右列）开始比较，若当前元素小于目标值，则向下寻找下一个更大的元素；若当前元素大于目标值，则从右往左依次比较，若目标值存在，则只可能在该行中。

2）算法的实现：

```
bool findkey(int A[][],int n,int k){
    int i=0,j=n-1;
    while(i<n&&j>=0){                    //离开边界时查找结束
        if(A[i][j]==k) return true;      //查找成功
        else if(A[i][j]>k)  j--;         //向左移动，在该行内寻找目标值
        else i++;                        //向下移动，查找下一个更大的元素
    }
    return false;                        //查找失败
}
```

3）比较次数不超过 $2n$ 次，时间复杂度为 $O(n)$；空间复杂度为 $O(1)$。

07.【解答】

1）折半查找要求元素有序顺序存储，字符串默认按字典序排序（字典序是一种比较两个字符串大小的方法，它按字母顺序从左到右逐个比较对应的字符，若某一位可比较出大小，则不再继续比较后面的字符，如 abd<acd、abc<abcd 等），对本题来说 do<for<repeat<while。若各个元素的查找概率不同，折半查找的性能不一定优于顺序查找。采用顺序查找时，元素按其查找概率的降序排列时查找长度最小。

采用顺序存储结构，数据元素按其查找概率降序排列。采用顺序查找方法。

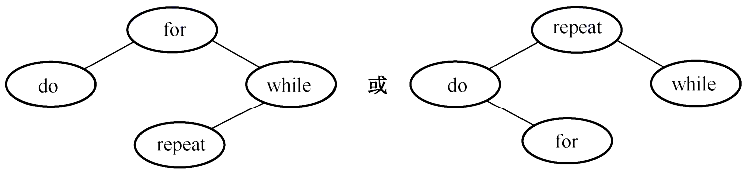
查找成功时的平均查找长度 $=0.35\times1+0.35\times2+0.15\times3+0.15\times4=2.1$。

此时，显然查找长度比折半查找的更短。

2）答案 1：采用链式存储结构时，只能采用顺序查找，其性能和顺序表一样，类似于上题。数据元素按其查找概率降序排列，构成单链表。采用顺序查找方法。

查找成功时的平均查找长度 $=0.35\times1+0.35\times2+0.15\times3+0.15\times4=2.1$。

答案 2：还可以构造成二叉排序树的形式。采用二叉链表的存储结构，构造二叉排序树，元素的存储方式见下图。采用二叉排序树的查找方法。



二叉排序树1　　二叉排序树2

查找成功时的平均查找长度 $=0.15\times1+0.35\times2+0.35\times2+0.15\times3=2.0$。

7.3　树形查找

7.3.1　二叉排序树（BST）

构造一棵二叉排序树的目的并不是排序，而是提高查找、插入和删除关键字的速度，二叉排序树这种非线性结构也有利于插入和删除的实现。

1．二叉排序树的定义

命题追踪 ▶ 二叉排序树的应用（2013）

二叉排序树（也称二叉查找树）或者是一棵空树，或者是具有下列特性的二叉树：

1）若左子树非空，则左子树上所有结点的值均小于根结点的值。

2）若右子树非空，则右子树上所有结点的值均大于根结点的值。

3）左、右子树也分别是一棵二叉排序树。

命题追踪 ▶ 二叉排序树中结点值之间的关系（2015、2018、2024）

根据二叉排序树的定义，左子树结点值 < 根结点值 < 右子树结点值，因此对二叉排序树进行中序遍历，可以得到一个递增的有序序列。例如，图 7.4 所示二叉排序树的中序遍历序列为 123468。

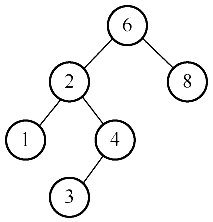


图 7.4 一棵二叉排序树

2．二叉排序树的查找

二叉排序树的查找是从根结点开始，沿某个分支逐层向下比较的过程。若二叉排序树非空，先将给定值与根结点的关键字比较，若相等，则查找成功；若不等，若小于根结点的关键字，则在根结点的左子树上查找，否则在根结点的右子树上查找。这显然是一个递归的过程。

二叉排序树的非递归查找算法：

```
BSTNode *BST_Search(BiTree T,ElemType key){
    while(T!=NULL&&key!=T->data){     //若树空或等于根结点值，则结束循环
        if(key<T->data) T=T->lchild;  //小于，则在左子树上查找
        else T=T->rchild;             //大于，则在右子树上查找
    }
    return T;
}
```

例如，在图 7.4 中查找值为 4 的结点。首先 4 与根结点 6 比较。因为 4 小于 6，所以在根结点 6 的左子树中继续查找。因为 4 大于 2，所以在结点 2 的右子树中查找，查找成功。

同样，二叉排序树的查找也可用递归算法实现，递归算法比较简单，但执行效率较低。具体的代码实现，留给读者思考。

3．二叉排序树的插入

二叉排序树作为一种动态树表，其特点是树的结构通常不是一次生成的，而是在查找过程中，当树中不存在关键字值等于给定值的结点时再进行插入的。

插入结点的过程如下：若原二叉排序树为空，则直接插入；否则，若关键字 k 小于根结点值，则插入到左子树，若关键字 k 大于根结点值，则插入到右子树。新插入的结点一定是一个叶结点，且是查找失败时的查找路径上访问的最后一个结点的左孩子或右孩子。如图 7.5 所示在一棵二叉排序树中依次插入结点 28 和结点 58，虚线表示的边是其查找的路径。

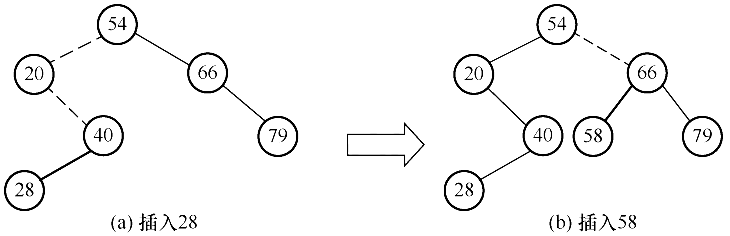


图 7.5 向二叉排序树中插入结点

二叉排序树插入操作的算法描述如下：

```
int BST_Insert(BiTree &T,KeyType k){
    if(T==NULL){                    //原树为空，新插入的记录为根结点
        T=(BiTree)malloc(sizeof(BSTNode));
        T->data=k;
        T->lchild=T->rchild=NULL;
        return 1;                   //返回 1，插入成功
    }
    else if(k==T->data)             //树中存在相同关键字的结点，插入失败
        return 0;
    else if(k<T->data)              //插入 T 的左子树
        return BST_Insert(T->lchild,k);
    else                            //插入 T 的右子树
        return BST_Insert(T->rchild,k);
}
```

4．二叉排序树的构造

命题追踪 ▶ 构造二叉排序树的过程（2020）

从一棵空树出发，依次输入元素，将它们插入二叉排序树中的合适位置。设插入的关键字序列为{45, 24, 53, 45, 12, 24}，则生成的二叉排序树如图 7.6 所示。

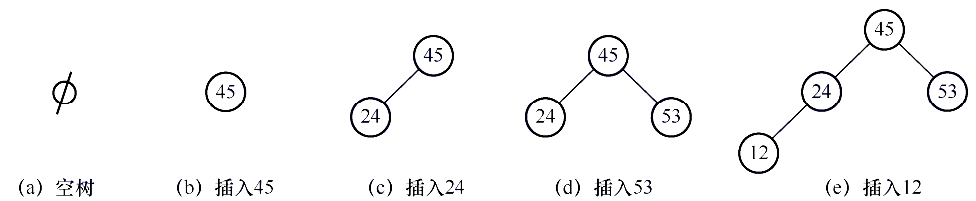


图 7.6　二叉排序树的构造过程

构造二叉排序树的算法描述如下：

```
void Creat_BST(BiTree &T,KeyType str[],int n){
    T=NULL;             //初始时 T 为空树
    int i=0;
    while(i<n){         //依次将每个关键字插入二叉排序树
        BST_Insert(T,str[i]);
        i++;
    }
}
```

5．二叉排序树的删除

在二叉排序树中删除一个结点时，不能把以该结点为根的子树上的结点都删除，必须先把被删除结点从存储二叉排序树的链表上摘下，将因删除结点而断开的二叉链表重新链接起来，同时确保二叉排序树的性质不会丢失。删除操作的实现过程按 3 种情况来处理：

① 若被删除结点 z 是叶结点，则直接删除，不会破坏二叉排序树的性质。

② 若结点 z 只有一棵左子树或右子树，则让 z 的子树成为 z 父结点的子树，替代 z 的位置。

③ 若结点 z 有左、右两棵子树，则令 z 的直接后继（或直接前驱）替代 z，然后从二叉排序树中删去这个直接后继（或直接前驱），这样就转换成了第一或第二种情况。

图 7.7 显示了在 3 种情况下分别删除结点 45, 78, 78 的过程。

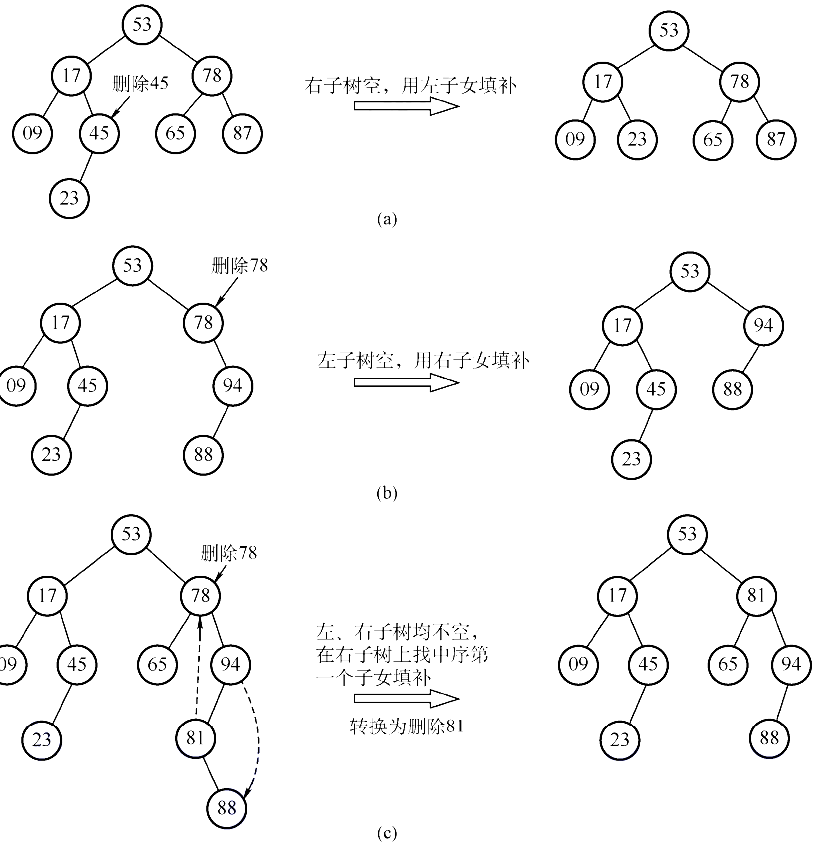


图 7.7　3 种情况下的删除过程

命题追踪 ▶ 二叉排序树中删除并插入某结点的分析（2013）

思考：若在二叉排序树中删除并插入某结点，得到的二叉排序树是否和原来的相同？

6．二叉排序树的查找效率分析

二叉排序树的查找效率，主要取决于树的高度。若二叉排序树的左、右子树的高度之差的绝对值不超过 1（平衡二叉树，下一节），它的平均查找长度和 $O(\log_2 n)$成正比。在最坏情况下，即构造二叉排序树的输入序列是有序的，则会形成一个只有右孩子的单支树，此时二叉排序树的性能显著变坏，树的高度为 n，则其平均查找长度为$(n+1)/2$，如图 7.8(b)所示。

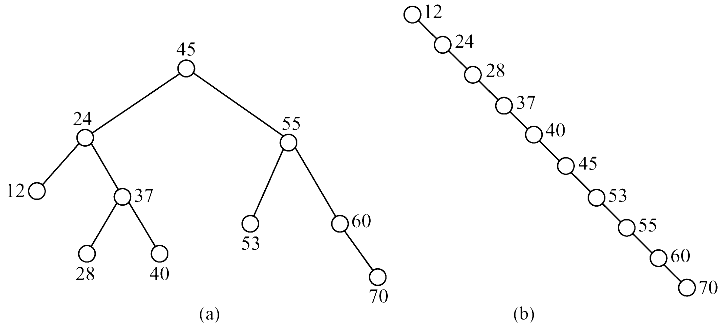


图 7.8　相同关键字组成的不同二叉排序树

在等概率情况下，图 7.8(a)查找成功的平均查找长度为

$$ASL_a = (1 + 2\times2 + 3\times4 + 4\times3)/10 = 2.9$$

而图 7.8(b)查找成功的平均查找长度为

$$ASL_b = (1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10)/10 = 5.5$$

从查找过程看，二叉排序树与二分查找相似。就平均时间性能而言，二叉排序树上的查找和二分查找差不多。但二分查找的判定树唯一，而二叉排序树的查找不唯一，相同的关键字其插入顺序不同可能生成不同的二叉排序树，如图 7.8 所示。

就维护表的有序性而言，二叉排序树无须移动结点，只需修改指针即可完成插入和删除操作，平均执行时间为 $O(\log_2 n)$。二分查找的对象是有序顺序表，若有插入和删除结点的操作，所花的代价是 $O(n)$。当有序表是静态查找表时，宜用顺序表作为其存储结构，而采用二分查找实现其查找操作；若有序表是动态查找表，则应选择二叉排序树作为其逻辑结构。

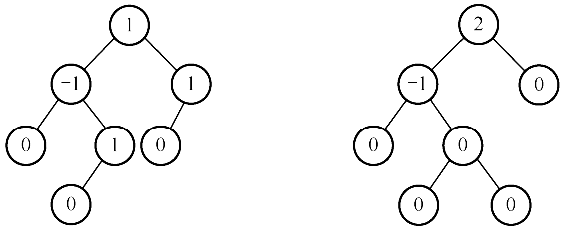
7.3.2　平衡二叉树

1．平衡二叉树的定义

为了避免树的高度增长过快，降低二叉排序树的性能，规定在插入和删除结点时，要保证任意结点的左、右子树高度差的绝对值不超过 1，将这样的二叉树称为平衡二叉树（Balanced Binary Tree），也称 AVL 树。定义结点左子树与右子树的高度差为该结点的平衡因子，则平衡二叉树结点的平衡因子的值只可能是−1、0 或 1。

命题追踪 ▶ 平衡二叉树的定义（2009）

因此，平衡二叉树可定义为或是一棵空树，或是具有下列性质的二叉树：它的左子树和右子树都是平衡二叉树，且左子树和右子树的高度差的绝对值不超过 1。图 7.9(a)所示是平衡二叉树，图 7.9(b)所示是不平衡的二叉树。结点中的数字为该结点的平衡因子。



(a) 平衡二叉树　　(b) 不平衡的二叉树

图 7.9　平衡二叉树和不平衡的二叉树

2．平衡二叉树的插入

二叉排序树保证平衡的基本思想如下：每当在二叉排序树中插入（或删除）一个结点时，首先检查其插入路径上的结点是否因为此次操作而导致了不平衡。若导致了不平衡，则先找到插入路径上离插入结点最近的平衡因子的绝对值大于 1 的结点 A，再对以 A 为根的子树，在保持二叉排序树特性的前提下，调整各结点的位置关系，使之重新达到平衡。

命题追踪 ▶ 平衡二叉树中插入操作的特点（2015）

注意，每次调整的对象都是最小不平衡子树，即以插入路径上离插入结点最近的平衡因子的绝对值大于 1 的结点作为根的子树。图 7.10 中的虚线框内为最小不平衡子树。

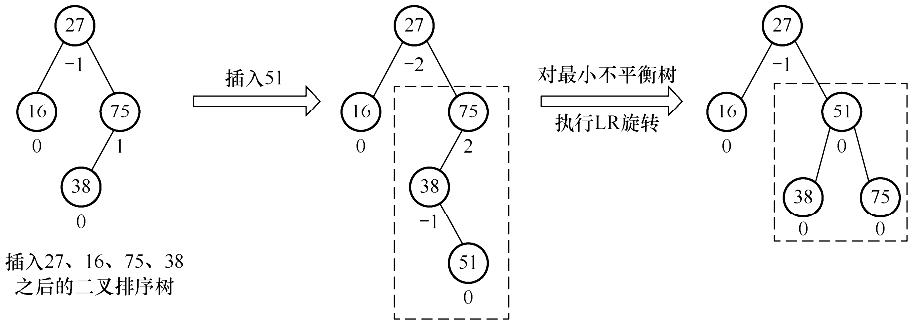


图 7.10　最小不平衡子树示意

命题追踪　▶　平衡二叉树的插入及调整操作的实例（2010、2019、2021）

平衡二叉树的插入过程的前半部分与二叉排序树相同，但在新结点插入后，若造成查找路径上的某个结点不再平衡，则需要做出相应的调整。可将调整的规律归纳为下列 4 种情况：

1）LL 平衡旋转（右单旋转）。由于在结点 A 的左孩子（L）的左子树（L）上插入了新结点，A 的平衡因子由 1 增至 2，导致以 A 为根的子树失去平衡，需要一次向右的旋转操作。将 A 的左孩子 B 向右上旋转代替 A 成为根结点，将 A 向右下旋转成为 B 的右孩子，而 B 的原右子树则作为 A 的左子树。如图 7.11 所示，结点旁的数值代表结点的平衡因子，而用方块表示相应结点的子树，下方数值代表该子树的高度。

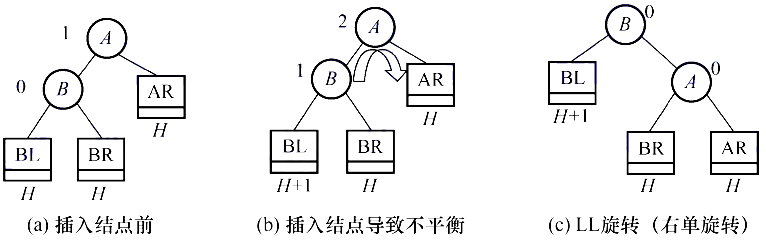


图 7.11　LL 平衡旋转

2）RR 平衡旋转（左单旋转）。由于在结点 A 的右孩子（R）的右子树（R）上插入了新结点，A 的平衡因子由-1 减至-2，导致以 A 为根的子树失去平衡，需要一次向左的旋转操作。将 A 的右孩子 B 向左上旋转代替 A 成为根结点，将 A 向左下旋转成为 B 的左孩子，而 B 的原左子树则作为 A 的右子树，如图 7.12 所示。

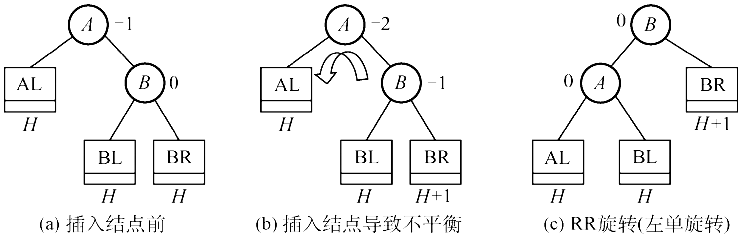
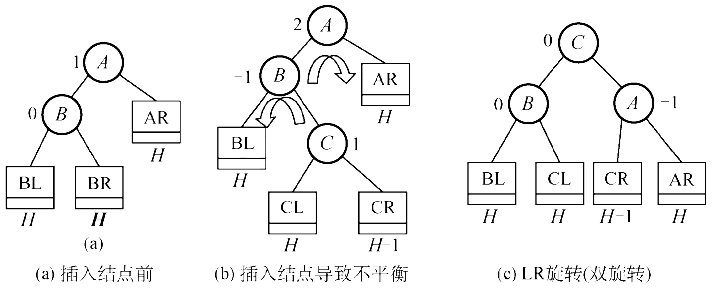


图 7.12　RR 平衡旋转

3）LR 平衡旋转（先左后右双旋转）。由于在结点 A 的左孩子（L）的右子树（R）上插入新结点，A 的平衡因子由 1 增至 2，导致以 A 为根的子树失去平衡，需要进行两次旋转操作，先左旋转后右旋转。先将 A 的左孩子 B 的右子树的根结点 C 向左上旋转提升到 B 的位置，

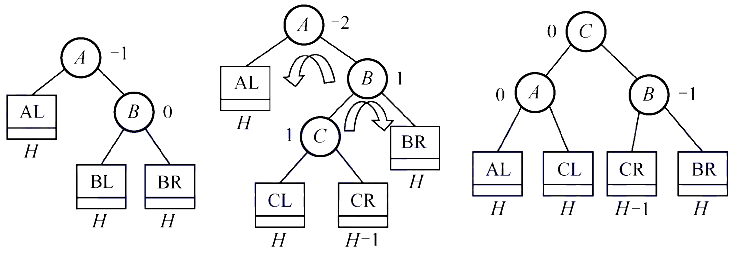
然后把 C 向右上旋转提升到 A 的位置，如图 7.13 所示。



(a) 插入结点前　(b) 插入结点导致不平衡　(c) LR旋转(双旋转)

图 7.13　LR 平衡旋转

4）RL 平衡旋转（先右后左双旋转）。由于在结点 A 的右孩子（R）的左子树（L）上插入新结点，A 的平衡因子由-1 减至-2，导致以 A 为根的子树失去平衡，需要进行两次旋转操作，先右旋转后左旋转。先将 A 的右孩子 B 的左子树的根结点 C 向右上旋转提升到 B 的位置，然后把 C 向左上旋转提升到 A 的位置，如图 7.14 所示。



(a) 插入结点前　(b) 插入结点导致不平衡　(c) RL旋转(双旋转)

图 7.14　RL 平衡旋转

注　意　公众号：小兔网盘　免费分享无水印PDF

LR 和 RL 旋转时，新结点究竟是插入 C 的左子树还是插入 C 的右子树不影响旋转过程，而图 7.13 和图 7.14 中以插入 C 的左子树中为例。

命题追踪　构造平衡二叉树的过程（2013）

以关键字序列{15, 3, 7, 10, 9, 8}构造一棵平衡二叉树的过程为例，图 7.15(d)插入 7 后导致不平衡，最小不平衡子树的根为 15，插入位置为其左孩子的右子树，所以执行 LR 旋转，先左后右双旋转，调整后的结果如图 7.15(e)所示。图 7.15(g)插入 9 后导致不平衡，最小不平衡子树的根为 15，插入位置为其左孩子的左子树，所以执行 LL 旋转，右单旋转，调整后的结果如图 7.15(h)所示。图 7.15(i)插入 8 后导致不平衡，最小不平衡子树的根为 7，插入位置为其右孩子的左子树，所以执行 RL 旋转，先右后左双旋转，调整后的结果如图 7.15(j)所示。

3．平衡二叉树的删除

与平衡二叉树的插入操作类似，以删除结点 w 为例来说明平衡二叉树删除操作的步骤：

1）用二叉排序树的方法对结点 w 执行删除操作。

2）若导致了不平衡，则从结点 w 开始向上回溯，找到第一个不平衡的结点 z（最小不平衡子树）；y 为结点 z 的高度最高的孩子；x 是结点 y 的高度最高的孩子。

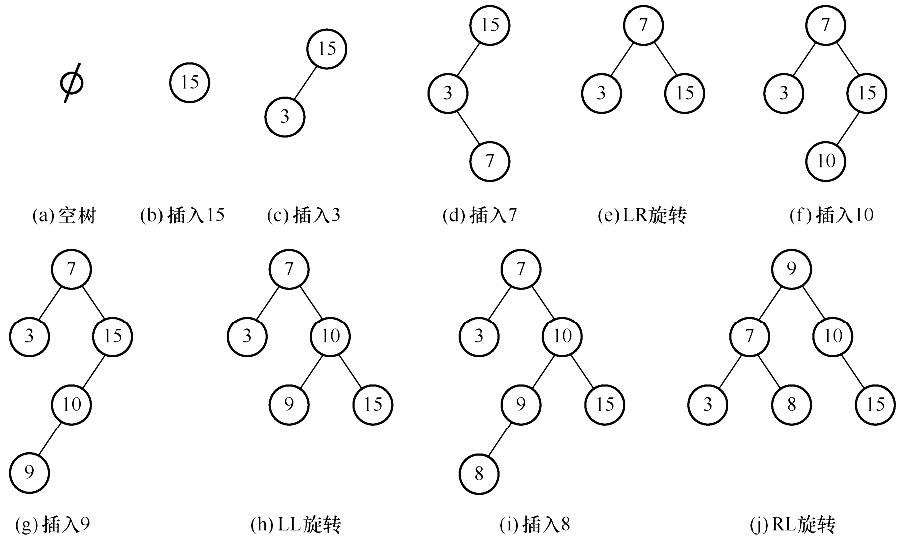


图 7.15　平衡二叉树的生成过程

3）然后对以 z 为根的子树进行平衡调整，其中 x、y 和 z 可能的位置有 4 种情况：

- y 是 z 的左孩子，x 是 y 的左孩子（LL，右单旋转）；
- y 是 z 的左孩子，x 是 y 的右孩子（LR，先左后右双旋转）；
- y 是 z 的右孩子，x 是 y 的右孩子（RR，左单旋转）；
- y 是 z 的右孩子，x 是 y 的左孩子（RL，先右后左双旋转）。

这四种情况与插入操作的调整方式一样。不同之处在于，插入操作仅需要对以 z 为根的子树进行平衡调整；而删除操作就不一样，先对以 z 为根的子树进行平衡调整，若调整后子树的高度减 1，则可能需要对 z 的祖先结点进行平衡调整，甚至回溯到根结点（导致树高减 1）。

以删除图 7.16(a)的结点 32 为例，由于 32 为叶结点，直接删除即可，向上回溯找到第一个不平衡结点 44（z），z 的高度最高的孩子结点为 78（y），y 的高度最高的孩子结点为 50（x），满足 RL 情况，先右后左双旋转，调整后的结果如图 7.16(c)所示。

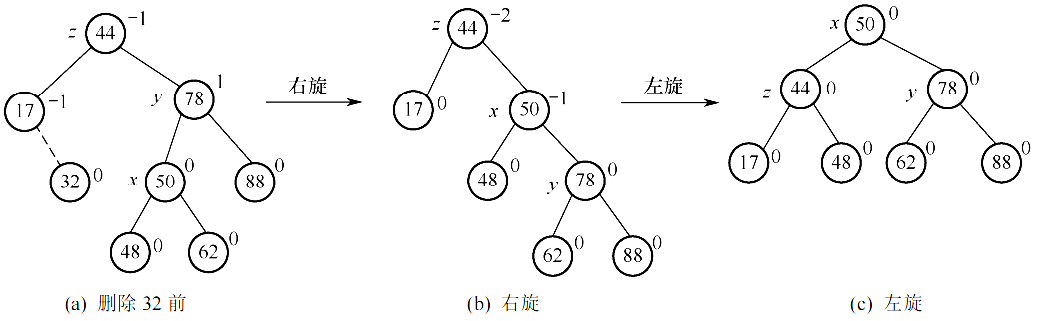


图 7.16　平衡二叉树的删除

4．平衡二叉树的查找

命题追踪 ▶ 指定条件下平衡二叉树的结点数的分析（2012）

在平衡二叉树上进行查找的过程与二叉排序树的相同。因此，在查找过程中，进行关键字的比较次数不超过树的深度。假设以 n_h 表示深度为 h 的平衡二叉树中含有的最少结点数。显然，有

$n_0=0, n_1=1, n_2=2$，并且有 $n_h=n_{h-2}+n_{h-1}+1$，如图 7.17 所示，依次推出 $n_3=4, n_4=7, n_5=12, \cdots$。含有 n 个结点的平衡二叉树的最大深度为 $O(\log_2 n)$，因此平均查找效率为 $O(\log_2 n)$。

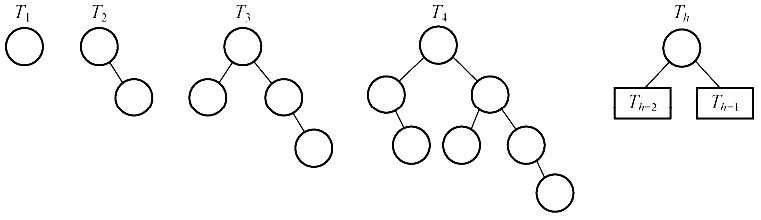


图 7.17　结点个数 n 最少的平衡二叉树

注　意

该结论可用于求解给定结点数的平衡二叉树的查找所需的最多比较次数（或树的最大高度）。如在含有 12 个结点的平衡二叉树中查找某个结点的最多比较次数？

深度为 h 的平衡二叉树中含有的最多结点数显然是满二叉树的情况。

7.3.3　红黑树

1．红黑树的定义

为了保持 AVL 树的平衡性，在插入和删除操作后，会非常频繁地调整全树整体拓扑结构，代价较大。为此在 AVL 树的平衡标准上进一步放宽条件，引入了红黑树的结构。

一棵红黑树是满足如下红黑性质的二叉排序树：

① 每个结点或是红色，或是黑色的。

② 根结点是黑色的。

③ 叶结点（虚构的外部结点、NULL 结点）都是黑色的。

④ 不存在两个相邻的红结点（红结点的父结点和孩子结点均是黑色的）。

⑤ 对每个结点，从该结点到任意一个叶结点的简单路径上，所含黑结点的数量相同。

与折半查找树和 B 树类似，为了便于对红黑树的实现和理解，引入了 $n+1$ 个外部叶结点，以保证红黑树中每个结点（内部结点）的左、右孩子均非空。图 7.18 所示是一棵红黑树。

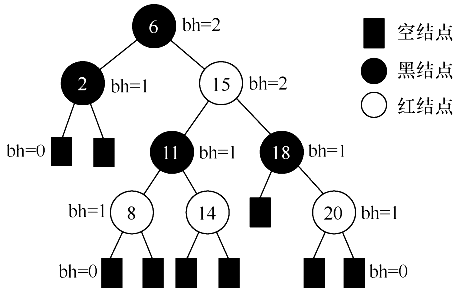


图 7.18　一棵红黑树

从某结点出发（不含该结点）到达一个叶结点的任意一个简单路径上的黑结点总数称为该结点的黑高（记为 bh），黑高的概念是由性质⑤确定的。根结点的黑高称为红黑树的黑高。

结论 1：从根到叶结点的最长路径不大于最短路径的 2 倍。

由性质⑤，当从根到任意一个叶结点的简单路径最短时，这条路径必然全由黑结点构成。由性质④，当某条路径最长时，这条路径必然是由黑结点和红结点相间构成的，此时红结点和

黑结点的数量相同。图 7.18 中的 6—2 和 6—15—18—20 就是这样的两条路径。

结论 2：有 n 个内部结点的红黑树的高度 $h \leqslant 2\log_2(n+1)$。

证明：由结论 1 可知，从根到叶结点（不含叶结点）的任何一条简单路径上都至少有一半是黑结点，因此，根的黑高至少为 $h/2$，于是有 $n \geqslant 2^{h/2}-1$，即可求得结论。

由结论 2 也可推出，黑高为 h 的红黑树的内部结点数最少是 2^h-1，最多是 $2^{2h}-1$。

可见，红黑树的“适度平衡”，由 AVL 树的“高度平衡”，降低到“任意一个结点左右子树的高度，相差不超过 2 倍”，也降低了动态操作时调整的频率。对于一棵动态查找树，若插入和删除操作比较少，查找操作比较多，则采用 AVL 树比较合适，否则采用红黑树更合适。但由于维护这种高度平衡所付出的代价比获得的效益大得多，红黑树的实际应用更广泛，C++中的 `map` 和 `set`（Java 中的 `TreeMap` 和 `TreeSet`）就是用红黑树实现的。

2．红黑树的插入

红黑树的插入过程和二叉查找树的插入过程基本类似，不同之处在于，在红黑树中插入新结点后需要进行调整（主要通过重新着色或旋转操作进行），以满足红黑树的性质。

结论 3：新插入红黑树中的结点初始着为红色。

假设新插入的结点初始着为黑色，则这个结点所在的路径比其他路径多出一个黑结点（几乎每次插入都破坏性质⑤），调整起来也比较麻烦。若插入的结点是红色的，则此时所有路径上的黑结点数量不变，仅在出现连续两个红结点时才需要调整，而且这种调整也比较简单。

设结点 z 为新插入的结点。插入过程描述如下：

1）用二叉查找树插入法插入，并将结点 z 着为红色。若结点 z 的父结点是黑色的，无须做任何调整，此时就是一棵标准的红黑树，结束。

2）若结点 z 是根结点，则将 z 着为黑色（树的黑高增 1），结束。

3）若结点 z 不是根结点，且 z 的父结点 $z.p$ 是红色的，则分为下面三种情况，区别在于 z 的叔结点 y 的颜色不同，因 $z.p$ 是红色的，插入前的树是合法的，根据性质②和④，爷结点 $z.p.p$ 必然存在且为黑色。性质④只在 z 和 $z.p$ 之间被破坏了。

情况 1：z 的叔结点 y 是黑色的，且 z 是一个右孩子。

情况 2：z 的叔结点 y 是黑色的，且 z 是一个左孩子。

每棵子树 T_1、T_2、T_3 和 T_4 都有一个黑色根结点，且具有相同的黑高。

情况 1（LR，先左旋，再右旋），即 z 是其爷结点的左孩子的右孩子。先做一次左旋将此情形转变为情况 2（变为情况 2 后再做一次右旋），左旋后 z 和父结点 $z.p$ 交换位置。因为 z 和 $z.p$ 都是红色的，所以左旋操作对结点的黑高和性质⑤都无影响。

情况 2（LL，右单旋），即 z 是其爷结点的左孩子的左孩子。做一次右旋，并交换 z 的原父结点和原爷结点的颜色，就可以保持性质⑤，也不会改变树的黑高。这样，红黑树中也不再有连续两个红结点，结束。情况 1 和情况 2 的调整方式如图 7.19 所示。

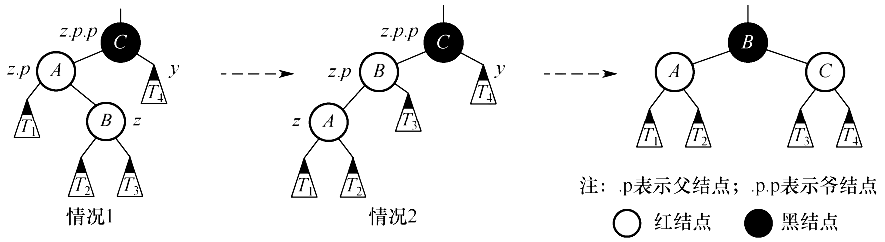


图 7.19　情况 1 和情况 2 的调整方式

若父结点 *z.p* 是爷结点 *z.p.p* 的右孩子，则还有两种对称的情况：RL（先右旋，再左旋）和 RR（左单旋），这里不再赘述。红黑树的调整方法和 AVL 树的调整方法有异曲同工之妙。

情况 3：*z* 的叔结点 *y* 是红色的。

情况 3（*z* 是左孩子或右孩子无影响），*z* 的父结点 *z.p* 和叔结点 *y* 都是红色的，因为爷结点 *z.p.p* 是黑色的，将 *z.p* 和 y 都着为黑色，将 *z.p.p* 着为红色，以在局部保持性质④和⑤。然后，把 *z.p.p* 作为新结点 *z* 来重复循环，指针 *z* 在树中上移两层。调整方式如图 7.20 所示。

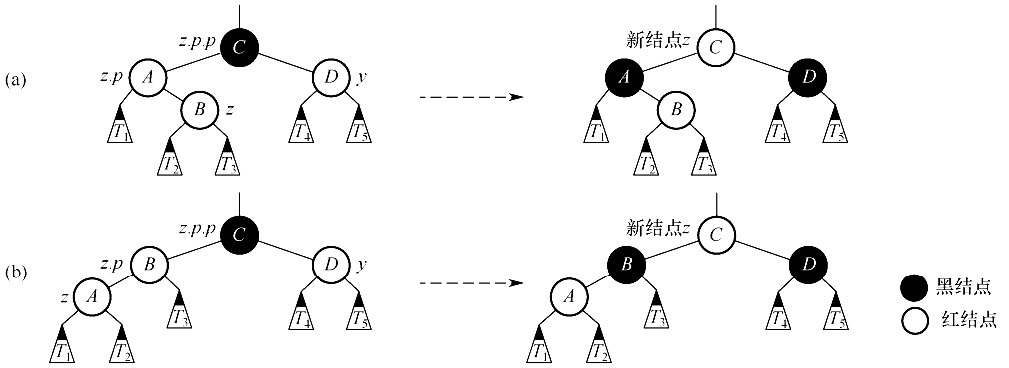


图 7.20　情况 3 的调整方式

若父结点 *z.p* 是爷结点 *z.p.p* 的右孩子，也还有两种对称的情况，不再赘述。

只要满足情况 3 的条件，就会不断循环，每次循环指针 *z* 都会上移两层，直到满足 2）（表示 *z* 上移到根结点）或情况 1 或情况 2 的条件。

可能的疑问：虽然插入的初始位置一定是红黑树的某个叶结点，但因为在情况 3 中，结点 *z* 存在不断上升的可能，所以对于三种情况，结点 *z* 都有存在子树的可能。

以图 7.21(a)中的红黑树为例（虚线表示插入后的状态），先后插入 5、4 和 12 的过程如图 7.21 所示。插入 5，为情况 3，将 5 的父结点 3 和叔结点 10 着为黑色，将 5 的爷结点变为红色，此时因为 7 已是根，所以又重新着为黑色，树的黑高加 1，结束。插入 4，为情况 1 的对称情况（RL），此时特别注意虚构黑色空结点的存在，先对以 5 为根的子树做右旋；转变为情况 2 的对称情况（RR），交换 3 和 4 的颜色，再对以 4 为根的子树做左旋，结束。插入 12，父结点是黑色的，无须任何调整，结束。

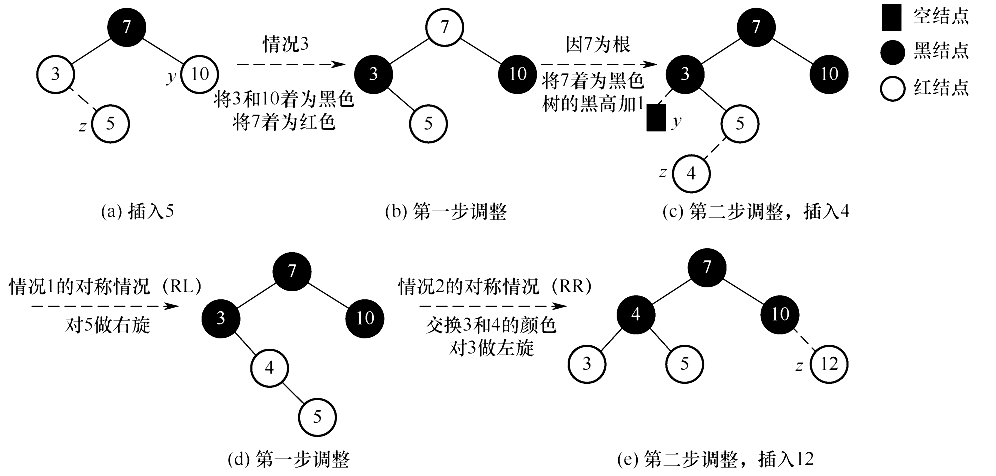


图 7.21　红黑树的插入过程

*3．红黑树的删除

注 意

本节难度较大，考查概率较低，读者可根据自身情况决定是否学习或学习的时机。

红黑树的插入操作容易导致连续的两个红结点，破坏性质④。而删除操作容易造成子树黑高的变化（删除黑结点会导致根结点到叶结点间的黑结点数量减少），破坏性质⑤。

删除过程也是先执行二叉查找树的删除方法。若待删结点有两个孩子，不能直接删除，而要找到该结点的中序后继（或前驱）填补，即右子树中最小的结点，然后转换为删除该后继结点。由于后继结点至多只有一个孩子，这样就转换为待删结点是终端结点或仅有一个孩子的情况。

最终，删除一个结点有以下两种情况：

- 待删结点只有右子树或左子树。
- 待删结点没有孩子。

1）若待删结点只有右子树或左子树，则只有两种情况，如图 7.22 所示。

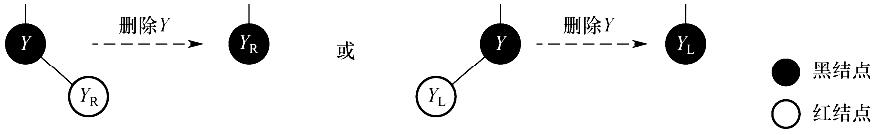


图 7.22 只有右子树或左子树的删除情况

只有这两种情况存在。子树只有一个结点，且必然是红色，否则会破坏性质⑤。

2）待删结点无孩子，且该结点是红色的，这时可直接删除，而不需要做任何调整。

3）待删结点无孩子，且该结点是黑色的，这时设待删结点为 y，x 是用来替换 y 的结点（注意，当 y 是终端结点时，x 是黑色的 NULL 结点）。删除 y 后将导致先前包含 y 的任何路径上的黑结点数量减 1，因此 y 的任何祖先都不再满足性质⑤，简单的修正办法就是将替换 y 的结点 x 视为还有额外一重黑色，定义为双黑结点。也就是说，若将任何包含结点 x 的路径上的黑结点数量加 1，则在此假设下，性质⑤得到满足，但破坏了性质①。于是，删除操作的任务就转化为将双黑结点恢复为普通结点。

分为以下四种情况，区别在于 x 的兄弟结点 w 及 w 的孩子结点的颜色不同。

情况 1：x 的兄弟结点 w 是红色的。

情况 1，w 必须有黑色左右孩子和父结点。交换 w 和父结点 $x.p$ 的颜色，然后对 $x.p$ 做一次左旋，而不会破坏红黑树的任何规则。现在，x 的新兄弟结点是旋转之前 w 的某个孩子结点，其颜色为黑色，这样，就将情况 1 转换为情况 2、3 或 4 处理。调整方式如图 7.23 所示。

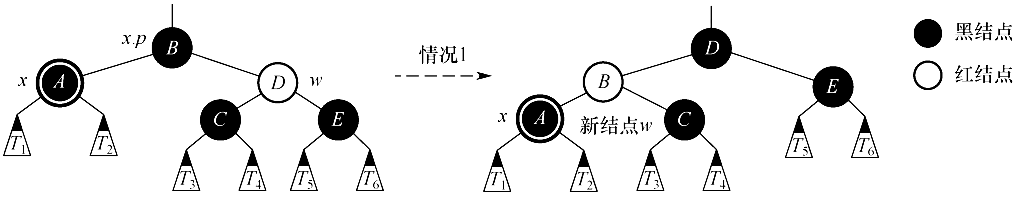


图 7.23 情况 1 的调整方式

情况 2：x 的兄弟结点 w 是黑色的，且 w 的右孩子是红色的。

情况 3：x 的兄弟结点 w 是黑色的，w 的左孩子是红色的，w 的右孩子是黑色的。

情况 2（RR，左单旋），即这个红结点是其爷结点的右孩子的右孩子。交换 w 和父结点 $x.p$

的颜色，把 w 的右孩子着为黑色，并对 x 的父结点 $x.p$ 做一次左旋，将 x 变为单重黑色，此时不再破坏红黑树的任何性质，结束。调整方式如图 7.24 所示。

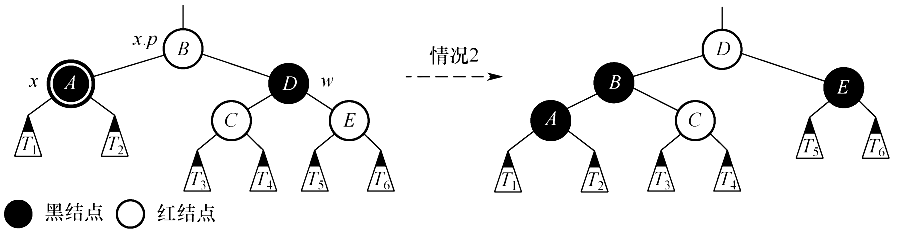


图 7.24　情况 2 的调整方式

情况 3（RL，先右旋，再左旋），即这个红结点是其爷结点的右孩子的左孩子。交换 w 和其左孩子的颜色，然后对 w 做一次右旋，而不破坏红黑树的任何性质。现在，x 的新兄弟结点 w 的右孩子是红色的，这样就将情况 3 转换为了情况 2。调整方式如图 7.25 所示。

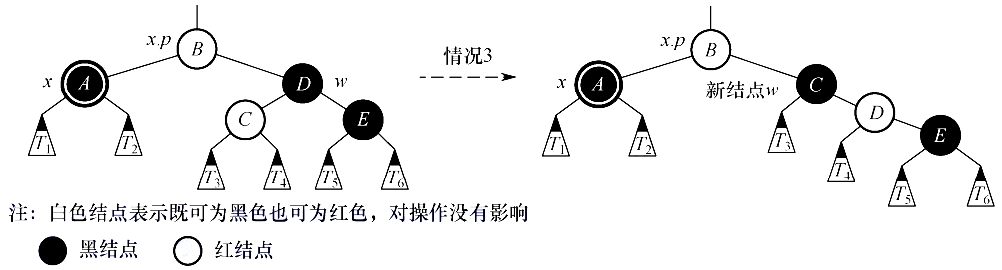


图 7.25　情况 3 的调整方式

情况 4：x 的兄弟结点 w 是黑色的，且 w 的两个孩子结点都是黑色的。

在情况 4 中，因为 w 也是黑色的，所以可从 x 和 w 上去掉一重黑色，使得 x 只有一重黑色而 w 变为红色。为了补偿从 x 和 w 中去掉的一重黑色，把 x 的父结点 $x.p$ 额外着一层黑色，以保持局部的黑高不变。通过将 $x.p$ 作为新结点 x 来循环，x 上升一层。若是通过情况 1 进入情况 4 的，因为原来的 $x.p$ 是红色的，将新结点 x 变为黑色，终止循环，结束。调整方式如图 7.26 所示。

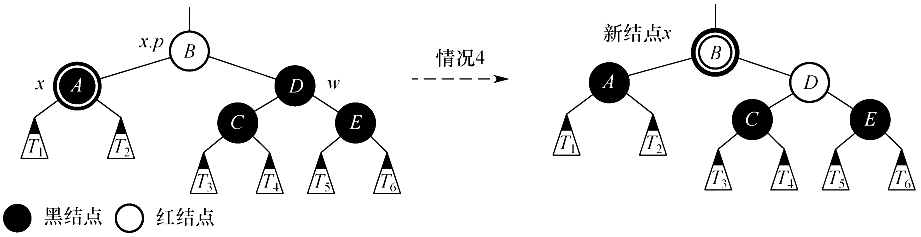


图 7.26　情况 4 的调整方式

若 x 是父结点 $x.p$ 的右孩子，则还有四种对称的情况，处理方式类似，不再赘述。

归纳总结：在情况 4 中，因 x 的兄弟结点 w 及左右孩子都是黑色，可以从 x 和 w 中各提取一重黑色（以让 x 变为普通黑结点），不会破坏性质④，并把调整任务向上“推”给它们的父结点 $x.p$。在情况 1、2 和 3 中，因为 x 的兄弟结点 w 或 w 左右孩子中有红结点，所以只能在 $x.p$ 子树内用调整和重新着色的方式，且不能改变 x 原根结点的颜色（否则向上可能破坏性质④）。情况 1 虽然可能会转换为情况 4，但因为新 x 的父结点 $x.p$ 是红色的，所以执行一次情况 4 就会结束。情况 1、2 和 3 在各执行常数次的颜色改变和至多 3 次旋转后便终止，情况 4 是可能重复执行的唯

一情况，每执行一次指针 x 上升一层，至多 $O(\log_2 n)$次。

以图 7.27(a)中的红黑树为例（虚线表示删除前的状态），依次删除 5 和 15 的过程如图 7.27 所示。删除 5，用虚构的黑色 NULL 结点（图中为 N 结点）替换，视为双黑 NULL 结点，为情况 1，交换兄弟结点 12 和父结点 8 的颜色，对 8 做一次左旋；转变为情况 4，从双黑 NULL 结点和 10 中各提取一重黑色（提取后，双黑 NULL 结点变为普通 NULL 结点，图中省略，10 变为红色），因原父结点 8 是红色，所以将 8 变为黑色，结束。删除 15，为情况 3 的对称情况（LR），交换 8 和 10 的颜色，对 8 做左旋；转变为情况 2 的对称情况（LL），交换 10 和 12 的颜色（两者颜色一样，无变化），将 10 的左孩子 8 着为黑色，对 12 做右旋，结束。

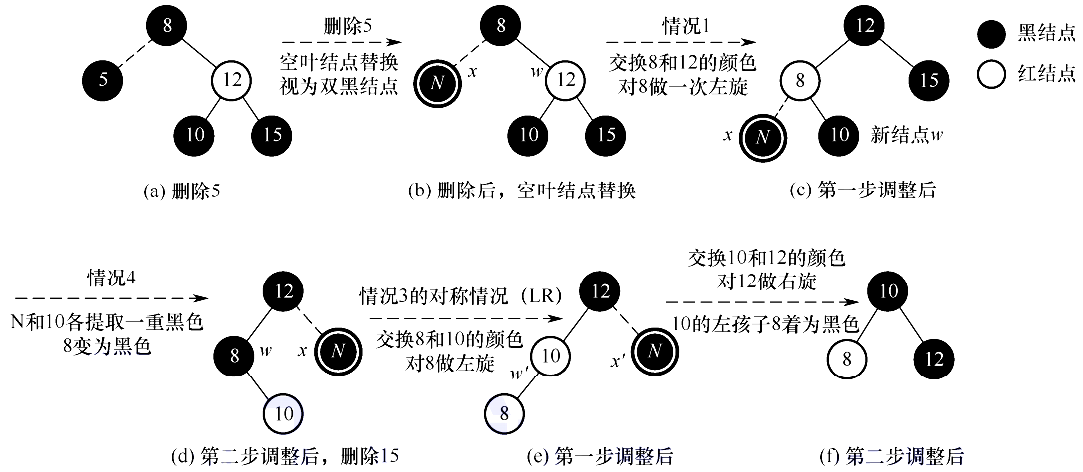


图 7.27　红黑树的删除过程

7.3.4　本节试题精选

一、单项选择题

01. 对于二叉排序树，下面的说法中，（　）是正确的。

A. 二叉排序树是动态树表，查找失败时插入新结点，会引起树的重新分裂和组合

B. 对二叉排序树进行层序遍历可得到有序序列

C. 用逐点插入法构造二叉排序树，若先后插入的关键字有序，二叉排序树的深度最大

D. 在二叉排序树中进行查找，关键字的比较次数不超过结点数的 1/2

02. 按（　）遍历二叉排序树得到的序列是一个有序序列。

A. 先序　　B. 中序　　C. 后序　　D. 层次

03. 在二叉排序树中进行查找的效率与（　）有关。

A. 二叉排序树的深度　　B. 二叉排序树的结点的个数

C. 被查找结点的度　　D. 二叉排序树的存储结构

04. 在常用的描述二叉排序树的存储结构中，关键字值最大的结点（　）。

A. 左指针一定为空　　B. 右指针一定为空

C. 左右指针均为空　　D. 左右指针均不为空

05. 设二叉排序树中关键字由 1 到 1000 的整数构成，现要查找关键字为 363 的结点，下述关键字序列中，不可能是在二叉排序树上查找的序列是（　）。

A. 2, 252, 401, 398, 330, 344, 397, 363　　B. 924, 220, 911, 244, 898, 258, 362, 363

C. 925, 202, 911, 240, 912, 245, 363　　D. 2, 399, 387, 219, 266, 382, 381, 278, 363

06. 分别以下列序列构造二叉排序树，与用其他 3 个序列所构造的结果不同的是（　）。

A. (100, 80, 90, 60, 120, 110, 130)　　B. (100, 120, 110, 130, 80, 60, 90)

C. (100, 60, 80, 90, 120, 110, 130)　　D. (100, 80, 60, 90, 120, 130, 110)

07. 从空树开始，依次插入元素 52, 26, 14, 32, 71, 60, 93, 58, 24 和 41 后构成了一棵二叉排序树。在该树查找 60 要进行比较的次数为（　）。

A. 3　　B. 4　　C. 5　　D. 6

08. 在含有 n 个结点的二叉排序树中查找某个关键字的结点时，最多进行（　）次比较。

A. $n/2$　　B. $\log_2 n$　　C. $\log_2 n+1$　　D. n

09. 五个不同结点构造的二叉查找树的形态共有（　）种。

A. 20　　B. 30　　C. 32　　D. 42

10. 构造一棵具有 n 个结点的二叉排序树时，最理想情况下的深度为（　）。

A. $n/2$　　B. n　　C. $\lfloor \log_2(n+1) \rfloor$　　D. $\lceil \log_2(n+1) \rceil$

11. 含有 20 个结点的平衡二叉树的最大深度为（　）。

A. 4　　B. 5　　C. 6　　D. 7

12. 具有 5 层结点的平衡二叉树至少有（　）个结点。

A. 10　　B. 12　　C. 15　　D. 17

13. 高度为 3 的平衡二叉排序树的形态共有（　）种。

A. 13　　B. 14　　C. 16　　D. 15

14. 在平衡二叉树的基本操作中，可能发生两次旋转的操作是（　）。

A. 添加、删除结点　B. 仅删除结点　　C. 仅添加结点　　D. 都不会

15. 将关键字 1, 2, 3, …, 1024 依次插入到初始为空的平衡二叉树中，假设只有一个根结点的二叉树的高度为 0，则插入结束后的平衡二叉树的高度是（　）。

A. 8　　B. 9　　C. 10　　D. 11

16. 下列关于红黑树和 AVL 树的说法中，不正确的是（　）。

I. 一棵含有 n 个结点的红黑树的高度至多为 $2\log_2(n+1)$

II. 若一个结点是红色的，则它的父结点和孩子结点都是黑色的

III. 红黑树的查询效率一般要优于含有相同结点数的 AVL 树

IV. 若 AVL 树的某结点的左右孩子的平衡因子都是零，则该结点的平衡因子也是零

A. I、III　　B. III　　C. II、IV　　D. III、IV

17. 下列关于红黑树和 AVL 树的描述中，不正确的是（　）。

A. 两者都属于自平衡的二叉树

B. 两者查找、插入、删除的时间复杂度都相同

C. 红黑树插入和删除过程至多有 2 次旋转操作

D. 红黑树的任意一个结点的左右子树高度（含叶结点）之比不超过 2

18. 下列关于红黑树的说法中，正确的是（　）。

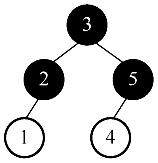
A. 红黑树的红结点的数目最多和黑结点的数目相同

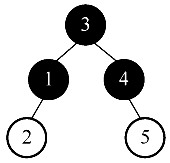
B. 若红黑树的所有结点都是黑色的，则它一定是一棵满二叉树

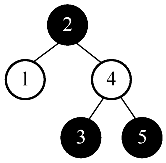
C. 红黑树的任何一个分支结点都有两个非空孩子结点

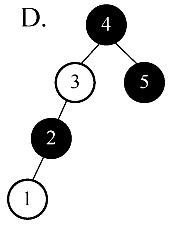
D. 红黑树的子树也一定是红黑树

19. 下列四个选项中，满足红黑树定义的是（　）。

A. 

B. 

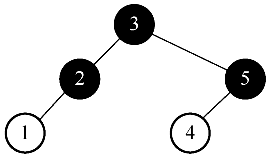
C. 

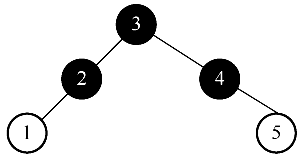
D. 

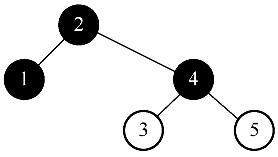
20. 将关键字 1, 2, 3, 4, 5, 6, 7 依次插入初始为空的红黑树 T，则 T 中红结点的个数是（ ）。

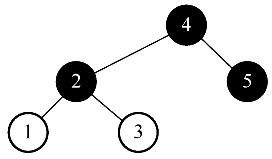
A. 1　　B. 2　　C. 3　　D. 4

21. 将关键字 5, 4, 3, 2, 1 依次插入初始为空的红黑树 T，则 T 的最终形态是（ ）。

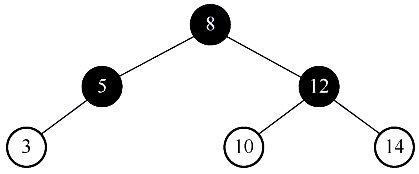
A. 

B. 

C. 

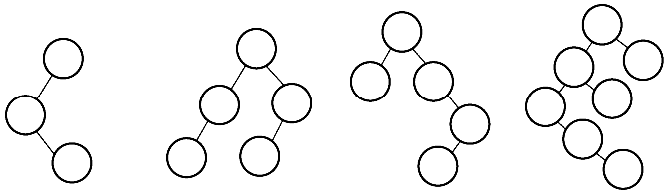
D. 

22. 在下图所示的红黑树中插入结点 2 且染成红色后，则下一步应进行的操作是（ ）。



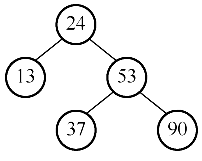
A. 左旋　　B. 右旋　　C. 变色　　D. 无须调整

23. 【2009 统考真题】下列二叉排序树中，满足平衡二叉树定义的是（ ）。



A.　　B.　　C.　　D.

24. 【2010 统考真题】在下图所示的平衡二叉树中插入关键字 48 后得到一棵新平衡二叉树，在新平衡二叉树中，关键字 37 所在结点的左、右子结点中保存的关键字分别是（ ）。



A. 13, 48　　B. 24, 48　　C. 24, 53　　D. 24, 90

25. 【2011 统考真题】对下列关键字序列，不可能构成某二叉排序树中一条查找路径的是（ ）。

A. 95, 22, 91, 24, 94, 71　　B. 92, 20, 91, 34, 88, 35

C. 21, 89, 77, 29, 36, 38　　D. 12, 25, 71, 68, 33, 34

26. 【2012 统考真题】若平衡二叉树的高度为 6，且所有非叶结点的平衡因子均为 1，则该平衡二叉树的结点总数为（　）。

A. 12　　B. 20　　C. 32　　D. 33

27. 【2013 统考真题】在任意一棵非空二叉排序树 T_1 中，删除某结点 v 之后形成二叉排序树 T_2，再将 v 插入 T_2 形成二叉排序树 T_3。下列关于 T_1 与 T_3 的叙述中，正确的是（　）。

I. 若 v 是 T_1 的叶结点，则 T_1 与 T_3 不同

II. 若 v 是 T_1 的叶结点，则 T_1 与 T_3 相同

III. 若 v 不是 T_1 的叶结点，则 T_1 与 T_3 不同

IV. 若 v 不是 T_1 的叶结点，则 T_1 与 T_3 相同

A. 仅 I、III　　B. 仅 I、IV　　C. 仅 II、III　　D. 仅 II、IV

28. 【2013 统考真题】若将关键字 1, 2, 3, 4, 5, 6, 7 依次插入初始为空的平衡二叉树 T，则 T 中平衡因子为 0 的分支结点的个数是（　）。

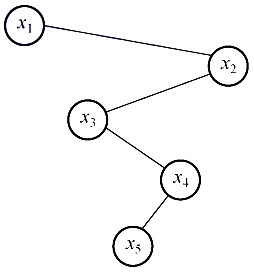
A. 0　　B. 1　　C. 2　　D. 3

29. 【2015 统考真题】现有一棵无重复关键字的平衡二叉树（AVL），对其进行中序遍历可得到一个降序序列。下列关于该平衡二叉树的叙述中，正确的是（　）。

A. 根结点的度一定为 2　　B. 树中最小元素一定是叶结点

C. 最后插入的元素一定是叶结点　　D. 树中最大元素一定是无左子树

30. 【2018 统考真题】已知二叉排序树如下图所示，元素之间应满足的大小关系是（　）。



A. $x_1 < x_2 < x_5$　　B. $x_1 < x_4 < x_5$　　C. $x_3 < x_5 < x_4$　　D. $x_4 < x_3 < x_5$

31. 【2019 统考真题】在任意一棵非空平衡二叉树（AVL 树）T_1 中，删除某结点 v 之后形成平衡二叉树 T_2，再将 v 插入 T_2 形成平衡二叉树 T_3。下列关于 T_1 与 T_3 的叙述中，正确的是（　）。

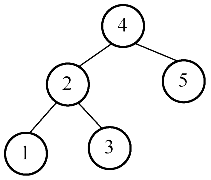
I. 若 v 是 T_1 的叶结点，则 T_1 与 T_3 可能不相同

II. 若 v 不是 T_1 的叶结点，则 T_1 与 T_3 一定不相同

III. 若 v 不是 T_1 的叶结点，则 T_1 与 T_3 一定相同

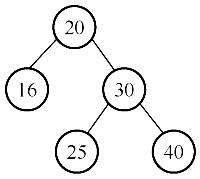
A. 仅 I　　B. 仅 II　　C. 仅 I、II　　D. 仅 I、III

32. 【2020 统考真题】下列给定的关键字输入序列中，不能生成右边二叉排序树的是（　）。



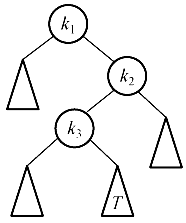
A. 4, 5, 2, 1, 3　　B. 4, 5, 1, 2, 3　　C. 4, 2, 5, 3, 1　　D. 4, 2, 1, 3, 5

33.【2021统考真题】给定平衡二叉树如下图所示，插入关键字23后，根中的关键字是（ ）。



A. 16　　B. 20　　C. 23　　D. 25

34.【2024统考真题】一棵二叉搜索树如下图所示，k_1、k_2、k_3分别是对应结点中保存的关键字。子树T的任一结点中保存的关键字x满足的是（ ）。



A. $x<k_1$　　B. $x>k_2$　　C. $k_1<x<k_3$　　D. $k_3<x<k_2$

二、综合应用题

01. 一棵二叉排序树按先序遍历得到的序列为(50, 38, 30, 45, 40, 48, 70, 60, 75, 80)，试画出该二叉排序树，并求出等概率下查找成功和查找失败的平均查找长度。

02. 按照序列(40, 72, 38, 35, 67, 51, 90, 8, 55, 21)建立一棵二叉排序树，画出该树，并求出在等概率的情况下，查找成功的平均查找长度。

03. 依次把结点(34, 23, 15, 98, 115, 28, 107)插入初始状态为空的平衡二叉排序树，使得在每次插入后保持该树仍然是平衡二叉树。请依次画出每次插入后所形成的平衡二叉排序树。

04. 给定一个关键字集合{25, 18, 34, 9, 14, 27, 42, 51, 38}，假定查找各关键字的概率相同，请画出其最佳二叉排序树。

05. 试编写一个算法，判断给定的二叉树是否是二叉排序树。

06. 设计一个算法，求出指定结点在给定二叉排序树中的层次。

07. 利用二叉树遍历的思想编写一个判断二叉树是否是平衡二叉树的算法。

08. 设计一个算法，求出给定二叉排序树中最小和最大的关键字。

09. 设计一个算法，从大到小输出二叉排序树中所有值不小于k的关键字。

10. 编写一个递归算法，在一棵有n个结点的、随机建立起来的二叉排序树上查找第k（$1\leqslant k\leqslant n$）小的元素，并返回指向该结点的指针。要求算法的平均时间复杂度为$O(\log_2 n)$。二叉排序树的每个结点中除`data`、`lchild`、`rchild`等数据成员外，增加一个`count`成员，保存以该结点为根的子树上的结点个数。

7.3.5 答案与解析

一、单项选择题

01. C

二叉排序树插入新结点时不会引起树的分裂组合。对二叉排序树进行中序遍历可得到有序序列。当插入的关键字有序时，二叉排序树会形成一个长链，此时深度最大。在此种情况下进行查找，有可能需要比较每个结点的关键字，超过总结点数的1/2。

02．B

由二叉排序树的定义不难得出中序遍历二叉树得到的序列是一个有序序列。

03．A

二叉排序树的查找路径是自顶向下的，其平均查找长度主要取决于树的高度。

04．B

在二叉排序树的存储结构中，每个结点由三部分构成，其中左（或右）指针指向比该结点的关键字值小（或大）的结点。关键字值最大的结点位于二叉排序树的最右位置，因此它的右指针一定为空（有可能不是叶结点）。还可用反证法，若右指针不为空，则右指针上的关键字肯定比原关键字大，所以原关键字结点一定不是值最大的，与条件矛盾，所以右指针一定为空。

05．C

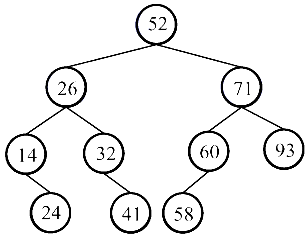
在二叉排序树上查找时，先与根结点值进行比较，若相同，则查找结束，否则根据比较结果，沿着左子树或右子树向下继续查找。根据二叉排序树的定义，有左子树结点值≤根结点值≤右子树结点值。C 序列中，比较 911 关键字后，应转向其左子树比较 240，左子树中不应出现比 911 更大的数值，但 240 竟有一个右孩子结点值为 912，所以不可能是正确的序列。

06．C

按照二叉排序树的构造方法，不难得到 A, B, D 序列的构造结果相同。

07．A

以第一个元素为根结点，依次将元素插入树，生成的二叉排序树如下图所示。进行查找时，先与根结点比较，然后根据比较结果，继续在左子树或右子树上进行查找。比较的结点依次为 52, 71, 60。



08．D

当输入序列是一个有序序列时，构造的二叉排序树是一个单支树，当查找一个不存在的关键字值或最后一个结点的关键字值时，需要 n 次比较。

09．D

五个不同结点构造的二叉查找树，中序序列是确定的。先序序列的个数为 $n=5$ 的卡特兰数，加上中序序列和先序序列能唯一确定一棵二叉树，因此二叉排序树的形态共有 Catalan(5) = 42 种。

10．D

当二叉排序树的叶结点全部都在相邻的两层内时，深度最小。理想情况是从第一层到倒数第二层为满二叉树。类比完全二叉树，可得深度为 $\lceil \log_2(n+1) \rceil$。

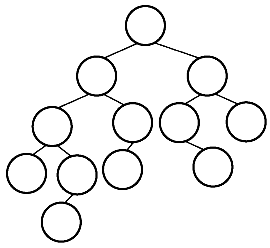
11．C

平衡二叉树结点数的递推公式为 $n_0=0$，$n_1=1$，$n_2=2$，$n_h=1+n_{h-1}+n_{h-2}$（h 为平衡二叉树高度，n_h 为构造此高度的平衡二叉树所需的最少结点数）。通过递推公式可得，构造 5 层平衡二叉树至少需 12 个结点，构造 6 层至少需要 20 个结点。

12．B

设 n_h 表示高度为 h 的平衡二叉树中含有的最少结点数，则有 $n_1=1, n_2=2, n_h=n_{h-1}+n_{h-2}+1$，由

此求出 $n_5=12$，对应的 AVL 如下图所示。



13．D

高度为 3 的平衡二叉树的左右子树的高度共有三种情况：①左右子树都是高度为 2 的平衡二叉树；②左子树是高度为 1 的平衡二叉树，右子树是高度为 2 的平衡二叉树；③左子树是高度为 2 的平衡二叉树，右子树是高度为 1 的平衡二叉树。高度为 1 的平衡二叉树只有 1 种形态，即单个结点，如图 1 所示；高度为 2 的平衡二叉树有 3 种形态，如图 2 所示。

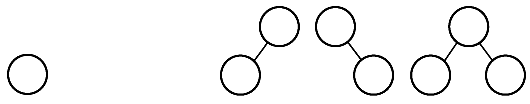


图 1　　图 2

因此，对于情况①，共有 3×3 = 9 种树形态；对于情况②，共有 1×3 = 3 种树形态；情况③和情况②类似，也有 3 种树形态，所以共有 9 + 3 + 3 = 15 种树形态。

14．A

插入和删除结点都有可能引起 LR 平衡旋转或者 RL 平衡旋转，发生两次旋转操作。

15．C

当按关键字有序的顺序插入初始为空的平衡二叉树时，若关键字个数 $n=2^k-1$ 时，则该平衡二叉树一定是一棵满二叉树（可以用 1～3、1～7 手工验证）。当插入关键字 1023 时，平衡二叉树正好是一棵满二叉树，高度是 9。因此，插入关键字 1024 后，平衡二叉树的高度是 10。

16．D

I 和 II 都是红黑树的性质。AVL 是高度平衡的二叉查找树，红黑树是适度平衡的二叉查找树，从这一点也可以看出 AVL 的查询效率往往更优，III 错误。AVL 的某结点的左右孩子的平衡因子都是零，并不能说明左右子树的高度相等，因此该结点的平衡因子不一定为零，IV 错误。

17．C

自平衡的二叉排序树是指在插入和删除时能自动调整以保持其所定义的平衡性，两者都属于自平衡二叉树，选项 A 正确。两者的查找、插入、删除操作的时间复杂度都为 $O(\log_2 n)$，选项 B 正确。在红黑树中删除结点时，情况 1 可能变为情况 2、3 或 4，情况 2 会变为情况 3，可能会出现旋转次数超过 2 次的情况，选项 C 错误。从任一结点到每个叶结点的所有路径都包含相同数目的黑结点，没有两个连续的红结点，且叶结点是黑色的，这意味着在任一结点到其左右子树中最远和最近的叶结点之间，红结点的数目小于或等于黑结点的数目，路径长度之比不超过 2，选项 D 正确。

18．B

红黑树的红结点数目最大可以是黑结点数目的 2 倍（如一棵有 3 个结点的红黑树，第 1 层为黑色，第 2 层为红色），选项 A 错误。从根结点出发到所有叶结点的黑结点数是相同的，若所有结

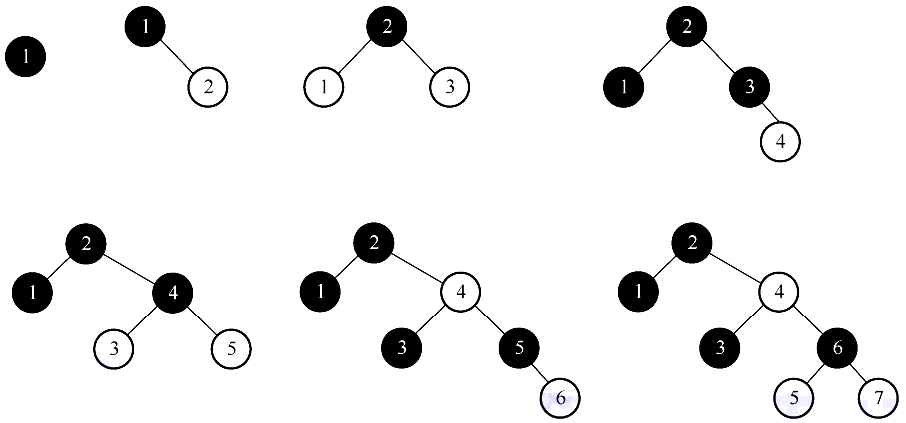
点都是黑色，则一定是满二叉树，选项 B 正确。考虑某个黑结点，它可以有一个空叶结点孩子和一个非空红结点孩子，选项 C 错误。红黑树中可能存在红结点，根结点为红色的子树不是红黑树，选项 D 错误。

19．A

红黑树是一种特殊的二叉排序树，选项 B 不满足二叉排序树的性质。选项 C 中，结点 2 的左右黑结点数不同。在选项 D 中，结点 3 的左右黑结点数不同。只有选项 A 满足红黑树的定义。

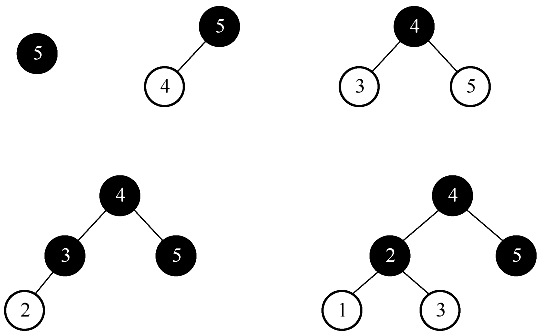
20．C

关键字 1, 2, 3, 4, 5, 6, 7 依次插入红黑树后的形态变化如下：



21．D

关键字 5, 4, 3, 2, 1 依次插入红黑树后的形态变化如下：



22．B

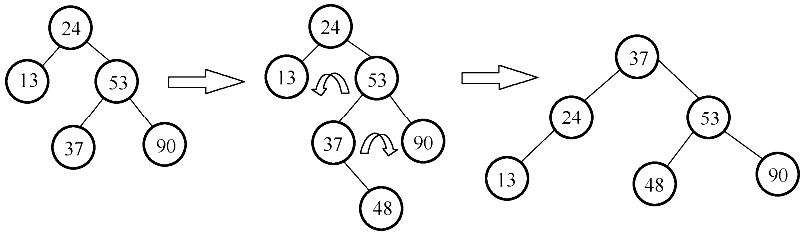
插入结点 2 且将其染成红色后违反不红红原则，并且叔结点是黑色，应进行 LL 旋转，将结点 3 右旋旋转到结点 5 的位置，结点 2 和结点 5 分别成为结点 3 的左、右孩子，然后将结点 3 染成黑色，结点 2 和结点 5 染成红色。因此，下一步应进行右旋操作。

23．B

根据平衡二叉树的定义，任意结点的左、右子树高度差的绝对值不超过 1。而其余 3 个答案均可以找到不满足条件的结点。答题时可以把每个非叶结点的平衡因子都写出来。

24．C

插入 48 以后，该二叉树根结点的平衡因子由 –1 变为 –2，在最小不平衡子树根结点的右子树（R）的左子树（L）中插入新结点引起的不平衡属于 RL 型平衡旋转（先右旋后左旋）。

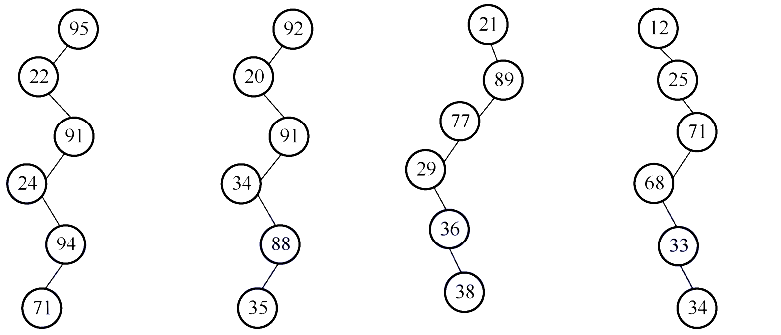


调整后，关键字 37 所在结点的左、右子结点中保存的关键字分别是 24、53。

25．A

在二叉排序树中，左子树结点值小于根结点，右子树结点值大于根结点。在选项 A 中，当查找到 91 后再向 24 查找，说明这一条路径（左子树）之后查找的数都要比 91 小，而后面却查找到了 94（解题过程中，建议配合画图），因此错误。

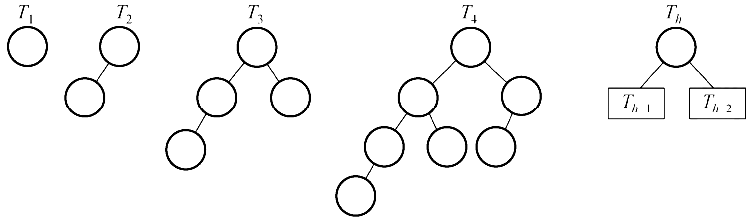
画图法：各选项对应的查找过如下图，选项 B、C、D 对应的查找树都是二叉排序树，选项 A 对应的查找树不是二叉排序树，因为在 91 为根的左子树中出现了比 91 大点的结点 94。



(a) 选项A的查找过程　(b) 选项B的查找过程　(c) 选项C的查找过程　(d) 选项D的查找过程

26．B

所有非叶结点的平衡因子均为 1，即平衡二叉树满足平衡的最少结点情况，如下图所示。对于高度为 n、左右子树的高度分别为 $n-1$ 和 $n-2$、所有非叶结点的平衡因子均为 1 的平衡二叉树，计算总结点数的公式为 $C_n = C_{n-1} + C_{n-2} + 1$, $C_1 = 1$, $C_2 = 2$, $C_3 = 2 + 1 + 1 = 4$，可推出 $C_6 = 20$。



画图法：先画出 T_1 和 T_2；然后新建一个根结点，连接 T_2、T_1 构成 T_3；新建一个根结点，连接 T_3、T_2 构成 T_4……直到画出 T_6，可知 T_6 的结点数为 20。

排除法：对于选项 A，高度为 6、结点数为 12 的树怎么也无法达到平衡。对于选项 C，结点较多时，考虑较极端的情形，即第 6 层只有最左叶子的完全二叉树刚好有 32 个结点，虽然满足平衡的条件，但显然再删去部分结点依然不影响平衡，不是最少结点的情况。同理，选项 D 错误。

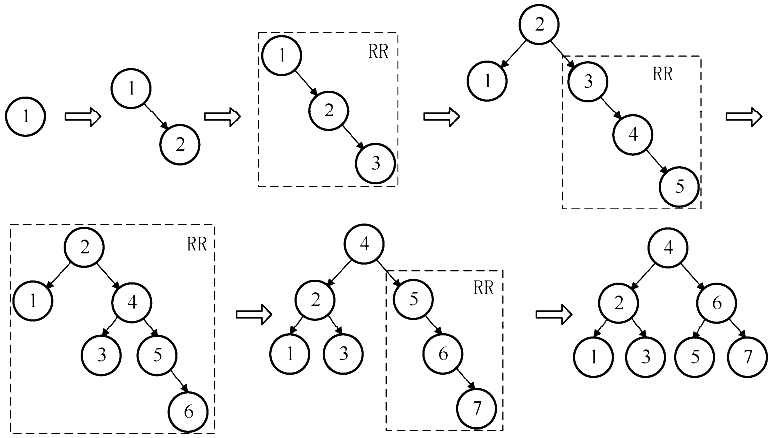
27．C

由于在二叉排序树中插入结点的位置是一个新的叶结点，若删除的是叶结点，则重新插入后得到的二叉排序树与原来的二叉排序树相同。若删除的是非叶结点，在删除过程中会找其他结点

填补，重新插入后变成叶结点，则得到的二叉排序树与原来的二叉排序树不同。

28．D

利用 7 个关键字构建平衡二叉树 T，平衡因子为 0 的分支结点个数为 3，构建的平衡二叉树及构造与调整过程如下图所示。



29．D

大多数教材将平衡二叉树定义为一种高度平衡的二叉排序树，二叉排序树的中序序列是一个升序序列，而题意正好相反。由此可知，命题老师认为平衡二叉树仅为一棵满足高度平衡的二叉树，不一定是二叉排序树。只有两个结点的平衡二叉树的根结点的度为 1，选项 A 错误。中序遍历后得到一个降序序列（与二叉排序树正好相反），树中最大元素一定无左子树（可能有右子树），这与二叉排序树也正好相反，也因此不一定是叶结点，选项 B 错误，选项 D 正确。最后插入的结点可能会导致平衡调整，而不一定是叶结点，选项 C 错误。

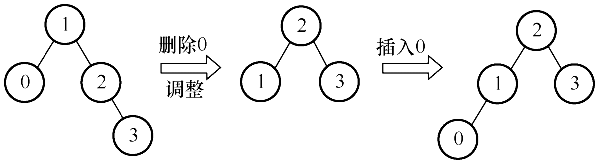
30．C

根据二叉排序树的特性：中序遍历（LNR）得到的是一个递增序列。图中二叉排序树的中序遍历序列为 x_1, x_3, x_5, x_4, x_2，可知 $x_3 < x_5 < x_4$。

31．A

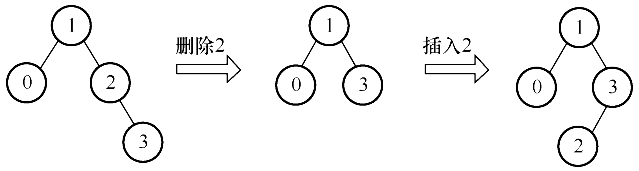
在非空平衡二叉树中插入结点，在失去平衡调整前，一定插入在叶结点的位置。

若删除的是 T_1 的叶结点，则删除后平衡二叉树可能不会失去平衡，即不会发生调整，再插入此结点得到的二叉平衡树 T_1 与 T_3 相同；若删除后平衡二叉树失去平衡而发生调整，再插入结点得到的二叉平衡树 T_3 与 T_1 可能不同。说法 I 正确。例如，如下图所示，删除结点 0，平衡二叉树失衡调整，再插入结点 0 后，平衡二叉树和以前不同。

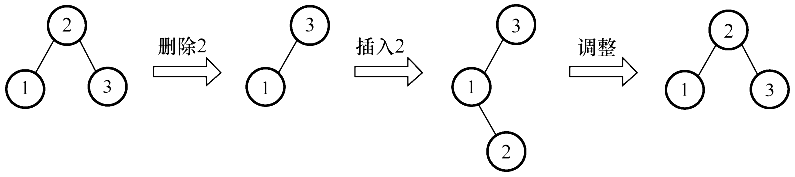


对于比较绝对的说法 II 和 III，通常只需举出反例即可。

若删除的是 T_1 的非叶结点，且删除和插入操作均没有导致平衡二叉树的调整（这时可以首先想到删除的结点只有一个孩子的情况），则该结点从非叶结点变成了叶结点，T_1 与 T_3 显然不同。例如，如下图所示，删除结点 2，用右孩子结点 3 填补，再插入结点 2，平衡二叉树和以前不同。

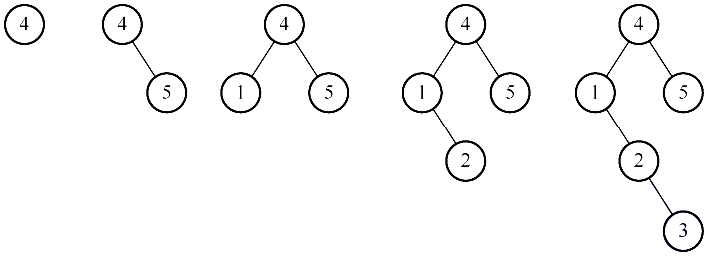


若删除的是 T_1 的非叶结点，且删除和插入操作后导致了平衡二叉树的调整，则该结点有可能通过旋转后继续变成非叶结点，T_1 与 T_3 相同。例如，如下图所示，删除结点 2，用右孩子结点 3 填补，再插入结点 2，平衡二叉树失衡调整，调整后的平衡二叉树和以前相同。



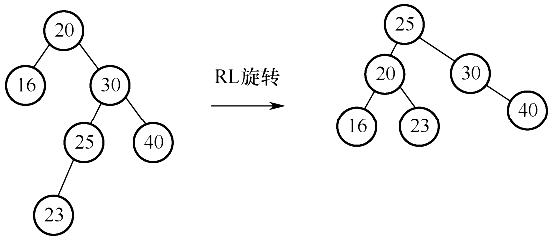
32．B

每个选项都逐一验证，选项 B 生成二叉排序树的过程如下图所示，显然错误。



33．D

关键字 23 的插入位置为 25 的左孩子，此时破坏了平衡的性质，需要对平衡二叉树进行调整。最小不平衡子树就是该树本身，插入位置在根结点的右子树的左子树上，因此需要进行 RL 旋转，RL 旋转过程如下图所示，旋转完成后根结点的关键字为 25，所以选择选项 D。



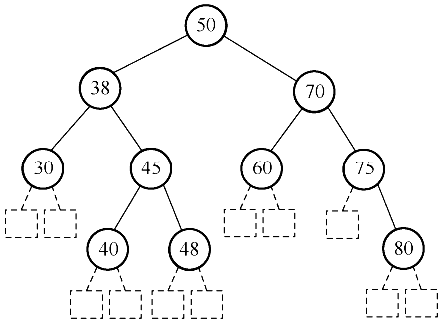
34．D

在二叉搜索树中，每个结点的左子树中的所有关键字都小于该结点的关键字，右子树中的所有关键字都大于该结点的关键字。k_2、k_3 所在结点都在 k_1 所在结点的右子树中，因此 $k_2 > k_1$、$k_3 > k_1$；k_3 所在结点在 k_2 所在结点的左子树中，因此 $k_3 < k_2$；子树 T 是 k_3 所在结点的右子树，因此子树 T 中任意结点的关键字 x 满足 $x > k_3$。综合可得，$k_3 < x < k_2$。

二、综合应用题

01．【解答】

先序序列为(50, 38, 30, 45, 40, 48, 70, 60, 75, 80)，二叉树的中序序列是一个有序序列，所以为(30, 38, 40, 45, 48, 50, 60, 70, 75, 80)，由先序序列和中序序列可以构造出对应的二叉树，如下图所示。



查找成功的平均查找长度为

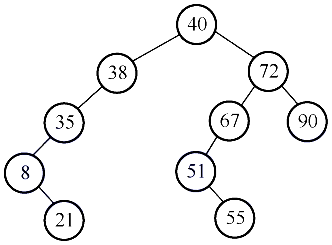
$$ASL = (1\times1 + 2\times2 + 3\times4 + 4\times3)/10 = 2.9$$

图中的方块结点为虚构的查找失败结点，其查找路径为从根结点到其父结点（圆形结点）的结点序列，所以对应的查找失败平均长度为

$$ASL = (3\times5 + 4\times6)/11 = 39/11$$

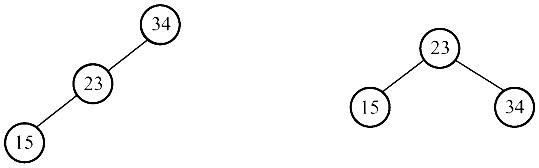
02.【解答】

根据二叉排序树的定义，该序列所对应的二叉排序树如下图所示。

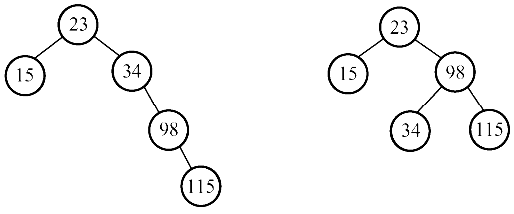


平均查找长度为 ASL = (1 + 2×2 + 3×3 + 4×2 + 5×2)/10 = 3.2。

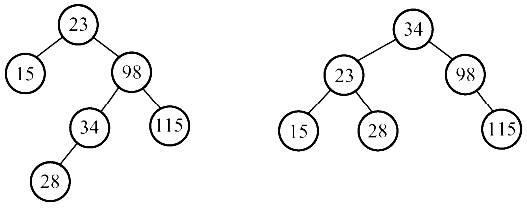
03.【解答】



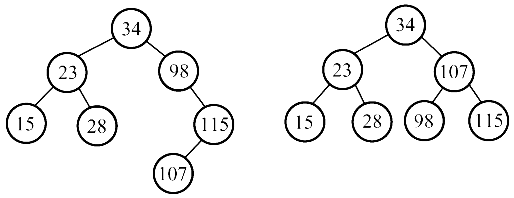
第一步：插入结点 34, 23, 15 后，需要根结点 34 的子树做 LL 调整。



第二步：插入结点 98, 115 后，需要根结点 34 的子树做 RR 调整。



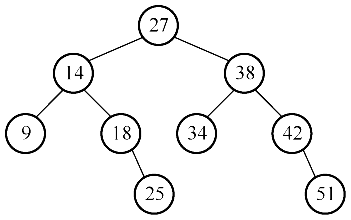
第三步：插入结点 28 后，需要根结点 23 的子树做 RL 调整。



第四步：插入结点 107 后，需要根结点 98 的子树做 RL 调整。

04.【解答】

当各关键字的查找概率相等时，最佳二叉排序树应是高度最小的二叉排序树。构造过程分两步走：首先对各关键字按值从小到大排序，然后仿照折半查找的判定树的构造方法构造二叉排序树。这样得到的就是最佳二叉排序树，结果如下图所示。



05.【解答】

对二叉排序树来说，其中序遍历序列为一个递增有序序列。因此，对给定的二叉树进行中序遍历，若始终能保持前一个值比后一个值小，则说明该二叉树是一棵二叉排序树。算法实现如下：

```
KeyType predt=-32767;                  //predt 为全局变量，保存当前结点中序前驱
                                         的值，初值为-∞
int JudgeBST(BiTree bt){
 int b1,b2;
 if(bt==NULL)                          //空树
     return  1;
 else{
     b1=JudgeBST(bt->lchild);          //判断左子树是否是二叉排序树
     if(b1==0||predt>=bt->data)        //若左子树返回值为 0 或前驱大于或等于当前结点
         return 0;                     //则不是二叉排序树
     predt=bt->data;                   //保存当前结点的关键字
     b2=JudgeBST(bt->rchild);          //判断右子树
     return  b2;                       //返回右子树的结果
 }
}
```

06.【解答】

算法思想：设二叉树采用二叉链表存储结构。在二叉排序树中，查找一次就下降一层。因此，查找该结点所用的次数就是该结点在二叉排序树中的层次。采用二叉排序树非递归查找算法，用 n 保存查找层次，每查找一次，n 就加 1，直到找到相应的结点。算法如下：

```
int level(BiTree bt,BSTNode *p){
 int n=0;                              //统计查找次数
 BiTree t=bt;
 if(bt!=NULL){
     n++;
     while(t->data!=p->data){
         if(p->data<t->data)           //在左子树中查找
             t=t->lchild;
```

```
            else                        //在右子树中查找
                t=t->rchild;
            n++;                        //层次加 1
        }
    }
    return  n;
}
```

07.【解答】

设置二叉树的平衡标记 balance，以标记返回二叉树 bt 是否为平衡二叉树，若为平衡二叉树，则返回 1，否则返回 0；h 为二叉树 bt 的高度。采用后序遍历的递归算法：

1）若 bt 为空，则高度为 0，balance=1。

2）若 bt 仅有根结点，则高度为 1，balance=1。

3）否则，对 bt 的左、右子树执行递归运算，返回左、右子树的高度和平衡标记，bt 的高度为最高子树的高度加 1。若左、右子树的高度差大于 1，则 balance=0；若左、右子树的高度差小于或等于 1，且左、右子树都平衡时，balance=1，否则 balance=0。

算法如下：

```
void Judge_AVL(BiTree bt,int &balance,int &h){
 int bl=0,br=0,hl=0,hr=0;               //左、右子树的平衡标记和高度
 if(bt==NULL){                          //空树，高度为 0
     h=0;
     balance=1;
 }
 else if(bt->lchild==NULL&&bt->rchild==NULL){//仅有根结点，则高度为 1
     h=1;
     balance=1;
 }
 else{
     Judge_AVL(bt->lchild,bl,hl);    //递归判断左子树
     Judge_AVL(bt->rchild,br,hr);    //递归判断右子树
     h=(hl>hr?hl:hr)+1;
     if(abs(hl-hr)<2)     //若子树高度差的绝对值<2，则看左、右子树是否都平衡
         balance=bl&&br; //&&为逻辑与，即左、右子树都平衡时，二叉树平衡
     else
         balance=0;
 }
}
```

08.【解答】

在一棵二叉排序树中，最左下结点即关键字最小的结点，最右下结点即关键字最大的结点，本算法只要找出这两个结点即可，而不需要比较关键字。算法如下：

```
KeyType MinKey(BSTNode *bt){
 while(bt->lchild!=NULL)
     bt=bt->lchild;
 return bt->data;
}
KeyType MaxKey(BSTNode *bt){
//求出二叉排序树中最大关键字结点
 while(bt->rchild!=NULL)
     bt=bt->rchild;
   return bt->data;
}
```

09.【解答】

由二叉排序树的性质可知，右子树中所有的结点值均大于根结点值，左子树中所有的结点值

均小于根结点值。为了从大到小输出，先遍历右子树，再访问根结点，后遍历左子树。算法如下：

```
void OutPut(BSTNode *bt,KeyType k)
{//本算法从大到小输出二叉排序树中所有值不小于 k 的关键字
 if(bt==NULL)
     return;
 if(bt->rchild!=NULL)
     OutPut(bt->rchild,k);          //递归输出右子树结点
 if(bt->data>=k)
     printf("%d",bt->data);         //只输出大于或等于 k 的结点值
 if(bt->lchild!=NULL)
     OutPut(bt->lchild,k);          //递归输出左子树的结点
}
```

本题也可采用中序遍历加辅助栈的方法实现。

10.【解答】

设二叉排序树的根结点为*t，根据结点存储的信息，有以下几种情况：

当 t->lchild 为空时，情况如下：

1）若 t->rchild 非空且 k==1，则*t 即第 k 小的元素，查找成功。

2）若 t->rchild 非空且 k!=1，则第 k 小的元素必在*t 的右子树。

当 t->lchild 非空时，情况如下：

1）t->lchild->count==k-1，*t 即第 k 小的元素，查找成功

2）t->lchild->count>k-1，第 k 小的元素必在*t 的左子树，继续到*t 的左子树中查找。

3）t->lchild->count<k-1，第 k 小的元素必在右子树，继续搜索右子树，寻找第 k-(t->lchild->count+1)小的元素。

对左右子树的搜索采用相同的规则，递归实现的算法描述如下：

```
BSTNode *Search_Small(BSTNode*t,int k){
//在以 t 为根的子树上寻找第 k 小的元素，返回其所在结点的指针。k 从 11 开始计算
//在树结点中增加一个 count 数据成员，存储以该结点为根的子树的结点个数
    if(k<1||k>t->count) return NULL;
    if(t->lchild==NULL){
        if(k==1) return t;
        else return Search_Small(t->rchild,k-1);
    }
    else{
        if(t->lchild->count==k-1) return t;
        if(t->lchild->count>k-1) return Search_Small(t->lchild,k);
        if(t->lchild->count<k-1)
            return Search_Small(t->rchild, k-(t->lchild->count+1));
    }
}
```

最大查找长度取决于树的高度。由于二叉排序树是随机生成的，其高度应是 $O(\log_2 n)$，算法的时间复杂度为 $O(\log_2 n)$。

7.4 B树[①]和B+树

考研大纲对B树和B+树的要求各不相同，重点在于考查B树，不仅要求理解B树的基本特点，还要求掌握B树的建立、插入和删除操作，而对B+树则只考查基本概念。

① 也可写成“B-树”，注意这里的“-”是连接词，不能读作“减”。

7.4.1　B 树及其基本操作

所谓 m 阶 B 树是所有结点的平衡因子均等于 0 的 m 路平衡查找树。

命题追踪 ▶ B 树的定义和特点（2009）

一棵 m 阶 B 树或为空树，或为满足如下特性的 m 叉树：

1）树中每个结点至多有 m 棵子树，即至多有 $m-1$ 个关键字。

2）若根结点不是叶结点，则至少有 2 棵子树，即至少有 1 个关键字。

3）除根结点外的所有非叶结点至少有$\lceil m/2 \rceil$棵子树，即至少有$\lceil m/2 \rceil - 1$个关键字。

4）所有非叶结点的结构如下：

n	P_0	K_1	P_1	K_2	P_2	…	K_n	P_n

其中，K_i（$i=1, 2, \cdots, n$）为结点的关键字，且满足 $K_1 < K_2 < \cdots < K_n$；P_i（$i=0, 1, \cdots, n$）为指向子树根结点的指针，且指针 P_{i-1} 所指子树中所有结点的关键字均小于 K_i，P_i 所指子树中所有结点的关键字均大于 K_i；n（$\lceil m/2 \rceil - 1 \leqslant n \leqslant m-1$）为结点中关键字的个数。

5）所有的叶结点[①]都出现在同一层次上，并且不带信息（可以视为外部结点或类似于折半查找判定树的失败结点，实际上这些结点并不存在，指向这些结点的指针为空）。

命题追踪 ▶ B 树中关键字数和结点数的分析（2013、2014、2018、2021）

图 7.28 所示为一棵 5 阶 B 树，可以借助该实例来分析上述性质：

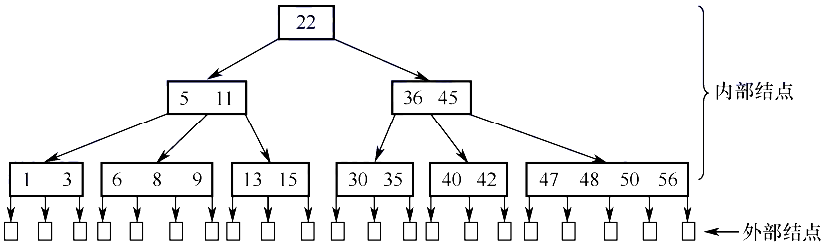


图 7.28　一棵 5 阶 B 树的实例

1）结点的孩子个数等于该结点中关键字个数加 1。

2）若根结点没有关键字就没有子树，则此时 B 树为空；若根结点有关键字，则其子树个数必然大于或等于 2，因为子树个数等于关键字个数加 1。

3）除根结点外的所有非叶结点至少有$\lceil m/2 \rceil = \lceil 5/2 \rceil = 3$棵子树（至少有$\lceil m/2 \rceil - 1 = \lceil 5/2 \rceil - 1 = 2$个关键字）；至多有 5 棵子树（至多有 4 个关键字）。

4）结点中的关键字从左到右递增有序，关键字两侧均有指向子树的指针，左侧指针所指子树的所有关键字均小于该关键字，右侧指针所指子树的所有关键字均大于该关键字。或者看成下层结点的关键字总是落在由上层结点的关键字所划分的区间内，如第二层最左结点的关键字划分成了 3 个区间：$(-\infty, 5)$, $(5, 11)$, $(11, +\infty)$，该结点中的 3 个指针所指子树的关键字均分别落在这 3 个区间内。

5）所有叶结点均在第 4 层，代表查找失败的位置。

1．B 树的查找

在 B 树上进行查找与二叉排序树很相似，只是每个结点都是多个关键字的有序表，在每个结

① 多数教材将 B 树的叶结点定义为失败结点，本书也采用这种定义，但 408 真题将 B 树的叶结点定义为最底层的终端结点。

点上所做的不是两路分支决定，而是根据该结点的子树所做的多路分支决定。

B树的查找包含两个基本操作：① 在B树中找结点；② 在结点内找关键字。B树常存储在磁盘上，因此前一查找操作是在磁盘上进行的，而后一查找操作是在内存中进行的，即在磁盘上找到目标结点后，先将结点信息读入内存，然后再采用顺序查找法或折半查找法。因此，在磁盘上进行查找的次数即目标结点在B树上的层次数，决定了B树的查找效率。

在B树上查找到某个结点后，先在有序表中进行查找，若找到则查找成功，否则按照对应的指针信息到所指的子树中去查找（例如，在图7.28中查找关键字42，首先从根结点开始，根结点只有一个关键字，且42 > 22，若存在，必在关键字22的右边子树上，右孩子结点有两个关键字，而36 < 42 < 45，则若存在，必在36和45中间的子树上，在该子结点中查到关键字42，查找成功）。查找到叶结点时（对应指针为空），则说明树中没有对应的关键字，查找失败。

2．B树的高度（磁盘存取次数）

由上一节得知，B树中的大部分操作所需的磁盘存取次数与B树的高度成正比。

下面来分析B树在不同情况下的高度。当然，首先应该明确B树的高度不包括最后的不带任何信息的叶结点所处的那一层（有些书对B树的高度的定义中，包含最后的那一层）。

若$n \geqslant 1$，则对任意一棵包含n个关键字、高度为h、阶数为m的B树：

1）若让每个结点中的关键字个数达到最多，则容纳同样多关键字的B树的高度达到最小。因为B树中每个结点最多有m棵子树，$m-1$个关键字，所以在一棵高度为h的m阶B树中关键字的个数应满足$n \leqslant (m-1)(1+m+m^2+\cdots+m^{h-1}) = m^h - 1$，因此有

$$h \geqslant \log_m(n+1)$$

2）若让每个结点中的关键字个数达到最少，则容纳同样多关键字的B树的高度达到最大。第一层至少有1个结点；第二层至少有2个结点；除根结点外的每个非叶结点至少有$\lceil m/2 \rceil$棵子树，则第三层至少有$2\lceil m/2 \rceil$个结点……第$h+1$层至少有$2(\lceil m/2 \rceil)^{h-1}$个结点，注意到第$h+1$层是不包含任何信息的叶结点。对于关键字个数为$n$的B树，叶结点即查找不成功的结点为$n+1$，由此有$n+1 \geqslant 2(\lceil m/2 \rceil)^{h-1}$，即$h \leqslant \log_{\lceil m/2 \rceil}((n+1)/2)+1$。

例如，假设一棵3阶B树共有8个关键字，则其高度范围为$2 \leqslant h \leqslant 3.17$，取整数。

3．B树的插入

命题追踪 ▶ 通过插入操作构造一棵初始为空的B树（2020）

与二叉排序树的插入操作相比，B树的插入操作要复杂得多。在B树中查找到插入的位置后，并不能简单地将其添加到终端结点（最底层的非叶结点）中，因为此时可能会导致整棵树不再满足B树定义中的要求。将关键字key插入B树的过程如下：

1）定位。利用前述的B树查找算法，找出插入该关键字的终端结点（在B树中查找key时，会找到表示查找失败的叶结点，因此插入位置一定是最底层的非叶结点）。

2）插入。每个非根结点的关键字个数都在$[\lceil m/2 \rceil - 1, m-1]$。若结点插入后的关键字个数小于$m$，可以直接插入；若结点插入后的关键字个数大于$m-1$，必须对结点进行分裂。

分裂的方法是：取一个新结点，在插入key后的原结点，从中间位置（$\lceil m/2 \rceil$）将其中的关键字分为两部分，左部分包含的关键字放在原结点中，右部分包含的关键字放到新结点中，中间位置（$\lceil m/2 \rceil$）的结点插入原结点的父结点。若此时导致其父结点的关键字个数也超过了上限，则继续进行这种分裂操作，直至这个过程传到根结点为止，进而导致B树高度增1。

对于$m=3$的B树，所有结点中最多有$m-1=2$个关键字，若某结点中已有两个关键字，则结点已满，如图7.29(a)所示。插入一个关键字60后，结点内的关键字个数超过了$m-1$，如图7.29(b)所示，此时必须进行结点分裂，分裂的结果如图7.29(c)所示。

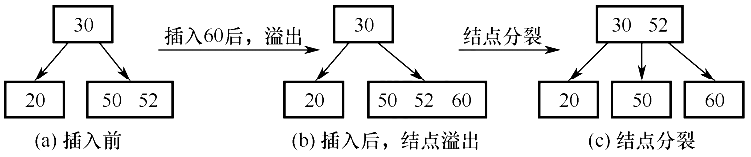


图 7.29　结点的“分裂”示意

4．B 树的删除

B 树的删除操作与插入操作类似，但要稍微复杂一些，即要使得删除后的结点中的关键字个数≥$\lceil m/2 \rceil - 1$，因此将涉及结点的“合并”问题。

命题追踪　▶　B 树的删除操作的实例（2012、2022）

当被删关键字 k 不在终端结点中时，可以用 k 的前驱（或后继）k'，即 k 的左侧子树中“最右下”的元素（或右侧子树中“最左下”的元素），来替代 k，然后在相应的结点中删除 k'，关键字 k'必定落在某个终端结点中，则转换成了被删关键字在终端结点中的情形。在图 7.30 的 4 阶 B 树中，删除关键字 80，用其前驱 78 替代，然后在终端结点中删除 78。

图 7.30　B 树中删除非终端结点关键字的取代

因此只需讨论被删关键字在终端结点中的情形，有下列三种情况：

1）直接删除关键字。若被删关键字所在结点删除前的关键字个数≥$\lceil m/2 \rceil$，表明删除该关键字后仍满足 B 树的定义，则直接删去该关键字。

2）兄弟够借。若被删关键字所在结点删除前的关键字个数=$\lceil m/2 \rceil - 1$，且与该结点相邻的右（或左）兄弟结点的关键字个数≥$\lceil m/2 \rceil$，则需要调整该结点、右（或左）兄弟结点及其双亲结点（父子换位法），以达到新的平衡。在图 7.31(a)中删除 4 阶 B 树的关键字 65，右兄弟关键字个数≥$\lceil m/2 \rceil = 2$，将 71 取代原 65 的位置，将 74 调整到 71 的位置。

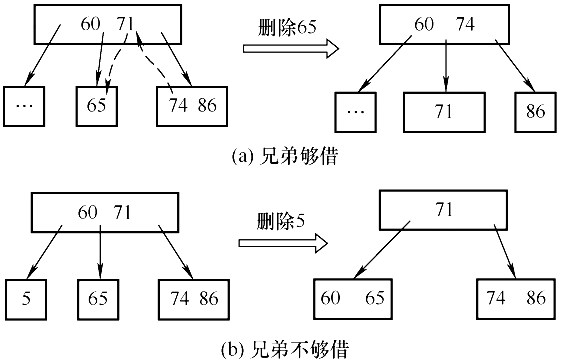


图 7.31　4 阶 B 树中删除终端结点关键字的示意图

3）兄弟不够借。若被删关键字所在结点删除前的关键字个数=$\lceil m/2 \rceil - 1$，且此时与该结点相邻的左、右兄弟结点的关键字个数都=$\lceil m/2 \rceil - 1$，则将关键字删除后与左（或右）兄弟结点及双亲结点中的关键字进行合并。在图 7.31(b)中删除 4 阶 B 树的关键字 5，它及其右兄弟结点的关键字个数=$\lceil m/2 \rceil - 1 = 1$，所以在 5 删除后将 60 合并到 65 结点中。

命题追踪 ▶ 非空 B 树的查找、插入、删除操作的特点（2023）

在合并过程中，双亲结点中的关键字个数会减 1。若其双亲结点是根结点且关键字个数减少至 0（根结点关键字个数为 1 时，有 2 棵子树），则直接将根结点删除，合并后的新结点成为根；若双亲结点不是根结点，且关键字个数减少到$\lceil m/2\rceil-2$，则又要与它自己的兄弟结点进行调整或合并操作，并重复上述步骤，直至符合 B 树的要求为止。

7.4.2 B+树的基本概念

命题追踪 ▶ B+树的应用场合（2017）

B+树是应数据库所需而出现的一种 B 树的变形树。

一棵 m 阶 B+树应满足下列条件：

1）每个分支结点最多有 m 棵子树（孩子结点）。

2）非叶根结点至少有两棵子树，其他每个分支结点至少有$\lceil m/2\rceil$棵子树。

3）结点的子树个数与关键字个数相等。

4）所有叶结点包含全部关键字及指向相应记录的指针，叶结点中将关键字按大小顺序排列，并且相邻叶结点按大小顺序相互链接起来（支持顺序查找）。

5）所有分支结点（可视为索引的索引）中仅包含它的各个子结点（下一级的索引块）中关键字的最大值及指向其子结点的指针。

命题追踪 ▶ B 树和 B+树的差异的分析（2016）

m 阶 B+树与 m 阶 B 树的主要差异如下：

1）在 B+树中，具有 n 个关键字的结点只含有 n 棵子树，即每个关键字对应一棵子树；而在 B 树中，具有 n 个关键字的结点含有 $n+1$ 棵子树。

2）在 B+树中，每个结点（非根内部结点）的关键字个数 n 的范围是$\lceil m/2\rceil\leqslant n\leqslant m$（非叶根结点：$2\leqslant n\leqslant m$）；而在 B 树中，每个结点（非根内部结点）的关键字个数 n 的范围是$\lceil m/2\rceil-1\leqslant n\leqslant m-1$（根结点：$1\leqslant n\leqslant m-1$）。

3）在 B+树中，叶结点包含了全部关键字，非叶结点中出现的关键字也会出现在叶结点中；而在 B 树中，最外层的终端结点包含的关键字和其他结点包含的关键字是不重复的。

4）在 B+树中，叶结点包含信息，所有非叶结点仅起索引作用，非叶结点的每个索引项只含有对应子树的最大关键字和指向该子树的指针，不含有对应记录的存储地址。这样能使一个磁盘块存储更多的关键字，使得磁盘读/写次数更少，查找速度更快。

5）在 B+树中，用一个指针指向关键字最小的叶结点，将所有叶结点串成一个线性链表。

图 7.32 所示为一棵 4 阶 B+树。可以看出，分支结点的关键字是其子树中最大关键字的副本。通常在 B+树中有两个头指针：一个指向根结点，另一个指向关键字最小的叶结点。因此，可以对 B+树进行两种查找运算：一种是从最小关键字开始的顺序查找，另一种是从根结点开始的多路查找。

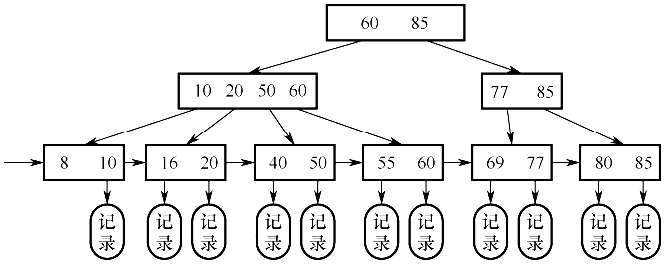


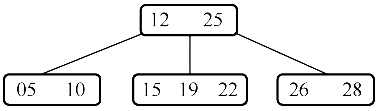
图 7.32 B+树结构示意图

B+树的查找、插入和删除操作和 B 树的基本类似。只是在查找过程中，非叶结点上的关键字值等于给定值时并不终止，而是继续向下查找，直到叶结点上的该关键字为止。因此，在 B+树中查找成功或失败（≤最大关键字）时，每次查找都是一条从根结点到叶结点的路径。

7.4.3　本节试题精选

一、单项选择题

01. 下图所示是一棵（　）。



A. 4 阶 B 树　　B. 3 阶 B 树　　C. 4 阶 B+树　　D. 无法确定

02. 下列关于 m 阶 B 树的说法中，错误的是（　）。

A. 根结点至多有 m 棵子树

B. 所有叶结点都在同一层次上

C. 非叶结点至少有 $m/2$（m 为偶数）或$(m+1)/2$（m 为奇数）棵子树

D. 根结点中的数据是有序的

03. 下列关于高度为 3 的 3 阶 B 树的说法中，正确的是（　）。

I. 每个结点至少有两棵非空子树

II. 树中每个结点至多有 2 个关键字

III. 树中最多能存储 26 个关键字

IV. 插入一个元素引起 B 树结点分裂后，树的高度变为 4

A. I、II　　B. II、III　　C. III、IV　　D. I、II、IV

04. 在一棵 m 阶 B 树中做插入操作前，若一个结点中的关键字个数等于（　），则插入操作后必须分裂成两个结点；在一棵 m 阶 B 树中做删除操作前，若一个结点中的关键字个数等于（　），则删除操作后可能需要同它的左兄弟或右兄弟结点合并成一个结点。

A. m，$\lceil m/2\rceil-2$　　B. $m-1$，$\lceil m/2\rceil-1$

C. $m+1$，$\lceil m/2\rceil$　　D. $m/2$，$\lceil m/2\rceil+1$

05. 具有 n 个关键字的 m 阶 B 树，应有（　）个叶结点。

A. $n+1$　　B. $n-1$　　C. mn　　D. $nm/2$

06. 高度为 5 的 3 阶 B 树至少有（　）个结点，至多有（　）个结点。

A. 32　　B. 31　　C. 120　　D. 121

07. 含有 n 个非叶结点的 m 阶 B 树中至少包含（　）个关键字。

A. $n(m+1)$　　B. n　　C. $n(\lceil m/2\rceil-1)$　　D. $(n-1)(\lceil m/2\rceil-1)+1$

08. 已知一棵 5 阶 B 树中共有 53 个关键字，则树的最大高度为（　），最小高度为（　）。

A. 2　　B. 3　　C. 4　　D. 5

09. 已知一棵 3 阶 B 树中共有 2047 个关键字，则树的最大高度为（　），最小高度为（　）。

A. 11　　B. 10　　C. 8　　D. 7

10. 在 7 阶 B 树中搜索第 2016 个关键字，若根结点已读入内存，则最多需启动（　）次 I/O。

A. 4　　B. 5　　C. 6　　D. 7

11. 在一棵高度为 h 的 B 树中插入一个新关键字，假设在插入过程中读入的结点一直在内存中，根结点的高度为 1，且初始时未读入内存，则下列叙述中错误的是（　）。（注意，本

题中的新结点是指新产生的结点，如一次分裂才产生一个新结点。）

A. 若插入操作导致树的高度变为 $h+1$，则本次插入一定导致了根结点的分裂

B. 若插入操作导致旧结点的分裂，则树的高度一定会变为 $h+1$

C. 由于本次插入操作而产生的新结点的个数最多为 $h+1$

D. 由于本次插入操作而产生的读/写磁盘的次数最多为 $3h+1$

12. 下列关于B树和B+树的叙述中，错误的是（ ）。

A. B树和B+树都能有效地支持顺序查找

B. B树和B+树都能有效地支持随机查找

C. B树和B+树都是平衡的多叉树

D. B树和B+树都可以用于文件索引结构

13. 下列关于B树和B+树的查找操作的叙述中，错误的是（ ）。

A. B树查找成功时，不一定需要查找到最后一层的内部结点

B. B树查找失败时，一定需要查找到叶结点

C. B+树查找成功时，不一定需要查找到叶结点

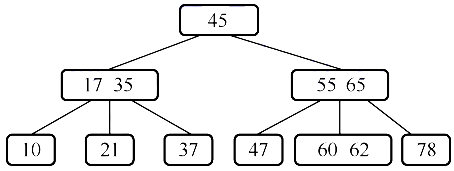
D. B+树查找成功时，每次查找的长度都相等

14. 【2009统考真题】下列叙述中，不符合 m 阶B树定义要求的是（ ）。

A. 根结点至多有 m 棵子树　　B. 所有叶结点都在同一层上

C. 各结点内关键字均升序或降序排列　　D. 叶结点之间通过指针链接

15. 【2012统考真题】已知一棵3阶B树，如下图所示。删除关键字78得到一棵新B树，其最右叶结点中的关键字是（ ）。



A. 60　　B. 60, 62　　C. 62, 65　　D. 65

16. 【2013统考真题】在一棵高度为2的5阶B树中，所含关键字的个数至少是（ ）。

A. 5　　B. 7　　C. 8　　D. 14

17. 【2014统考真题】在一棵有15个关键字的4阶B树中，含关键字的结点个数最多是（ ）。

A. 5　　B. 6　　C. 10　　D. 15

18. 【2016统考真题】B+树不同于B树的特点之一是（ ）。

A. 能支持顺序查找　　B. 结点中含有关键字

C. 根结点至少有两个分支　　D. 所有叶结点都在同一层上

19. 【2017统考真题】下列应用中，适合使用B+树的是（ ）。

A. 编译器中的词法分析　　B. 关系数据库系统中的索引

C. 网络中的路由表快速查找　　D. 操作系统的磁盘空闲块管理

20. 【2018统考真题】高度为5的3阶B树含有的关键字个数至少是（ ）。

A. 15　　B. 31　　C. 62　　D. 242

21. 【2020统考真题】依次将关键字5，6，9，13，8，2，12，15插入初始为空的4阶B树后，根结点中包含的关键字是（ ）。

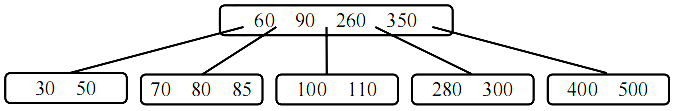
A. 8　　B. 6，9　　C. 8，13　　D. 9，12

22. 【2021统考真题】在一棵高度为3的3阶B树中，根为第1层，若第2层中有4个关键

字，则该树的结点数最多是（　）。

A. 11　　B. 10　　C. 9　　D. 8

23. 【2022 统考真题】在下图所示的 5 阶 B 树 T 中，删除关键字 260 之后需要进行必要的调整，得到新的 B 树 T_1。下列选项中，不可能是 T_1 根结点中关键字序列的是（　）。



A. 60, 90, 280　　B. 60, 90, 350

C. 60, 85, 110, 350　　D. 60, 90, 110, 350

24. 【2023 统考真题】下列关于非空 B 树的叙述中，正确的是（　）。

I. 插入操作可能增加树的高度　　II. 删除操作一定会导致叶结点的变化

III. 查找某关键字总是要查找到叶结点　　IV. 插入的新关键字最终位于叶结点中

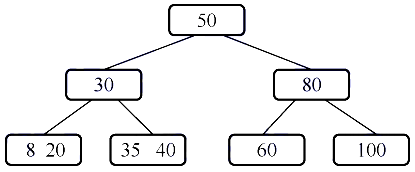
A. 仅 I　　B. 仅 I、II　　C. 仅 III、IV　　D. 仅 I、II、IV

二、综合应用题

01. 给定一组关键字{20, 30, 50, 52, 60, 68, 70}，给出创建一棵 3 阶 B 树的过程。

02. 对如下图所示的 3 阶 B 树，依次执行下列操作，画出各步操作的结果。

1）插入 90　　2）插入 25　　3）插入 45　　4）删除 60　　5）删除 80



03. 利用 B 树做文件索引时，若假设磁盘页块的大小是 4000B（实际应是 2 的次幂，此处是为了计算方便），指示磁盘地址的指针需要 5B。现有 20000000 个记录构成的文件，每个记录为 200B，其中包括关键字 5B。

试问在这个采用 B 树作索引的文件中，B 树的阶数应为多少？假定文件数据部分未按关键字有序排列，则索引部分需要占用多少磁盘页块？

7.4.4　答案与解析

一、单项选择题

01. D

关键字数目比子树数目少 1，首先可排除 B+树。对于 4 阶 B 树，根结点至少有 2 棵子树（关键字数至少为 1），其他非叶结点至少有$\lceil m/2 \rceil = 2$ 棵子树（关键字数至少为 1）、至多有 4 棵子树（关键字数至多为 3）。5 阶 B 树和 6 阶 B 树的分析也类似。题目所示的 B 树，同时满足 4 阶 B 树、5 阶 B 树和 6 阶 B 树的要求，因此不能确定是哪种类型的 B 树。

02. C

除根结点外的所有非叶结点至少有$\lceil m/2 \rceil$棵子树。对于根结点，最多有 m 棵子树，若其不是叶结点，则至少有 2 棵子树。

03. B

每个非根的内部结点必须至少有$\lceil m/2 \rceil$ 棵子树，而本题中的根结点不是叶结点，也至少要有

两棵子树，选项 I 正确。每个结点至多有 $m-1=2$ 个关键字，选项 II 正确。在高度为 h 的 m 阶 B 树中，关键字个数至多为$(m-1)(1+m+m^2+\cdots+m^{h-1})=m^h-1$，代入 $m=3$，$h=3$，即树中最多能存储 26 个关键字，选项 III 正确。对于选项 IV，插入一个元素引起 B 树结点分裂后，只要从根结点到该元素插入位置的路径上至少有一个结点未满，B 树就不会长高，如图 1 所示；只有当结点的分裂传到根结点，并使根结点也分裂时，才会导致树高增 1，如图 2 所示，选项 IV 错误。

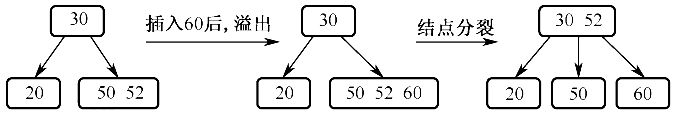


图 1　结点分裂不导致树高增 1（3 阶 B 树）

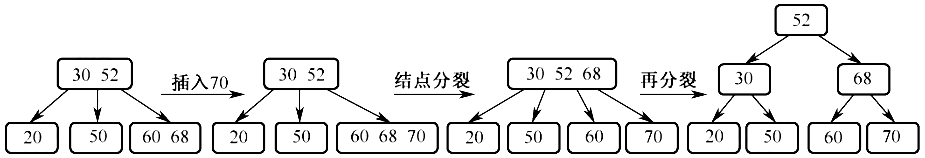


图 2　结点分裂导致树高增 1（3 阶 B 树）

04．B

因为 B 树每个结点内的关键字个数最多为 $m-1$，所以当关键字个数大于 $m-1$ 时，则应该分裂。每个结点内的关键字个数至少为$\lceil m/2\rceil-1$ 个，所以当关键字个数少于$\lceil m/2\rceil-1$ 时，则可能与其他结点合并（除非只有根结点）。若将本题题干改为 B+树，请读者思考上述问题的解答。

05．A

B 树的叶结点对应查找失败的情况，对有 n 个关键字的查找集合进行查找，失败可能性有 $n+1$ 种。

06．B、D

由 m 阶 B 树的性质可知，根结点至少有 2 棵子树；根结点外的所有非终端结点至少有$\lceil m/2\rceil$ 棵子树，结点数最少时，3 阶 B 树形状至少类似于一棵满二叉树，即高度为 5 的 B 树至少有 $2^5-1=31$ 个结点。因为每个结点最多有 m 棵子树，所以当结点数最多时，3 阶 B 树形状类似于满三叉树，结点数为$(3^5-1)/2=121$（注意，这里求的是结点数而非关键字数，若求的是关键字数，则还应把每个结点中关键字数的上下界确定出来）。

07．D

除根结点外，m 阶 B 树中的每个非叶结点至少有$\lceil m/2\rceil-1$ 个关键字，根结点至少有一个关键字，所以总共包含的关键字最少个数 $=(n-1)(\lceil m/2\rceil-1)+1$。

注　意

由以上题目可知 B 树和 B+树的定义与性质尤为重要，需要熟练掌握。

08．C、B

5 阶 B 树中共有 53 个关键字，由最大高度公式 $H\leqslant\log_{\lceil m/2\rceil}((n+1)/2)+1$ 得最大高度 $H\leqslant\log_3[(53+1)/2]+1=4$，即最大高度为 4；由最小高度公式 $h\geqslant\log_m(n+1)$得最小高度 $h\geqslant\log_5 54\approx 2.5$，从而最小高度为 3。

09．A、D

利用前面的公式即最小高度 $h\geqslant\log_m(n+1)$和最大高度 $H\leqslant\log_{\lceil m/2\rceil}[(n+1)/2]+1$，易算出最大高度 $H\leqslant\log_2[(2047+1)/2]+1=11$，最小高度 $h\geqslant\log_3 2048=6.9$，从而最小高度取 7（注意，有些

辅导书针对本题算出的高度要比这里给出的答案多 1，因为它们在对 B 树的高度定义中，把最底层不包含任何关键字的叶结点也算进去了）。

10．B

本题要计算的是最坏情况下第 2016 个关键字在 7 阶 B 树中的最大深度，即考查 B 树结点中关键字数最少的情况。按照 B 树的定义，根结点至少有 1 个关键字，第二层至少有 2 个结点，即 2×3 个关键字；第三层至少有 2×4 个结点，即 2×4×3 个关键字；第四层至少有 2×4^2 个结点，即 $2\times4^2\times3$ 个关键字；以此类推，第 h 层至少有 $2\times4^{h-2}$ 个结点，即 $2\times4^{h-2}\times3$ 个关键字，故前 h 层的关键字总数 $n=1+2\times3\times(1+4+\cdots+4^{h-2})=1+2\times(4^{h-1}-1)$。故当 $h=5$ 时，$n=511$；当 $h=6$ 时，$n=2047$，说明 5 层高的 7 阶 B 树最少有 511 个关键字，而 6 层高的 7 阶 B 树最少有 2047 个关键字，故最坏情况下第 2016 个关键字在 7 阶 B 树的第 6 层，需启动 5 次 I/O 操作。

11．B

考虑最坏情况，在待插入结点中插入一个关键码后，导致结点分裂，在该层分裂成 2 个结点，因此需要 2 次写磁盘操作；分裂操作逐层向上传导，导致每层都有结点分裂，因为结点分裂而导致的写磁盘操作共有 $2h$ 次，加上最后一次结点分裂形成新根结点也需要 1 次写磁盘操作，写磁盘的总次数为 $2h+1$，即由于本次插入而产生的新结点个数为 $h+1$，加上寻找插入位置而引起的 h 次的读磁盘操作，整个过程中读/写磁盘的总次数为 $3h+1$ 次，选项 C、D 正确。若插入操作导致了 B 树的高度增加，则分裂操作一定是从最底层传导至根结点的，即前面分析的最坏情况，选项 A 正确。若分裂操作没有传导至根结点，则 B 树的高度不变，选项 B 错误。

12．A

B 树和 B+树的差异主要体现在：①结点关键字和子树的个数；②B+树非叶结点仅起索引作用；③B 树叶结点关键字和其他结点包含的关键字是不重复的；④B+树支持顺序查找和随机查找，而 B 树仅支持随机查找。B+树的所有叶结点中包含了全部的关键字信息，且叶结点本身依关键字从小到大顺序链接，因此可以进行顺序查找，而 B 树不支持顺序查找。B 树和 B+树都可用于文件索引结构，但 B+树更适合做数据库索引和文件索引，因为它的磁盘读/写代价更低。

13．C

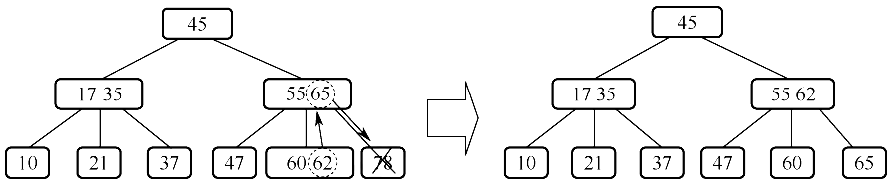
在 B 树的查找操作中，若目标元素在某个内部结点中，则查找结束，不需要进入叶结点；查找失败时，叶结点是所有路径的终点，选项 A、B 正确。B+树中仅叶结点包含信息，非叶结点仅起到索引作用，查找成功时一定会找到相应的叶结点，所经过的路径长度都相等，选项 C 错误，选项 D 正确。

14．D

m 阶 B 树不要求将各叶结点之间用指针链接。选项 D 描述的实际上是 B+树。

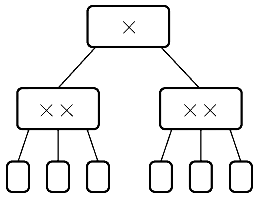
15．D

对于图中所示的 3 阶 B 树，被删关键字 78 所在的结点在删除前的关键字个数= 1 =⌈3/2⌉− 1，且其左兄弟结点的关键字个数= 2 ≥⌈3/2⌉，属于“兄弟够借”的情况，因此要把该结点的左兄弟结点中的最大关键字上移到双亲结点中，同时把双亲结点中大于上移关键字的关键字下移到要删除关键字的结点中，这样就达到了新的平衡，如下图所示。



16. A

对于5阶B树，根结点的分支数最少为2（关键字数最少为1），其他非叶结点的分支数最少为$\lceil n/2 \rceil = 3$（关键字数最少为2），因此关键字个数最少的情况如下图所示（叶结点不计入高度）。



注 意

一般对于某个具体的B树图形，并不能确定是几阶B树。对于本题所述的5阶B树，不要误认为："存在至少有一个含关键字结点中的关键字达到4"才符合5阶B树的要求，因为5阶B树中各个结点包含的关键字个数最少为2（$\lceil 5/2 \rceil - 1 = 2$），最多为4（$5 - 1 = 4$）。当5阶B树中各个结点包含的关键字个数为2时，也满足5阶B树的要求。

17. D

关键字数量不变，要求结点数量最多，即要求每个结点中含关键字的数量最少。根据4阶B树的定义，根结点最少含1个关键字，非根结点中最少含$\lceil 4/2 \rceil - 1 = 1$个关键字，所以每个结点中关键字数量最少都为1个，即每个结点都有2个分支，类似于排序二叉树，而15个结点正好可以构造一个4层的4阶B树，使得终端结点全在第四层，符合B树的定义。

18. A

B+树的所有叶结点中包含了全部的关键字信息，且叶结点本身依关键字从小到大顺序链接，因此可以进行顺序查找，而B树不支持顺序查找（只支持多路查找）。

19. B

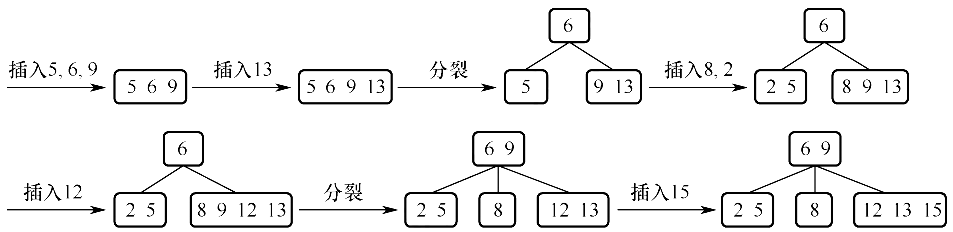
B+树是应文件系统所需而产生的B树的变形，前者比后者更加适用于实际应用中的操作系统的文件索引和数据库索引，因为前者的磁盘读/写代价更低，查询效率更加稳定。编译器中的词法分析使用有穷自动机和语法树。网络中的路由表快速查找主要靠高速缓存、路由表压缩技术和快速查找算法。系统一般使用空闲空间链表管理磁盘空闲块。

20. B

m阶B树的基本性质：根结点以外的非叶结点最少含有$\lceil m/2 \rceil - 1$个关键字，代入$m = 3$得到每个非叶结点中最少包含1个关键字，而根结点含有1个关键字，因此所有非叶结点都有两个孩子。此时其树形与$h = 5$的满二叉树相同，可求得关键字最少为31个。

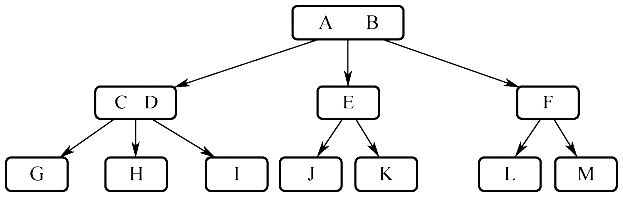
21. B

一个4阶B树的任意非叶结点至多含有$m - 1 = 3$个关键字，在关键字依次插入的过程中，会导致结点的不断分裂，插入过程如下图所示。得到根结点包含的关键字为6, 9。



22. A

在阶为 3 的 B 树中，每个结点至多含有 2 个关键字（至少 1 个），至多有 3 棵子树。本题规定第二层有 4 个关键字，欲使 B 树的结点数达到最多，则这 4 个关键字包含在 3 个结点中，B 树树形如下图所示，其中 A, B, C, … , M 表示关键字，最多有 11 个结点。



23．D

在 5 阶 B 树中，除根结点外的非叶结点的关键字数 k 需要满足 $2 \leqslant k \leqslant 4$ 。当被删关键字 x 不在终端结点（最底层非叶结点）时，可以用 x 的前驱（或后继）关键字 y 来替代 x，然后在相应结点中删除 y。情况①：删除 260，将其前驱 110 放入 260 处，删除 110 后的结点<100>不满足 5 阶 B 树定义，从左兄弟中借 85，将 85 放入根中，将根中的 90 移入结点<100>变为<90, 100>。情况②：删除 260，将其后继 280 放入 260 处，结点<300>不满足 5 阶 B 树定义且左右兄弟都不够借，结点<300>可以和左兄弟<100, 110>以及关键字 280 合并成一个新的结点<100, 110, 280, 300>。情况③：在情况②中，结点<300>也可以和右兄弟<400, 500>以及关键字 350 合并成一个新的结点<300, 350, 400, 500>。综上，T_1 根结点中的关键字序列可能是<60, 85, 110, 350>或<60, 90, 350>或<60, 90, 280>，仅选项 D 不可能。

快速解法：假如选项 D 的 60, 90, 110, 350 作为根结点，则在 90 和 110 之间只有 100 这一个数据，显然不符合 5 阶 B 树的定义，因此选项 D 不可能。

24．B

B 树的插入操作可能导致叶结点分裂，叶结点分裂可能导致父结点分裂，甚至会传导到根结点，从而导致 B 树高度增 1，选项 I 正确。若被删结点是叶结点，则显然会导致叶结点变化；若被删结点不是叶结点，则要先将被删结点和它的前驱或后继交换，最终转换为删除叶结点，还是导致叶结点变化，选项 II 正确。若在非叶结点中查找到了给定的关键字，则不用向下继续查找，选项 III 错误。插入关键字的初始位置是最底层叶结点，但可能因结点分裂而被转移到父结点中，选项 IV 错误。

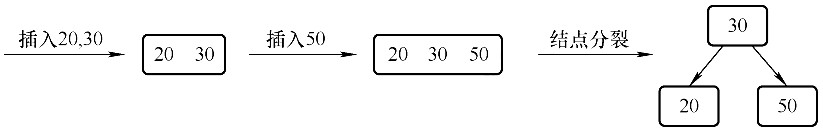
注　意

由本题可知，与大多数教材不同，统考真题中称最底层终端结点为叶结点。

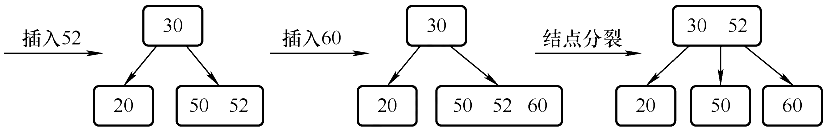
二、综合应用题

01．【解答】

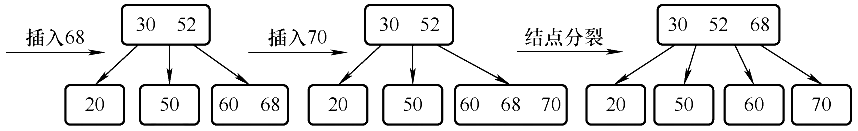
$m = 3$，因此除根结点外，非叶结点关键字个数为 1～2。



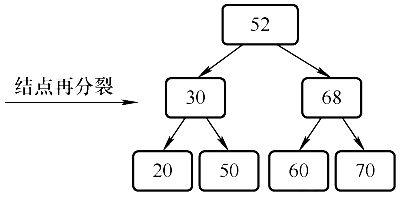
如上图所示，首先插入 20, 30，结点内关键字个数不超过 $m-1=2$，不会引起分裂；插入 50，插入 20, 30 所在的结点，引起分裂，结点内第 $\lceil m/2 \rceil$ 个关键字 30 上升为父结点。



如上图所示，插入 52，插入 50 所在的结点，不会引起分裂；继续插入 60，插入 50, 52 所在的结点，引起分裂，52 上升到父结点中，不会引起父结点的分裂。

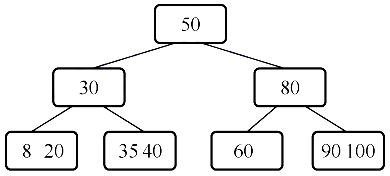


如上图所示，插入 68，插入 60 所在的结点，不会引起分裂；继续插入 70，插入 60, 68 所在的结点，引起分裂，68 上升为新的父结点，68 上升到 30, 52 所在的结点后，会继续引起该结点的分裂，所以 52 上升为新的根结点。最后得到的 B 树如下图所示。

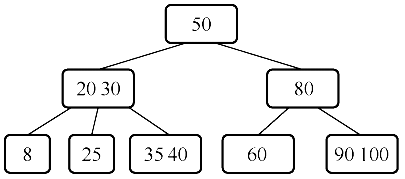


02.【解答】

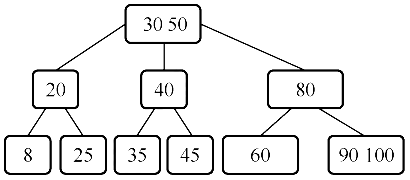
1）插入 90：将 90 插入 100 所在的结点，插入 90 后该结点中的元素个数不超过⌈3/2⌉= 2，不会引起结点的分裂，插入后的 B 树如下图所示。



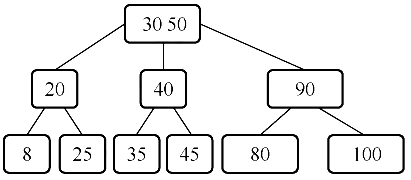
2）插入 25：将 25 插入 8, 20 所在的结点，插入后结点内的元素个数为 3，引起分裂。所以将结点内的中间元素 20 上升到父结点中，此时父结点中的元素个数为 2（元素 20 和 30），不会引起继续分裂，插入 25 后的 B 树如下图所示。



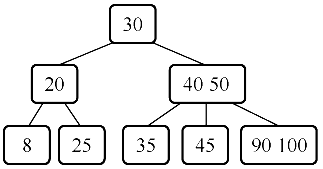
3）插入 45：将 45 插入 35, 40 所在的结点，引起分裂，中间元素 40 上升到父结点（20, 30 所在的结点）中，引起父结点分裂，中间元素 30 上升到父结点（50 所在的结点）中，两次分裂后的 B 树如下图所示。



4）删除 60：删除 60 后，其所在的结点元素为空，从而导致借用右兄弟结点的元素，调整后的 B 树如下图所示。



5）删除 80：删除 80 后，导致 80 所在结点的父结点与其右兄弟结点合并，这时父结点元素个数为 0，再次对父结点进行调整。将 50 与 40 合并成一个新结点，则 90, 100 所在结点为这个结点的子结点。从而构造的 B 树如下图所示。注意，这次调整的过程实际上包含多次调整过程，希望读者对照考点讲解中的删除过程仔细思考。



注　意　公众号：小兔网盘　免费分享无水印PDF

B 树中结点的插入、删除操作（特别是插入、删除后的结点分裂与合并）是本节的重点，也是难点，请读者务必熟练掌握。

03.【解答】

根据 B 树的概念，一个索引结点应适应操作系统一次读/写的物理记录大小，其大小应取不超过但最接近一个磁盘页块的大小。假设 B 树为 m 阶，一个 B 树结点最多存放 $m-1$ 个关键字（5B）和对应的记录地址（5B）、m 个子树指针（5B）和 1 个指示结点中的实际关键字个数的整数（2B），则有

$$(2\times(m-1)+m)\times 5+2\leqslant 4000$$

计算结果为 $m\leqslant 267$。

一个索引结点最多可以存放 $m-1=266$ 个索引项，最少可以存放 $\lceil m/2\rceil-1=133$ 个索引项。全部有 $n=20000000$ 个记录，每个记录占用空间 200B，每个页块可以存放 4000/200 = 20 个记录，则全部记录分布在 20000000/20 = 1000000 个页块中，最多需要占用 1000000/133 = 7519 个磁盘页块作为 B 树索引，最少需要占用 1000000/266 = 3760 个磁盘页块作为 B 树索引（注意 B 树与 B+树的不同，B 树所有对数据记录的索引项分布在各个层次的结点中，B+树所有对数据记录的索引项都在叶结点中）。

7.5　散列（Hash）表

7.5.1　散列表的基本概念

在前面介绍的线性表和树表的查找中，查找记录需进行一系列的关键字比较，记录在表中的位置与记录的关键字之间不存在映射关系，因此在这些表中的查找效率取决于比较的次数。

散列函数（也称哈希函数）：一个把查找表中的关键字映射成该关键字对应的地址的函数，记为 Hash(key) = Addr（这里的地址可以是数组下标、索引或内存地址等）。

散列函数可能会把两个或两个以上的不同关键字映射到同一地址，称这种情况为冲突，这些发生冲突的不同关键字称为同义词。一方面，设计得好的散列函数应尽量减少这样的冲突；另一方面，因为这样的冲突总是不可避免的，所以还要设计好处理冲突的方法。

散列表（也称*哈希表*）：根据关键字而直接进行访问的数据结构。也就是说，散列表建立了关键字和存储地址之间的一种直接映射关系。

理想情况下，对散列表进行查找的时间复杂度为 $O(1)$，即与表中元素的个数无关。下面分别介绍常用的散列函数和处理冲突的方法。

7.5.2 散列函数的构造方法

在构造散列函数时，必须注意以下几点：

1）散列函数的定义域必须包含全部关键字，而值域的范围则依赖于散列表的大小。

2）散列函数计算出的地址应尽可能均匀地分布在整个地址空间，尽可能地减少冲突。

3）散列函数应尽量简单，能在较短的时间内计算出任意一个关键字对应的散列地址。

下面介绍常用的散列函数。

1．直接定址法

直接取关键字的某个线性函数值为散列地址，散列函数为

$$H(\text{key}) = \text{key} \text{ 或 } H(\text{key}) = a \times \text{key} + b$$

式中，a 和 b 是常数。这种方法计算最简单，且不会产生冲突。它适合关键字的分布基本连续的情况，若关键字分布不连续，空位较多，则会造成存储空间的浪费。

2．除留余数法

这是一种最简单、最常用的方法，假定散列表表长为 m，取一个不大于 m 但最接近或等于 m 的质数 p，利用以下公式把关键字转换成散列地址。散列函数为

$$H(\text{key}) = \text{key} \% p$$

除留余数法的关键是选好 p，使得每个关键字通过该函数转换后等概率地映射到散列空间上的任意一个地址，从而尽可能减少冲突的可能性。

3．数字分析法

设关键字是 r 进制数（如十进制数），而 r 个数码在各位上出现的频率不一定相同，可能在某些位上分布均匀一些，每种数码出现的机会均等；而在某些位上分布不均匀，只有某几种数码经常出现，此时应选取数码分布较为均匀的若干位作为散列地址。这种方法适合于已知的关键字集合，若更换了关键字，则需要重新构造新的散列函数。

4．平方取中法

顾名思义，这种方法取关键字的平方值的中间几位作为散列地址。具体取多少位要视实际情况而定。这种方法得到的散列地址与关键字的每位都有关系，因此使得散列地址分布比较均匀，适用于关键字的每位取值都不够均匀或均小于散列地址所需的位数。

在不同的情况下，不同的散列函数具有不同的性能，因此不能笼统地说哪种散列函数最好。在实际选择中，采用何种构造散列函数的方法取决于关键字集合的情况。

7.5.3 处理冲突的方法

应该注意到，任何设计出来的散列函数都不可能绝对地避免冲突。为此，必须考虑在发生冲突时应该如何处理，即为产生冲突的关键字寻找下一个“空”的 Hash 地址。用 H_i 表示处理冲突

中第 i 次探测得到的散列地址，假设得到的另一个散列地址 H_1 仍然发生冲突，只得继续求下一个地址 H_2，以此类推，直到 H_k 不发生冲突为止，则 H_k 为关键字在表中的地址。

1．开放定址法

所谓开放定址法，是指表中可存放新表项的空闲地址既向它的同义词表项开放，又向它的非同义词表项开放。其数学递推公式为

$$H_i = (H(\text{key}) + d_i) \% m$$

式中，$H(\text{key})$为散列函数；$i = 1, 2, \cdots, k$（$k \leqslant m-1$）；m 表示散列表表长；d_i 为增量序列。

取定某一增量序列后，对应的处理方法就是确定的。通常有以下 4 种取法：

命题追踪 ▶ 堆积现象导致的结果（2014）

1）线性探测法，也称线性探测再散列法。$d_i = 1, 2, \cdots, m-1$。它的特点是：冲突发生时，顺序查看表中下一个单元（探测到表尾地址 $m-1$ 时，下一个探测地址是表首地址 0），直到找出一个空闲单元（当表未填满时一定能找到一个空闲单元）或查遍全表。

线性探测法可能使第 i 个散列地址的同义词存入第 $i+1$ 个散列地址，这样本应存入第 $i+1$ 个散列地址的元素就争夺第 $i+2$ 个散列地址的元素的地址……从而造成大量元素在相邻的散列地址上聚集（或堆积）起来，大大降低了查找效率。

2）平方探测法，也称二次探测法。$d_i = 1^2, -1^2, 2^2, -2^2, \cdots, k^2, -k^2$，其中 $k \leqslant m/2$，散列表长度 m 必须是一个可以表示成 $4k+3$ 的素数。

平方探测法是一种处理冲突的较好方法，可以避免出现“堆积”问题，它的缺点是不能探测到散列表上的所有单元，但至少能探测到一半单元。

3）双散列法。$d_i = i \times \text{Hash}_2(\text{key})$。需要使用两个散列函数，当通过第一个散列函数 $H(\text{key})$得到的地址发生冲突时，则利用第二个散列函数 $\text{Hash}_2(\text{key})$计算该关键字的地址增量。它的具体散列函数形式如下：

$$H_i = (H(\text{key}) + i \times \text{Hash}_2(\text{key})) \% m$$

初始探测位置 $H_0 = H(\text{key}) \% m$。i 是冲突的次数，初始为 0。

4）伪随机序列法。$d_i =$ 伪随机数序列。

命题追踪 ▶ 散列表中删除部分元素后的查找效率分析（2023）

注　意

采用开放定址法时，不能随便物理删除表中已有元素，否则会截断其他同义词元素的查找路径，删除元素时可以做一个删除标记，进行逻辑删除，具体举例见本书配套课程。但这样做的副作用是：执行多次删除后，表面上看起来散列表很满，实际上有许多位置未利用。

2．拉链法（链接法，chaining）

显然，对于不同的关键字可能会通过散列函数映射到同一地址，为了避免非同义词发生冲突，可以把所有的同义词存储在一个线性链表中，这个线性链表由其散列地址唯一标识。假设散列地址为 i 的同义词链表的头指针存放在散列表的第 i 个单元中，因而查找、插入和删除操作主要在同义词链中进行。拉链法适用于经常进行插入和删除的情况。

例如，关键字序列为{19, 14, 23, 01, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(\text{key}) = \text{key}\%13$，用拉链法处理冲突，建立的表如图 7.33 所示（学完下节内容后，可尝试计算本例的平均 ASL）。

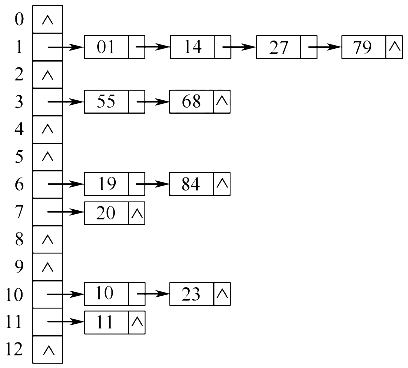


图 7.33　拉链法处理冲突的散列表

7.5.4　散列查找及性能分析的应用

命题追踪 ▶ 散列表的构造及查找效率的分析（2010、2018、2019、2024）

散列表的查找过程与构造散列表的过程基本一致。对于一个给定的关键字 key，根据散列函数可以计算出其散列地址，执行步骤如下：

初始化：Addr=Hash(key)；

① 检测查找表中地址为 Addr 的位置上是否有记录，若无记录，返回查找失败；若有记录，比较它与 key 的值，若相等，则返回查找成功标志，否则执行步骤②。

② 用给定的处理冲突方法计算“下一个散列地址”，并把 Addr 置为此地址，转入步骤①。

例如，关键字序列{19, 14, 23, 01, 68, 20, 84, 27, 55, 11, 10, 79}按散列函数 H(key)=key%13 和线性探测处理冲突构造所得的散列表 L 如图 7.34 所示。

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	14	01	68	27	55	19	20	84	79	23	11	10			

图 7.34　用线性探测法得到的散列表 L

给定值 84 的查找过程为：首先求得散列地址 H(84)=6，因 L[6]不空且 L[6]≠84，则找第一次冲突处理后的地址 H_1=(6+1)%16=7，而 L[7]不空且 L[7]≠84，则找第二次冲突处理后的地址 H_2=(6+2)%16=8，L[8]不空且 L[8]=84，查找成功，返回记录在表中的序号 8。

给定值 38 的查找过程为：先求散列地址 H(38)=12，L[12]不空且 L[12]≠38，则找下一地址 H_1=(12+1)%16=13，因为 L[13]是空记录，所以表中不存在关键字为 38 的记录。

查找各关键字的比较次数如图 7.35 所示。

关键字	14	01	68	27	55	19	20	84	79	23	11	10
比较次数	1	2	1	4	3	1	1	3	9	1	1	3

图 7.35　查找各关键字的比较次数

平均查找长度 ASL 为

$$ASL=(1\times6+2+3\times3+4+9)/12=2.5$$

对同一组关键字，设定相同的散列函数，则不同的处理冲突的方法得到的散列表不同，它们的平均查找长度也不同，本例与上节采用拉链法的平均查找长度不同。

从散列表的查找过程可见：

1）虽然散列表在关键字与记录的存储位置之间建立了直接映像，但由于“冲突”的产生，

使得散列表的查找过程仍然是一个给定值和关键字进行比较的过程，因此仍然需要以平均查找长度作为衡量散列表的查找效率的度量。

命题追踪 ▶ 影响散列表查找效率的因素（2011、2022）

2）散列表的查找效率取决于三个因素：散列函数、处理冲突的方法和装填因子。

装填因子。散列表的装填因子一般记为α，定义为一个表的装满程度，即

$$\alpha=\frac{\text{表中记录数}n}{\text{散列表长度}m}$$

散列表的平均查找长度依赖于散列表的装填因子α，而不直接依赖于 n 或 m。直观地看，α 越大，表示装填的记录越“满”，发生冲突的可能性越大；反之发生冲突的可能性越小。

读者应能在给出散列表的长度、元素个数及散列函数和解决冲突的方法后，在求出散列表的基础上计算出查找成功时的平均查找长度和查找不成功的平均查找长度。

7.5.5　本节试题精选

一、单项选择题

01. 只能在顺序存储结构上进行的查找方法是（　）。

A. 顺序查找法　　B. 折半查找法　　C. 树形查找法　　D. 散列查找法

02. 散列查找一般适用于（　）的情况下的查找。

A. 查找表为链表

B. 查找表为有序表

C. 关键字集合比地址集合大得多

D. 关键字集合与地址集合之间存在对应关系

03. 下列关于散列表的说法中，正确的是（　）。

I. 若散列表的填装因子 $\alpha<1$，则可避免碰撞的产生

II. 散列查找中不需要任何关键字的比较

III. 散列表在查找成功时平均查找长度仅与表长有关

IV. 若在散列表中删除一个元素，不能简单地将该元素删除

A. I 和 IV　　B. II 和 III　　C. III　　D. IV

04. 在开放定址法中散列到同一个地址而引起的“堆积”问题是由（　）引起的。

A. 同义词之间发生冲突

B. 非同义词之间发生冲突

C. 同义词之间或非同义词之间发生冲突

D. 散列表“溢出”

05. 下列关于散列冲突处理方法的说法中，正确的有（　）。

I. 采用平方探测法处理冲突时不易产生聚集

II. 采用线性探测法处理冲突时，所有同义词在散列表中一定相邻

III. 采用链地址法处理冲突时，若限定在链首插入，则插入任意一个元素的时间相同

IV. 采用链地址法处理冲突易引起聚集现象

A. I 和 III　　B. I、II 和 III　　C. III 和 IV　　D. I 和 IV

06. 设有一个含有 200 个元素的散列表，用线性探测法解决冲突，按关键字查询时找到一个表项的平均探测次数不超过 1.5，则散列表应能够容纳（　）个元素（设查找成功的平均查找长度为 $ASL=[1+1/(1-\alpha)]/2$，其中α为装填因子）。

A. 400 B. 526 C. 624 D. 676

07. 假定有 K 个关键字互为同义词，若用线性探测法把这 K 个关键字填入散列表，至少要进行（ ）次探测。

A. $K-1$ B. K C. $K+1$ D. $K(K+1)/2$

08. 对包含 n 个元素的散列表进行查找，平均查找长度（ ）。

A. 为 $O(\log_2 n)$ B. 为 $O(1)$ C. 不直接依赖于 n D. 直接依赖于表长 m

09. 采用开放定址法解决冲突的散列查找中，发生聚集的原因主要是（ ）。

A. 数据元素过多 B. 负载因子过大

C. 散列函数选择不当 D. 解决冲突的方法选择不当

10. 当用线性探测再散列法解决冲突时，计算出的一系列“下一个空位”的要求是（ ）。

A. 必须大于或等于原散列地址 B. 必须小于或等于原散列地址

C. 可以大于或小于但不等于原散列地址 D. 对地址在何处没有限制

11. 一组记录的关键字为{19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，用链地址法构造散列表，散列函数为 H(key)=key mod 13，散列地址为 1 的链中有（ ）个记录。

A. 1 B. 2 C. 3 D. 4

12. 在采用链地址法处理冲突所构成的散列表上查找某一关键字，则在查找成功的情况下，所探测的这些位置上的关键字值（ ）；若采用线性探测法，则（ ）。

A. 一定都是同义词 B. 不一定都是同义词

C. 都相同 D. 一定都不是同义词

13. 若采用链地址法构造散列表，散列函数为 H(key)=key mod 17，则需（①）个链表。这些链的链首指针构成一个指针数组，数组的下标范围为（②）。

①A. 17 B. 13 C. 16 D. 任意

②A. 0 ~ 17 B. 1 ~ 17 C. 0 ~ 16 D. 1 ~ 16

14. 设散列表长 $m=14$，散列函数为 H(key)=key%11，表中仅有 4 个结点 H(15)=4，H(38)=5，H(61)=6，H(84)=7，若采用线性探测法处理冲突，则关键字为 49 的结点地址是（ ）。

A. 8 B. 3 C. 5 D. 9

15. 现有长度为 17、初始为空的散列表 HT，散列函数 H(key)=key%17，用线性探查法解决冲突。将关键字序列 26, 25, 72, 38, 8, 18, 59 依次插入 HT 后，则查找 59 需探查（ ）次。

A. 2 B. 3 C. 4 D. 5

16. 现有长度为 17、初始为空的散列表 HT，散列函数 H(key) = key%17，用平方探测法解决冲突：H_i(key)=(H(key)± i^2)%17。将关键字序列 6, 22, 7, 26, 9, 23 依次插入 HT 后，则关键字 23 存放在散列表中的位置是（ ）。

A. 0 B. 2 C. 6 D. 15

17. 将 10 个元素散列到 100000 个单元的散列表中，则（ ）产生冲突。

A. 一定会 B. 一定不会 C. 仍可能会 D. 不确定

18. 【2011 统考真题】为提高散列表的查找效率，可以采取的正确措施是（ ）。

I. 增大装填（载）因子

II. 设计冲突（碰撞）少的散列函数

III. 处理冲突（碰撞）时避免产生聚集（堆积）现象

A. 仅 I B. 仅 II C. 仅 I、II D. 仅 II、III

19.【2014 统考真题】用哈希（散列）方法处理冲突（碰撞）时可能出现堆积（聚集）现象，下列选项中，会受堆积现象直接影响的是（　）。

A. 存储效率　　B. 散列函数

C. 装填（装载）因子　　D. 平均查找长度

20.【2018 统考真题】现有长度为 7、初始为空的散列表 HT，散列函数 H(k)=k%7，用线性探测再散列法解决冲突。将关键字 22, 43, 15 依次插入 HT 后，查找成功的平均查找长度是（　）。

A. 1.5　　B. 1.6　　C. 2　　D. 3

21.【2019 统考真题】现有长度为 11 且初始为空的散列表 HT，散列函数是 H(key)=key%7，采用线性探查（线性探测再散列）法解决冲突。将关键字序列 87, 40, 30, 6, 11, 22, 98, 20 依次插入 HT 后，HT 查找失败的平均查找长度是（　）。

A. 4　　B. 5.25　　C. 6　　D. 6.29

22.【2022 统考真题】下列因素中，影响散列（哈希）方法平均查找长度的是（　）。

I. 装填因子　　II. 散列函数　　III. 冲突解决策略

A. 仅 I、II　　B. 仅 I、III　　C. 仅 II、III　　D. I、II、III

23.【2023 统考真题】现有长度为 5、初始为空的散列表 HT，散列函数 H(k)=(k+4)%5，用线性探查再散列法解决冲突。若将关键字序列 2022,12,25 依次插入 HT，然后删除关键字 25，则 HT 中查找失败的平均查找长度为（　）。

A. 1　　B. 1.6　　C. 1.8　　D. 2.2

二、综合应用题

01. 若要在散列表中删除一个记录，应如何操作？为什么？按照处理冲突的方法为开放地址法和拉链法分别说明。

02. 假定把关键字 key 散列到有 n 个表项（从 0 到 $n-1$ 编址）的散列表中。对于下面的每个函数 H(key)（key 为整数），这些函数能够当作散列函数吗？若能，它是一个好的散列函数吗？说明理由。设函数 random(n)返回一个 0 到 $n-1$ 之间的随机整数(包括 0 与 $n-1$ 在内）。

1）H(key)=key/n。

2）H(key)=1。

3）H(key)=(key+random(n))%n。

4）H(key)=key%p(n)；其中 p(n)是不大于 n 的最大素数。

03. 使用散列函数 H(key)=key%11，把一个整数值转换成散列表下标，散列表的长度为 11，现在要把数据{1, 13, 12, 34, 38, 33, 27, 22}依次插入散列表。

1）使用线性探测法来构造散列表。

2）使用链地址法构造散列表。

试针对这两种情况，分别确定查找成功所需的平均查找长度，及查找不成功所需的平均查找长度。

04. 已知一组关键字为{26, 36, 41, 38, 44, 15, 68, 12, 6, 51, 25}，用链地址法解决冲突，假设装填因子 $\alpha=0.73$，散列函数的形式为 H(key)=key%P，P 为不大于表长的最大素数，请回答以下问题：

1）构造出散列函数。

2）分别计算出等概率情况下查找成功和查找失败的平均查找长度（查找失败的计算中只将与关键字的比较次数计算在内即可）。

05. 设散列表为 HT[0…12]，即表的大小为 $m=13$。现采用双散列法解决冲突，散列函数和再散列函数分别为：

H_0(key)=key%13　　注：%是取模运算（=mod）

$H_i=(H_{i-1}+REV(key+1)\%11+1)\%13$；　$i=1,2,3,\cdots,m-1$

其中，函数 REV(x)表示颠倒十进制数 x 的各位，如 REV(37)=73，REV(7)=7 等。若插入的关键码序列为(2, 8, 31, 20, 19, 18, 53, 27)，请回答：

1）画出插入这 8 个关键码后的散列表。

2）计算查找成功的平均查找长度 ASL。

06.【2010 统考真题】将关键字序列(7, 8, 30, 11, 18, 9, 14)散列存储到散列表中。散列表的存储空间是一个下标从 0 开始的一维数组，散列函数为 H(key)=(key×3) mod 7，处理冲突采用线性探测再散列法，要求装填（载）因子为 0.7。

1）请画出所构造的散列表。

2）分别计算等概率情况下，查找成功和查找不成功的平均查找长度。

07.【2024 统考真题】将关键字序列 20, 3, 11, 18, 9, 14, 7 依次存储到初始为空、长度为 11 的散列表 HT 中，散列函数 H(key)=(key×3)%11。H(key)计算出的初始散列地址为 H_0，发生冲突时探查地址序列是 $H_1, H_2, H_3,\cdots$，其中，$H_k=(H_0+k^2)\%11$，$k=1,2,3,\cdots$。

请回答下列问题：

1）画出所构造的 HT，并计算 HT 的装填因子。

2）给出在 HT 中查找关键字 14 的关键字比较序列。

3）在 HT 中查找关键字 8，确认查找失败时的散列地址是多少？

7.5.6 答案与解析

一、单项选择题

01. B

顺序查找可以是顺序存储或链式存储；折半查找只能是顺序存储且要求关键字有序；树形查找法要求采用树的存储结构，既可以采用顺序存储也可以采用链式存储；散列查找中的链地址法解决冲突时，采用的是顺序存储与链式存储相结合的方式。

02. D

关键字集合与地址集合之间存在对应关系时，通过散列函数表示这种关系。这样，查找以计算散列函数而非比较的方式进行查找。

03. D

冲突（碰撞）是不可避免的，与装填因子无关，因此需要设计处理冲突的方法，选项 I 错误。散列查找的思想是计算出散列地址来进行查找，然后比较关键字以确定是否查找成功，选项 II 错误。散列查找成功的平均查找长度与装填因子有关，与表长无直接关系，选项 III 错误。在开放定址的情形下，不能随便删除散列表中的某个元素，否则可能会导致搜索路径被中断（因此通常的做法是在要删除的地方做删除标记，而不是直接删除），选项 IV 正确。

04. C

在开放定址法中散列到同一个地址而产生的“堆积”问题，是同义词冲突的探查序列和非同义词之间不同的探查序列交织在一起，导致关键字查询需要经过较长的探测距离，降低了散列的效率。因此要选择好的处理冲突的方法来避免“堆积”。

05. A

平方探测法采用的增量序列是非线性的，它可以跳过一些已被占用的单元，而不是顺序地探测下一单元，这样能减小冲突的概率，选项 I 正确。散列地址 i 的关键字，和为解决冲突形成的某次探测地址为 i 的关键字，都争夺地址 $i, i+1, \cdots$，因此不一定相邻，选项 II 错误。选项 III 正确。同义词冲突不等于聚集，链地址法处理冲突时将同义词放在同一个链表中，不会引起聚集现象，选项 IV 错误。

06．A

若有 200 个元素要放入散列表，采用线性探测法解决冲突，限定查找成功的平均查找长度不超过 1.5，则

$$\mathrm{ASL}_{\text{成功}}=\frac{1}{2}\left(1+\frac{1}{1-\alpha}\right)\leqslant 1.5 \Rightarrow \alpha=\frac{200}{m}\leqslant\frac{1}{2}\Rightarrow m\geqslant 400$$

07．D

K 个关键字在依次填入的过程中，只有第一个不会发生冲突，所以探测次数为 $1+2+3+\cdots+K=K(K+1)/2$。

08．C

散列表的平均查找长度与装填因子 α 直接相关，表的查找效率不直接依赖于表中已有表项个数 n 或表长 m。若表中存放的记录全是某个地址的同义词，则平均查找长度为 $O(n)$而非 $O(1)$。

09．D

聚集是因选取不当的处理冲突的方法，而导致不同关键字的元素对同一散列地址进行争夺的现象。用线性探查法时，容易引发聚集现象。

10．C

“下一个空位”可以大于或小于但不等于原散列地址，等于原散列地址是没有意义的。

11．D

由散列函数计算可知，14, 1, 27, 79 散列后的地址都是 1，所以有 4 个记录。

12．A，B

因为在链地址法中，映射到同一地址的关键字都会链到与此地址相对应的链表上，所以探测过程一定是在此链表上进行的，从而这些位置上的关键字均为同义词；但在线性探测法中出现两个同义关键字时，会把该关键字对应地址的下一个地址也占用掉，两个地址分别记为 Addr、Addr+1，查找一个满足 H(key)=Addr+1 的关键字 key 时，显然首次探测到的不是 key 的同义词。

13．A，C

H 的取值有 17 种可能，对应到不同的链表中，所以链表的个数应为 17。因为 H(key)的取值范围是 0～16，所以数组下标为 0～16。

14．A

线性探测法的公式为 H_i=(H(key)+d_i)%m，其中 $d_i=1,2,\cdots,m-1$。H(49)=49%11=5，有冲突；H_1=(H(49)+1)%14=6，有冲突；H_2=(H(49)+2)%14=7，有冲突；H_3=(H(49)+3)%14=8，无冲突。

15．C

插入过程如下：H(26)=9，不冲突；H(25)=8，不冲突；H(72)=4，不冲突；H(38)=4，冲突，冲突处理后的地址为 5；H(8)=8，冲突，冲突处理后的地址为 10；H(18)=1，不冲突；H(59)=8，冲突，冲突处理后的地址为 11。因此，在表中查找 59 需要探查 4 次。

16．B

插入过程如下：6%17=6；22%17=5；7%17=7；26%17=9；9%17=9，冲突，平方探测法探测 10（无冲突）；23%17=6，冲突，平方探测法探测 7（冲突），探测 5（冲突），探测 10（冲突），探测 2（无冲突）。因此，关键字 23 应放在位置 2。构造的散列表如下表所示。

地址	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
元素			23			22	6	7		26	9						

17．C

由于散列函数的选取，仍然有可能产生地址冲突，冲突不能绝对地避免。

18．D

散列表的查找效率取决于散列函数、处理冲突的方法和装填因子。显然，冲突的产生概率与装填因子（表中记录数与表长之比）的大小成正比，选项 I 与题意相反。选项 II 显然正确。采用合适的冲突处理方法可避免聚集现象，也将提高查找效率，选项 III 正确。例如，用链地址法处理冲突时不存在聚集现象，用线性探测法处理冲突时易引起聚集现象。

19．D

堆积现象因冲突而产生，它对存储效率、散列函数和装填因子均不会有影响，而平均查找长度会因为堆积现象而增大。散列函数是指将关键字映射到哈希地址的函数。存储效率和装填（装载）因子的定义相同，指哈希表中已存储的元素个数与哈希表长度的比值。这些因素都与堆积现象无关，而只与哈希表的结构和设计有关。

20．C

根据题意，得到的 HT 如下：

0	1	2	3	4	5	6
	22	43	15			

$ASL_{成功}=(1+2+3)/3=2$。

21．C

采用线性探查法计算每个关键字的存放情况如下表所示。

散列地址	0	1	2	3	4	5	6	7	8	9	10
关键字	98	22	30	87	11	40	6	20			

由于 H(key)=0～6，查找失败时可能对应的地址有 7 个，对于计算出地址为 0 的关键字 key0，只有比较完 0～8 号地址后才能确定该关键字不在表中，比较次数为 9；对于计算出地址为 1 的关键字 key1，只有比较完 1～8 号地址后才能确定该关键字不在表中，比较次数为 8；以此类推。需要特别注意的是，散列函数不可能计算出地址 7，因此有

$$ASL_{失败}=(9+8+7+6+5+4+3)/7=6$$

22．D

原题再现。填装因子越大，说明哈希表中存储的元素越满，发生冲突的可能性就越高，导致平均查找长度越大。散列函数、冲突解决策略也会影响发生冲突的可能性。选项 I、II、III 都正确。

23．C

当采用开放定址法时，不能随便物理删除表中的已有元素，因为若删除元素，则可能截断其他具有相同散列地址的元素的查找地址。因此，当要删除一个元素时，可给它做一个删除标记。依次将 2022, 12, 25 插入散列表，然后删除 25，得到的散列表如下：

地址	0	1	2	3	4
关键字		2022	12		25（删除）
查找失败次数	1	3	2	1	2

当查找位置是删除标记时，应继续往后查找。

查找失败的平均查找长度为(1 + 3 + 2 + 1 + 2)/5 = 1.8。

二、综合应用题

01.【解答】

在散列表中删除一个记录，在拉链法情况下可以物理地删除。但在开放定址法情况下，不能物理地删除，只能做删除标记。该地址可能是该记录的同义词查找路径上的地址，物理地删除就中断了查找路径，因为查找时碰到空地址就认为是查找失败。

02.【解答】

1）不能作为散列函数，因为 key/*n* 可能大于 *n*，这样就无法找到适合的位置。

2）能够作为散列函数，但不是一个好的散列函数，因为所有关键字都映射到同一位置，造成大量的冲突机会。

3）不能当作散列函数，因为该函数的返回值不确定，这样无法进行正常的查找。

4）能够作为散列函数，是一个好的散列函数。

03.【解答】

由散列函数可知散列地址的范围为 0～10。

采用线性探测法构造散列表时，首先应计算出关键字对应的散列地址，然后检查散列表中对应的地址是否已经有元素。若没有元素，则直接将该关键字放入散列表对应的地址中；若有元素，则采用线性探测的方法查找下一个地址，从而决定该关键字的存放位置。

采用链地址法构造散列表时，在直接计算出关键字对应的散列地址后，将关键字结点插入此散列地址所在的链表。

具体解答如下。

1）线性探测法。

H(1)=1，无冲突，地址 1 存放关键字 1。H(13)=2，无冲突，地址 2 存放关键字 13。H(12)=1，发生冲突，根据线性探测法：H_1=2，发生冲突，继续探测 H_2=3，无冲突，于是 12 存放在地址为 3 的表项中。H(34)=1，发生冲突，根据线性探测法：H_1=2，发生冲突，H_2=3，发生冲突，H_3=4，没有冲突，于是 34 存放在地址为 4 的表项中。

同理，可以计算其他的数据存放情况，最后结果如下表所示。

散列地址	0	1	2	3	4	5	6	7	8	9	10
关键字	33	1	13	12	34	38	27	22			
冲突次数	0	0	0	2	3	0	1	7			

下面计算平均查找长度：

查找成功时，显然查找每个元素的概率都是 1/8。对于 33，因为冲突次数为 0，所以仅需 1 次比较便可查找成功；对于 22，因为计算出的地址为 0，但需要 8 次比较才能查找成功，所以 22 的查找长度为 8；其他元素的分析类似。因此有

$$ASL_{成功} = (1 + 1 + 1 + 3 + 4 + 1 + 2 + 8)/8 = 21/8$$

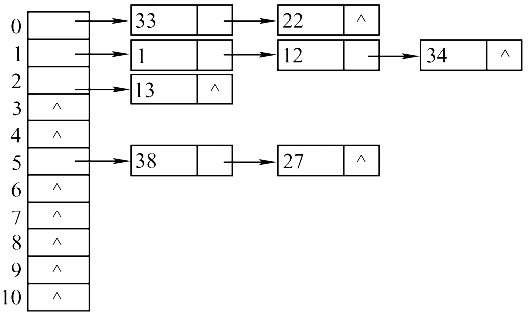
查找失败时，H(key)=0～10，因此对每个位置查找的概率都是 1/11，对于计算出的地址为 0 的关键字 key0，只有探测完 0～8 号地址后才能确定该元素不在表中，比较次数为 9；对于计算出的地址为 1 的关键字 key1，只有探测完 1～8 号地址后，才能确定该元素不在表中，比较次数为 8，以此类推。而对于计算出的地址为 8, 9, 10 的关键字，这些单元中没有存放元素，所以只需比较 1 次便可确定查找失败，因此有

$$ASL_{失败} = (9+8+7+6+5+4+3+2+1+1+1)/11 = 47/11$$

2）链地址法构造的表如下：

在链地址表中查找成功时，查找关键字为 33 的记录需进行 1 次比较，查找关键字为 22 的记录需进行 2 次比较，以此类推。因此有

$$ASL_{成功} = (1\times4+2\times3+3)/8 = 13/8$$



查找失败时，对于地址 0，比较 3 次后确定元素不在表中（空指针算 1 次），所以其查找长度为 3；对于地址 1，其查找长度为 4；对于地址 2，查找长度为 2；以此类推。因此有

$$ASL_{失败} = (3+4+2+1+1+3+1+1+1+1+1)/11 = 19/11$$

注 意

求查找失败的平均查找长度时有两种观点：其一，认为比较到空结点才算失败，所以比较次数等于冲突次数加 1；其二，认为只有与关键字的比较才算比较次数。

04.【解答】

由装填因子的计算公式$\alpha = n/N$（n 为关键字个数，N 为表长），不难得出表长，而根据散列函数的选择要求，P 应该取不大于表长的最大素数，从而可以确定 P 的大小，也就构造出了散列函数。采用链地址法解决冲突。具体解答如下。

1）由$\alpha = n/N$ 得 $N = n/\alpha$，即 $N = n/\alpha \approx 15$，从而 $P = 13$。因此散列函数为 `H(key)=key%13`。

2）由 1）求出的散列函数，计算各关键字对应的散列地址如下表所示。

关键字	26	36	41	38	44	15	68	12	6	51	25
散列地址	0	10	2	12	5	2	3	12	6	12	12

由此构造的链地址法处理冲突的散列表为